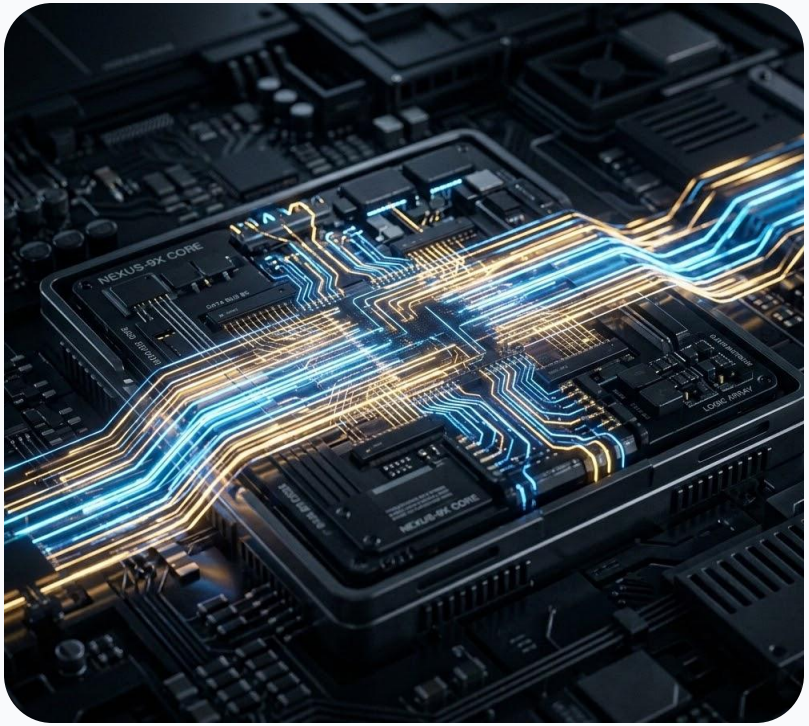
Arquitectura Horizontal

Ejecución del microprograma

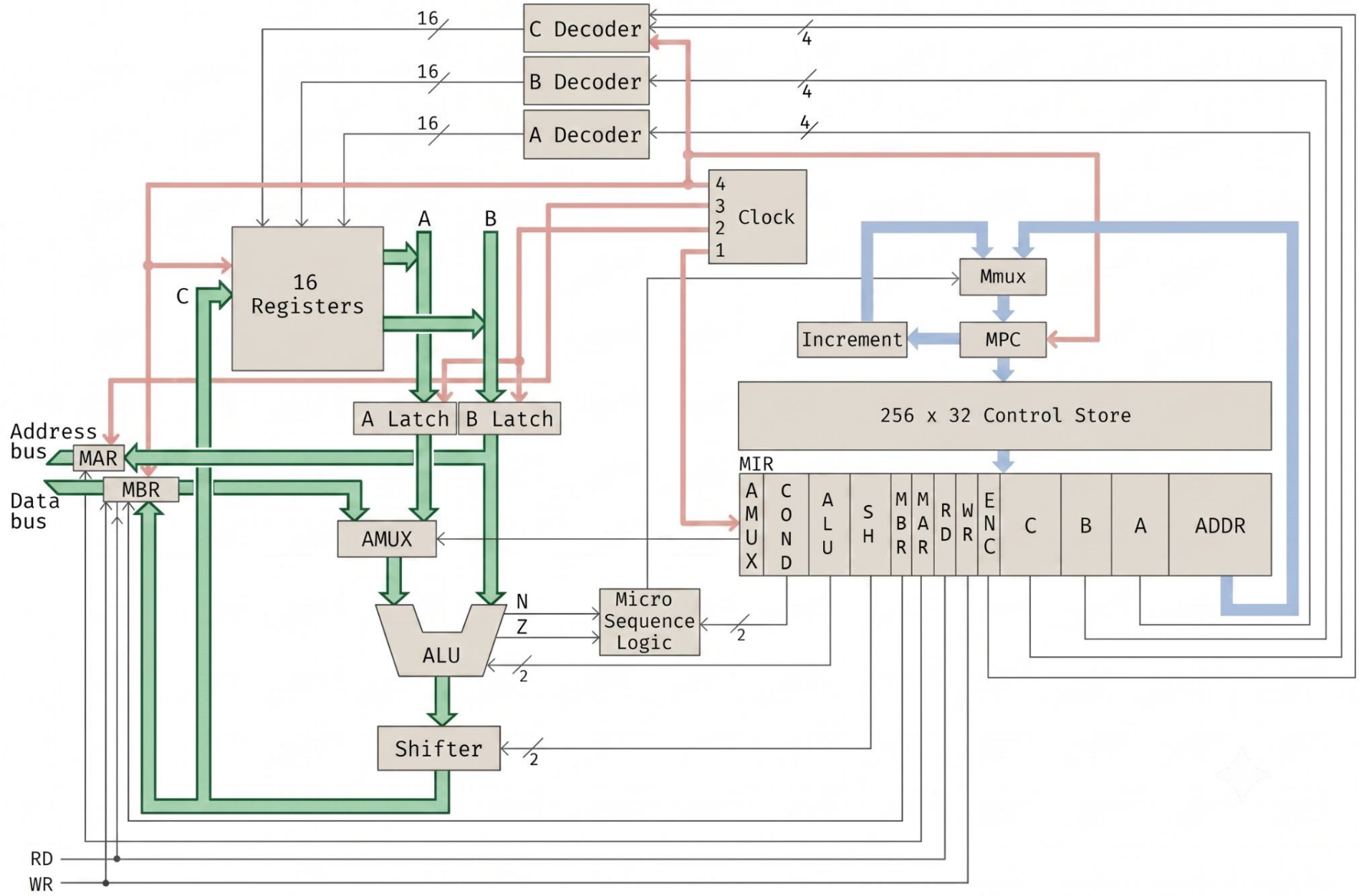
Fundamentos de Arquitectura de computadoras

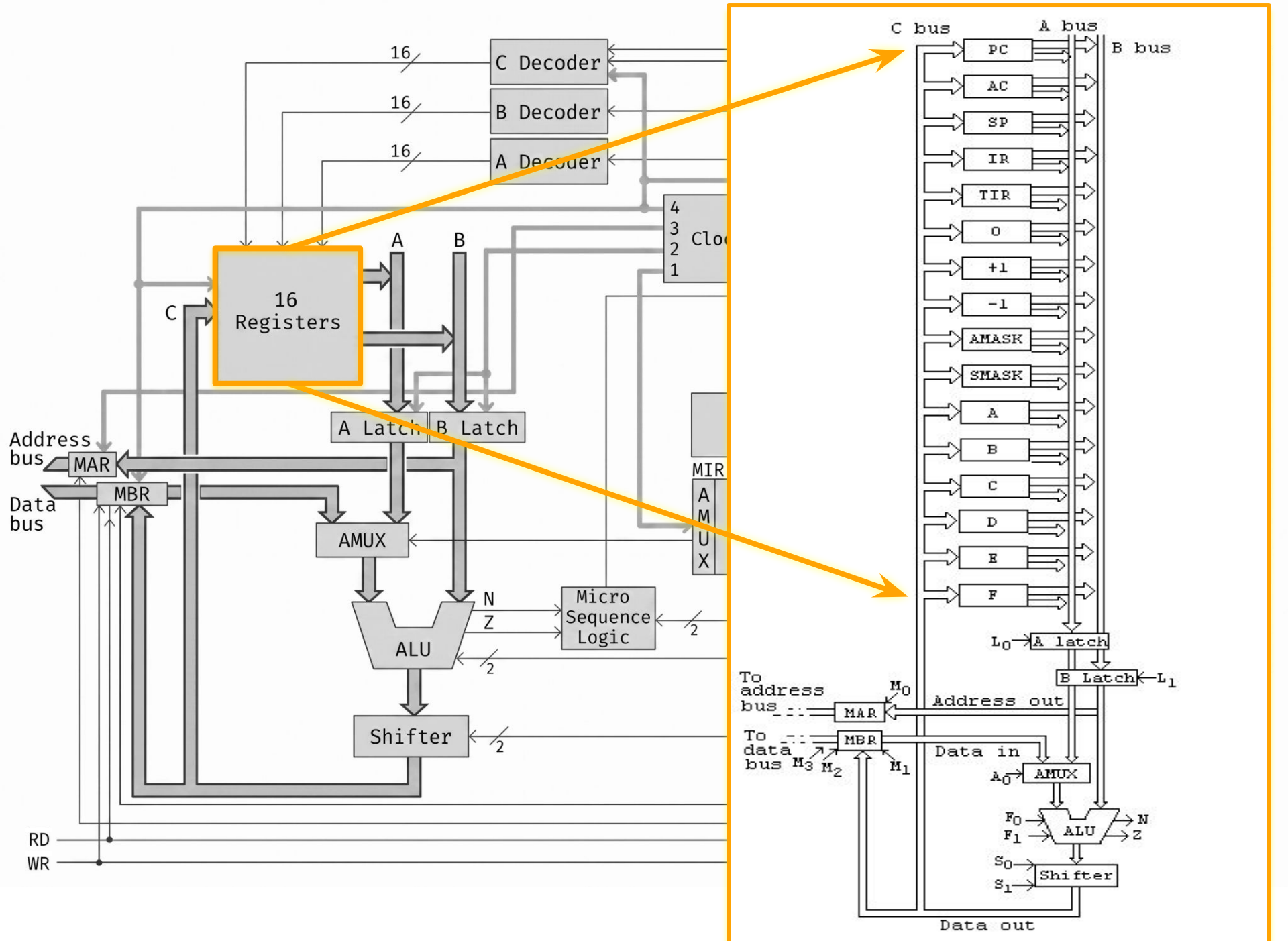
El control del procesador. Análisis de la secuencias de pasos del micro programa para la ejecución de un programa de usuario.

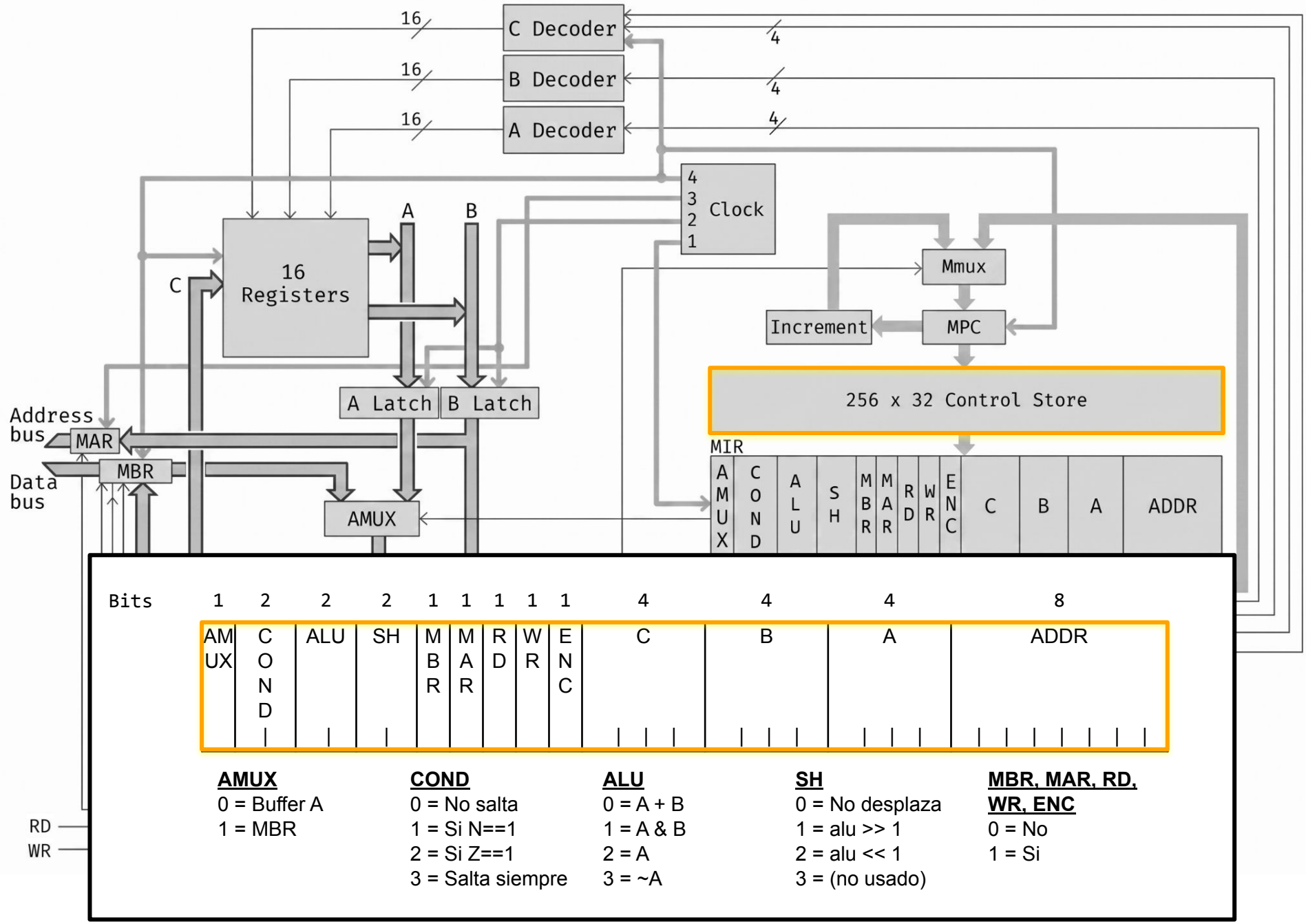
- Control Store
- Micro instrucción
- Microprograma

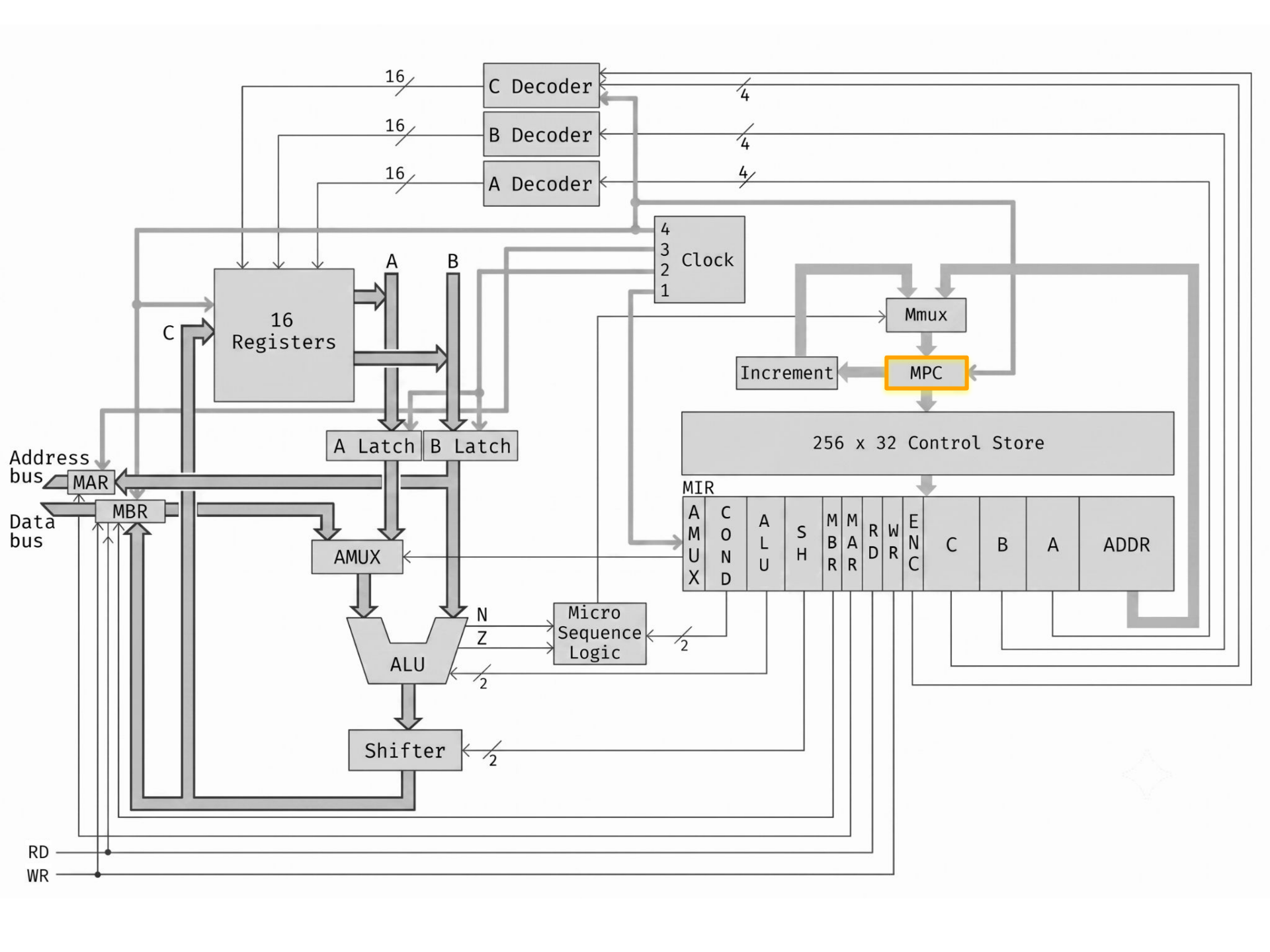


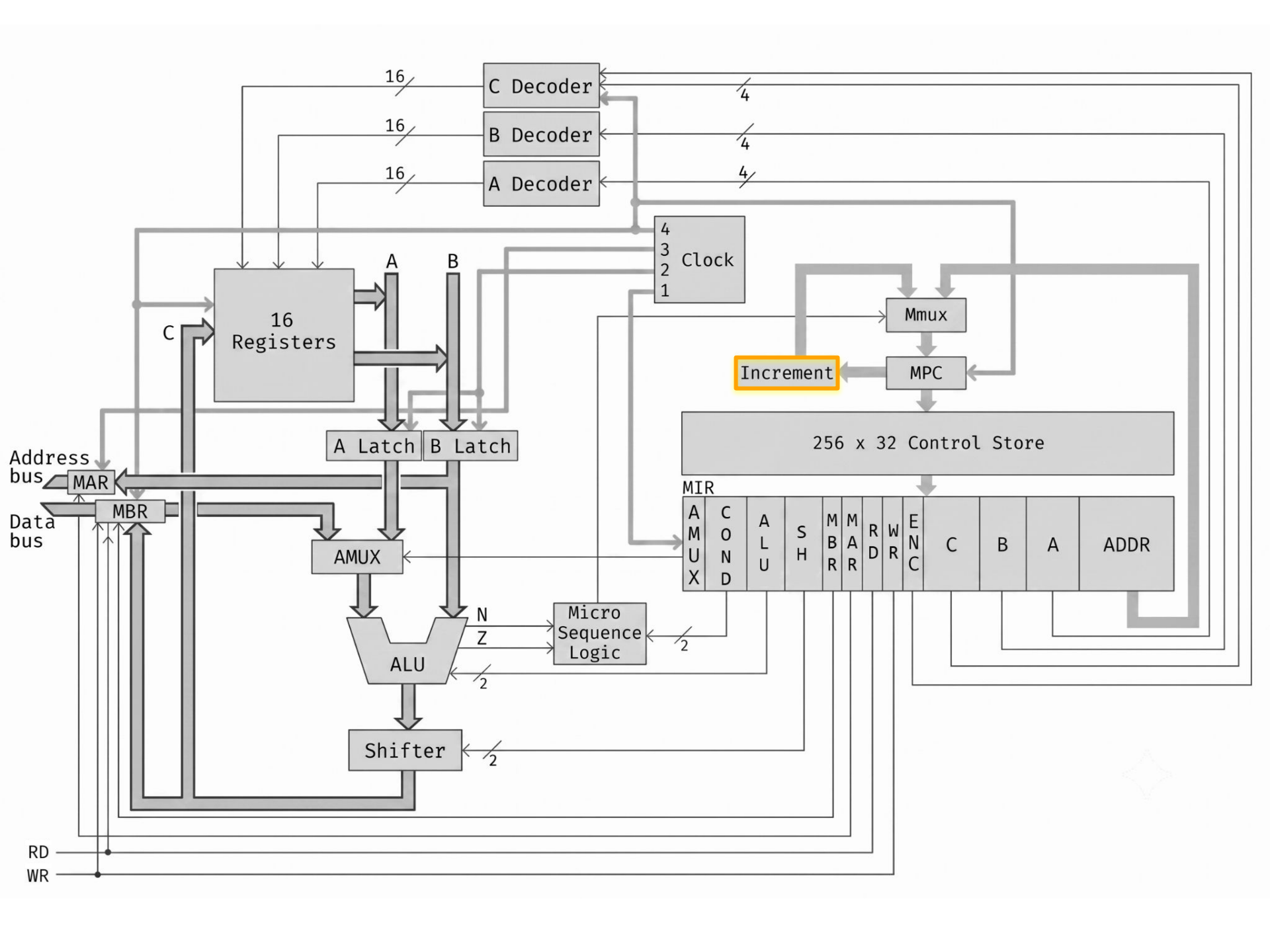
Máquina multinivel

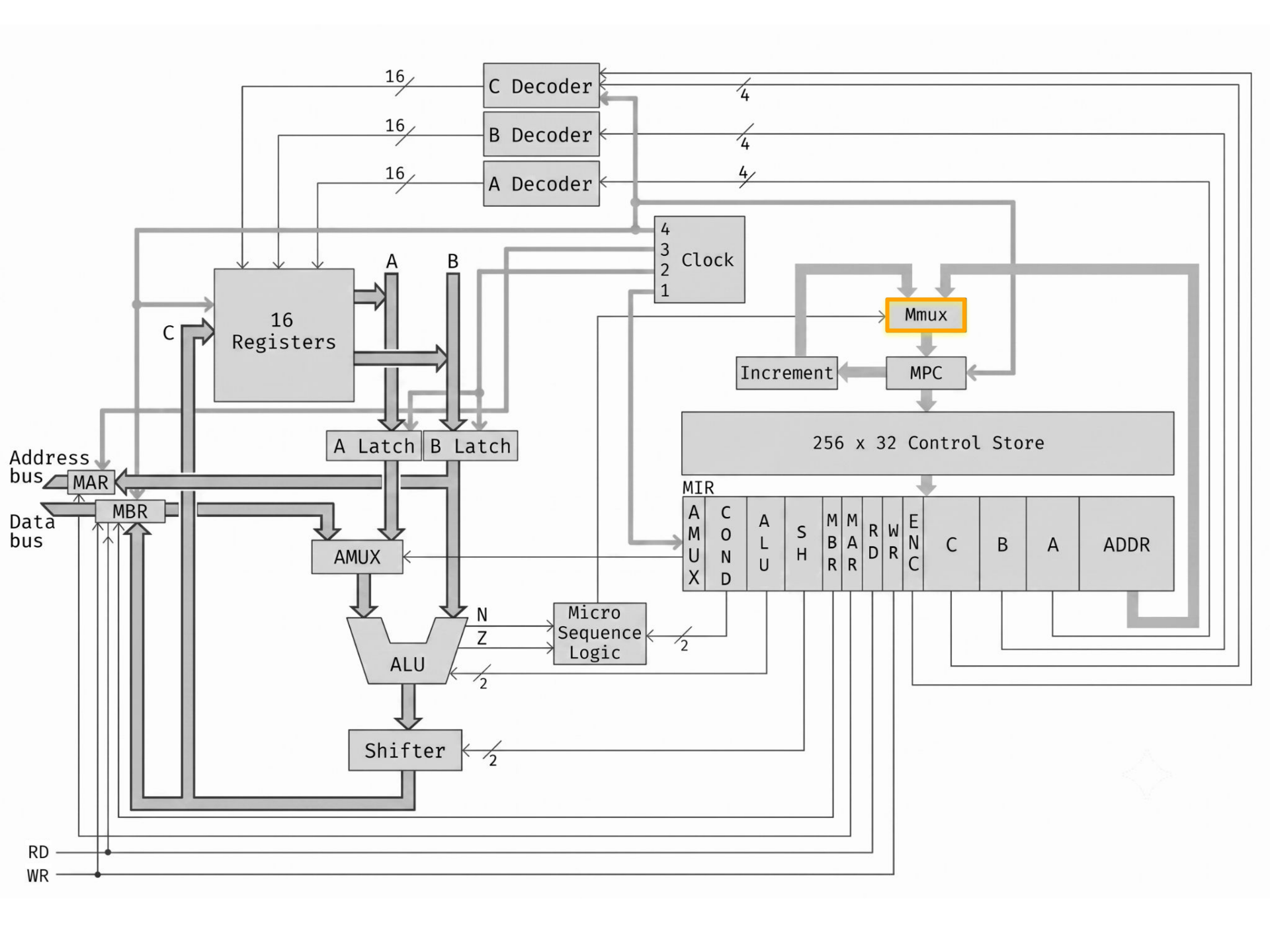

16
C Decoder
16
B Decoder
16
A Decoder
4
4
4
4
3
2
1
Clock
A
B
C
16
Registers
Mmux
Increment
MPC
256 x 32 Control Store
MIR
A Latch
B Latch
Address bus
MAR
MBR
Data bus
AMUX
AMUX
COND
ALU
SH
MBR
MAR
RD
WR
ENC
C
B
A
ADDR
N
Z
Micro Sequence Logic
ALU
2
2
Shifter
2
RD
WR

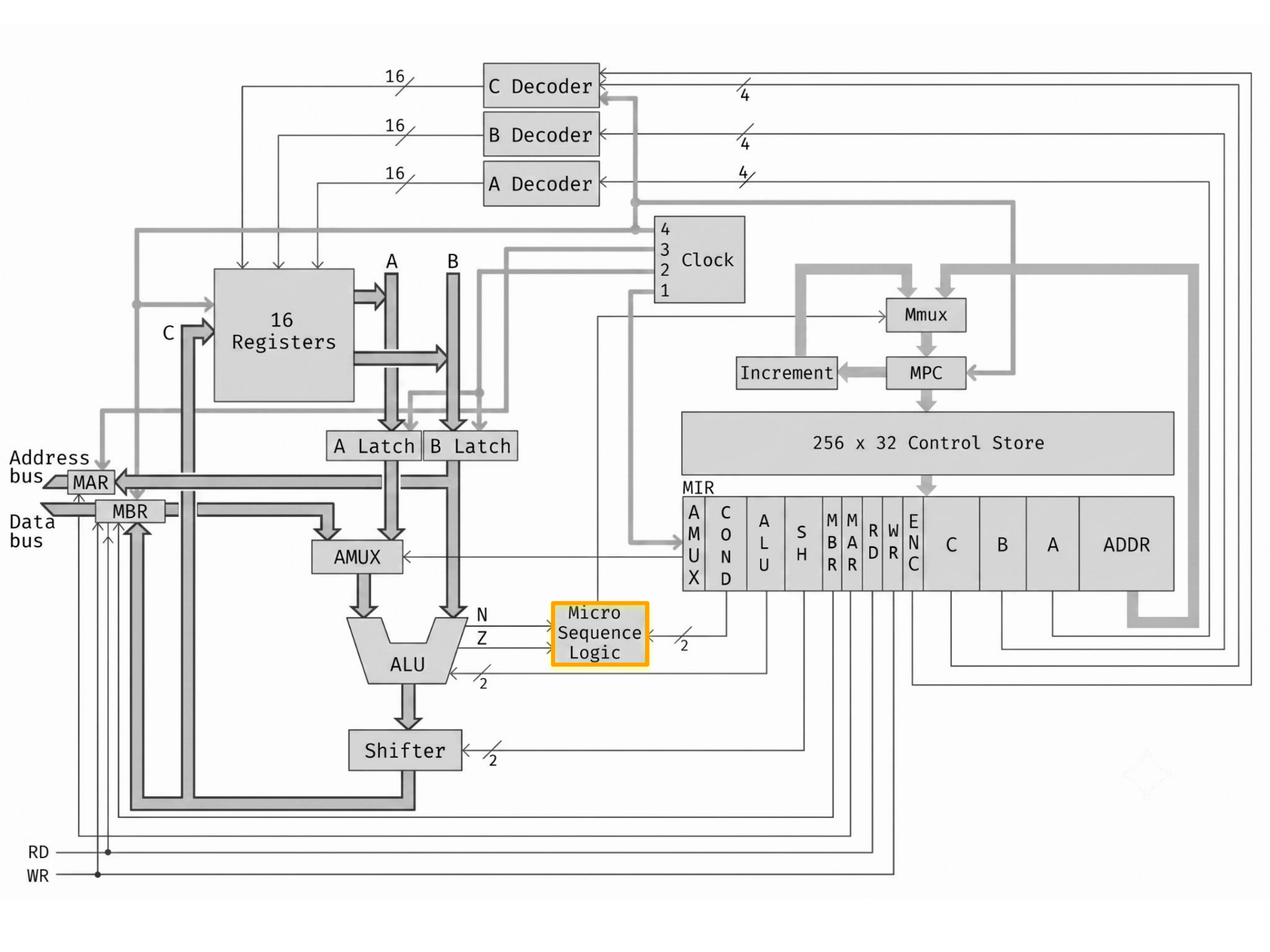

16
C Decoder
16
B Decoder
16
A Decoder
4
3
2
1
Clo
16
Registers
C
A
B
A Latch
B Latch
Address
bus
MAR
MBR
Data
bus
AMUX
MIR
A
M
U
X
N
Z
Micro
Sequence
Logic
ALU
2
2
Shifter
2
RD
WR
C bus
A bus
B bus
PC
AC
SP
IR
TIR
0
+1
-1
AMASK
SMASK
A
B
C
D
E
F
L_0
A latch
B Latch
L_1
To
address
bus
M_0
MAR
Address out
To
data
bus
MBR
M_3
M_2
M_1
Data in
A_0
AMUX
F_0
F_1
ALU
N
Z
S_0
S_1
Shifter
Data out


16
C Decoder
16
B Decoder
16
A Decoder
4
4
4
4
3
2
1
Clock
A
B
C
16
Registers
Mmux
Increment
MPC
256 x 32 Control Store
A Latch
B Latch
Address
bus
MAR
MBR
Data
bus
AMUX
MIR
AMUX
COND
ALU
SH
MBR
MAR
RD
WR
ENC
C
B
A
ADDR
RD
WR
Bits 1 2 2 2 1 1 1 1 1 4 4 4 8
AMUX
COND
ALU
SH
MBR
MAR
RD
WR
ENC
C
B
A
ADDR
AMUX
0 = Buffer A
1 = MBR
COND
0 = No salta
1 = Si N==1
2 = Si Z==1
3 = Salta siempre
ALU
0 = A + B
1 = A & B
2 = A
3 = ~A
SH
0 = No desplaza
1 = alu >> 1
2 = alu << 1
3 = (no usado)
MBR, MAR, RD, WR, ENC
0 = No
1 = Si


16
C Decoder
4
16
B Decoder
4
16
A Decoder
4
4
3
2
1
Clock
A
B
C
16
Registers
Mmux
Increment
MPC
256 x 32 Control Store
MIR
A Latch
B Latch
Address bus
MAR
MBR
Data bus
AMUX
AMUX
COND
ALU
SH
MBR
MAR
RD
WR
ENC
C
B
A
ADDR
N
Z
Micro Sequence Logic
2
ALU
2
Shifter
2
RD
WR


16
C Decoder
16
B Decoder
16
A Decoder
4
4
4
4
3
2
1
Clock
A
B
C
16
Registers
Mmux
Increment
MPC
256 x 32 Control Store
MIR
A Latch
B Latch
Address bus
MAR
MBR
Data bus
AMUX
ALU
N
Z
Micro Sequence Logic
2
2
Shifter
2
AMUX
COND
ALU
SH
MBR
MAR
RD
WR
ENC
C
B
A
ADDR
RD
WR


16
C Decoder
16
B Decoder
16
A Decoder
4
4
4
4
3
2
1
Clock
A
B
C
16
Registers
Mmux
Increment
MPC
256 x 32 Control Store
MIR
A Latch
B Latch
Address bus
MAR
MBR
Data bus
AMUX
AMUX
COND
ALU
SH
MBR
MAR
RD
WR
ENC
C
B
A
ADDR
N
Z
Micro Sequence Logic
2
ALU
2
Shifter
2
RD
WR


16
C Decoder
16
B Decoder
16
A Decoder
4
4
4
4
3
2
1
Clock
A
B
C
16
Registers
Mmux
Increment
MPC
256 x 32 Control Store
MIR
A Latch
B Latch
Address bus
MAR
MBR
Data bus
AMUX
AMUX
COND
ALU
SH
MBR
MAR
RD
WR
ENC
C
B
A
ADDR
N
Z
Micro Sequence Logic
2
ALU
2
Shifter
2
RD
WR

Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxxxxxxxxxx	ADDD	Add direct	ac:=ac+m[x]
0011xxxxxxxxxxxx	SUBD	Subtract direct	ac:=ac-m[x]
0100xxxxxxxxxxxx	JPOS	Jump positive	if ac>=0 then pc:=x
0101xxxxxxxxxxxx	JZER	Jump zero	if ac=0 then pc:=x
0110xxxxxxxxxxxx	JUMP	Jump	pc:=x
0111xxxxxxxxxxxx	LOCO	Load constant	ac:=x (0<=x<=4095)
1000xxxxxxxxxxxx	LODL	Load local	ac:=m[sp+x]
1001xxxxxxxxxxxx	STOL	Store local	m[x+sp]:=ac
1010xxxxxxxxxxxx	ADDL	Add local	ac:=ac+m[sp+x]
1011xxxxxxxxxxxx	SUBL	Subtract local	ac:=ac-m[sp+x]
1100xxxxxxxxxxxx	JNEG	Jump negative	if ac<0 then pc:=x
1101xxxxxxxxxxxx	JNZE	Jump nonzero	if ac<>0 then pc:=x
1110xxxxxxxxxxxx	CALL	Call procedure	sp:=sp-1; m[sp]:=pc; pc:=x
1111000000000000	PSHI	Push indirect	sp:=sp-1; m[sp]:=m[ac]
1111001000000000	POPI	Pop indirect	m[ac]:=m[sp]; sp:=sp+1
1111010000000000	PUSH	Push onto stack	sp:=sp-1; m[sp]:=ac
1111011000000000	POP	Pop from stack	ac:=m[sp]; sp:=sp+ 1
1111100000000000	RETN	Return	pc:=m[sp]; sp:=sp+ 1
1111101000000000	SWAP	Swap ac sp	tmp:=ac; ac:=sp; sp:=tmp
11111100yyyyyyyy	INSP	Increment sp	sp:=sp+y (0<=y<=255)
11111110yyyyyyyy	DESP	Decrement sp	sp:=sp-y (0<=y<=255)

STORE DIRECT

. . .
STOD 9
. . .

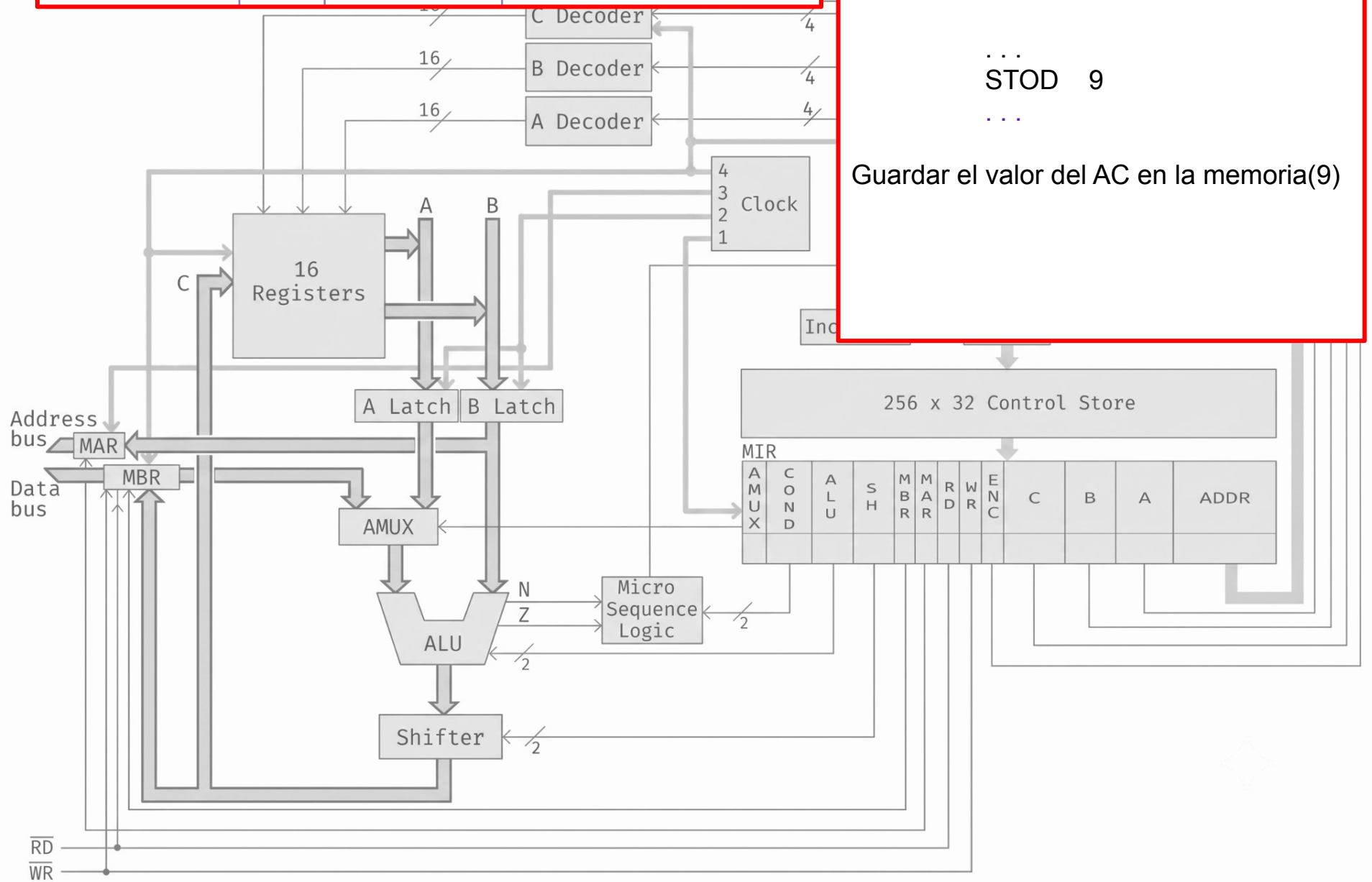
Guardar el valor del AC en la memoria(9)

0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac

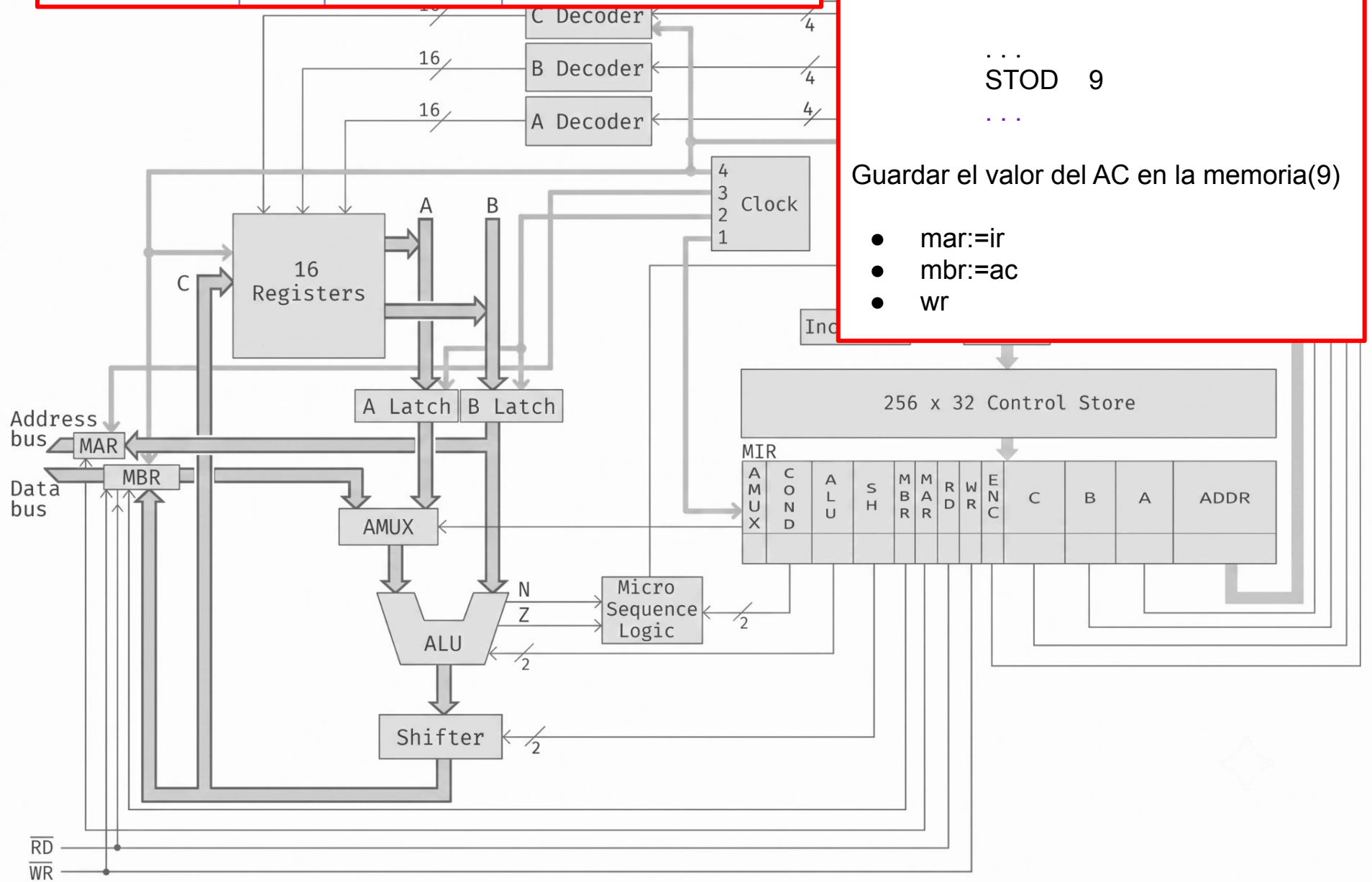
STORE DIRECT

. . .
STOD 9
. . .

Guardar el valor del AC en la memoria(9)



0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac



STORE DIRECT

. . .
STOD 9
. . .

Guardar el valor del AC en la memoria(9)

- mar:=ir
- mbr:=ac
- wr

EXPRESIÓN	AMUX	COND	ALU	SH	MBR	MAR	RD	WR	ENC	C	B	A	ADDR
mar := ir ; mbr := ac ; *wr* ;	0	0	2	0	1	1	0	1	0	0	3	1	0

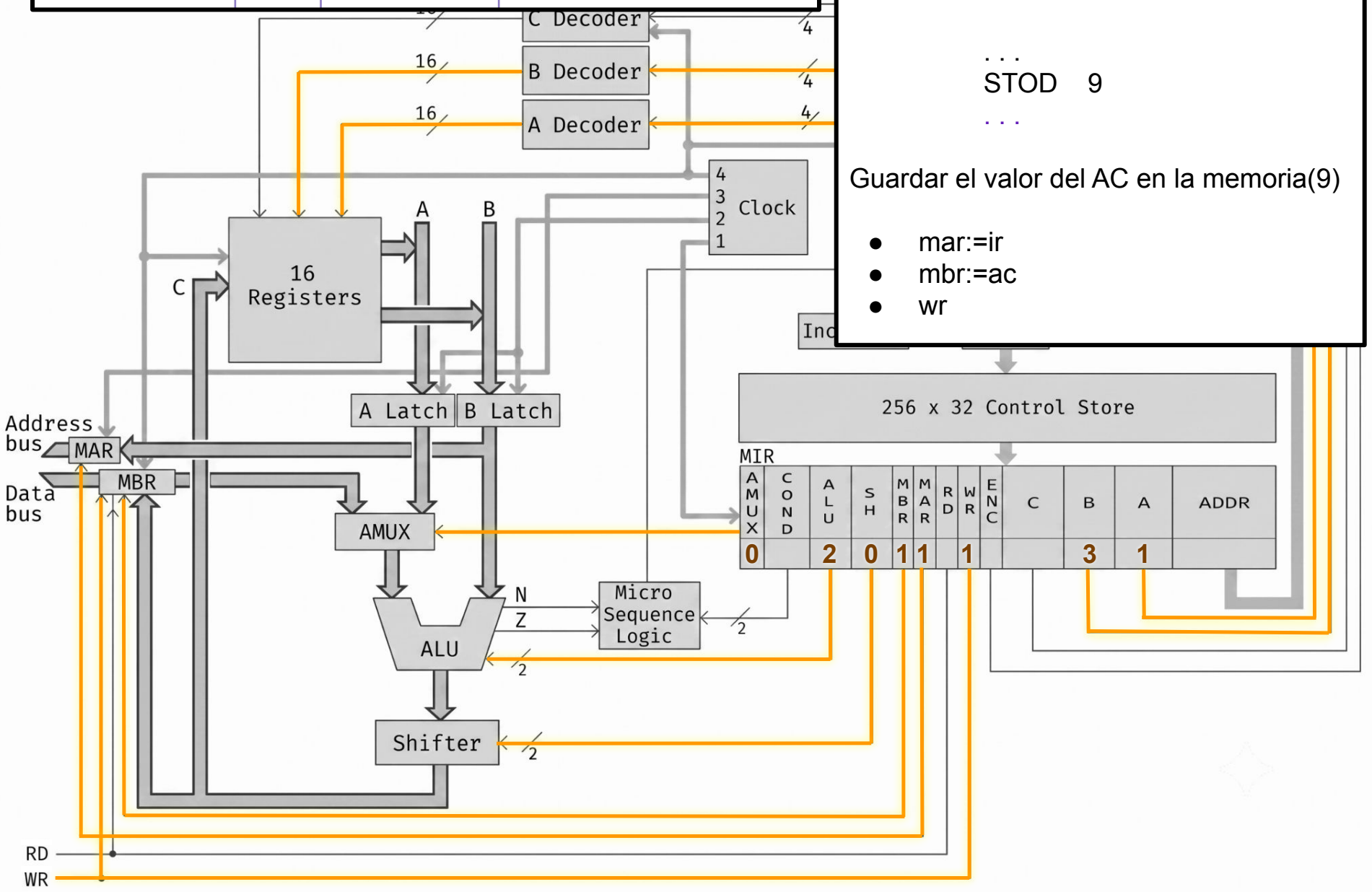
0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac

STORE DIRECT

. . .
STOD 9
. . .

Guardar el valor del AC en la memoria(9)

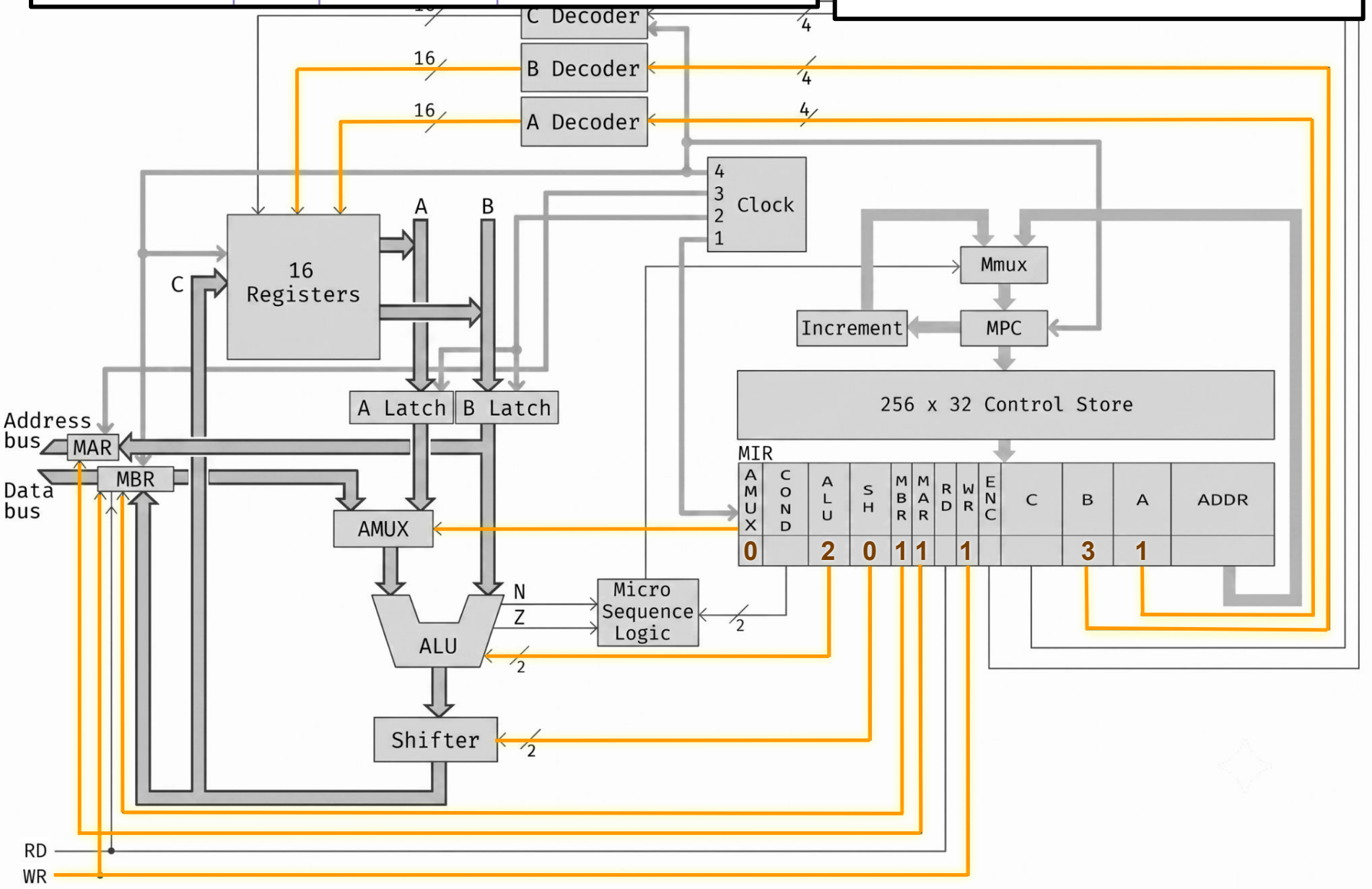
- mar:=ir
- mbr:=ac
- wr



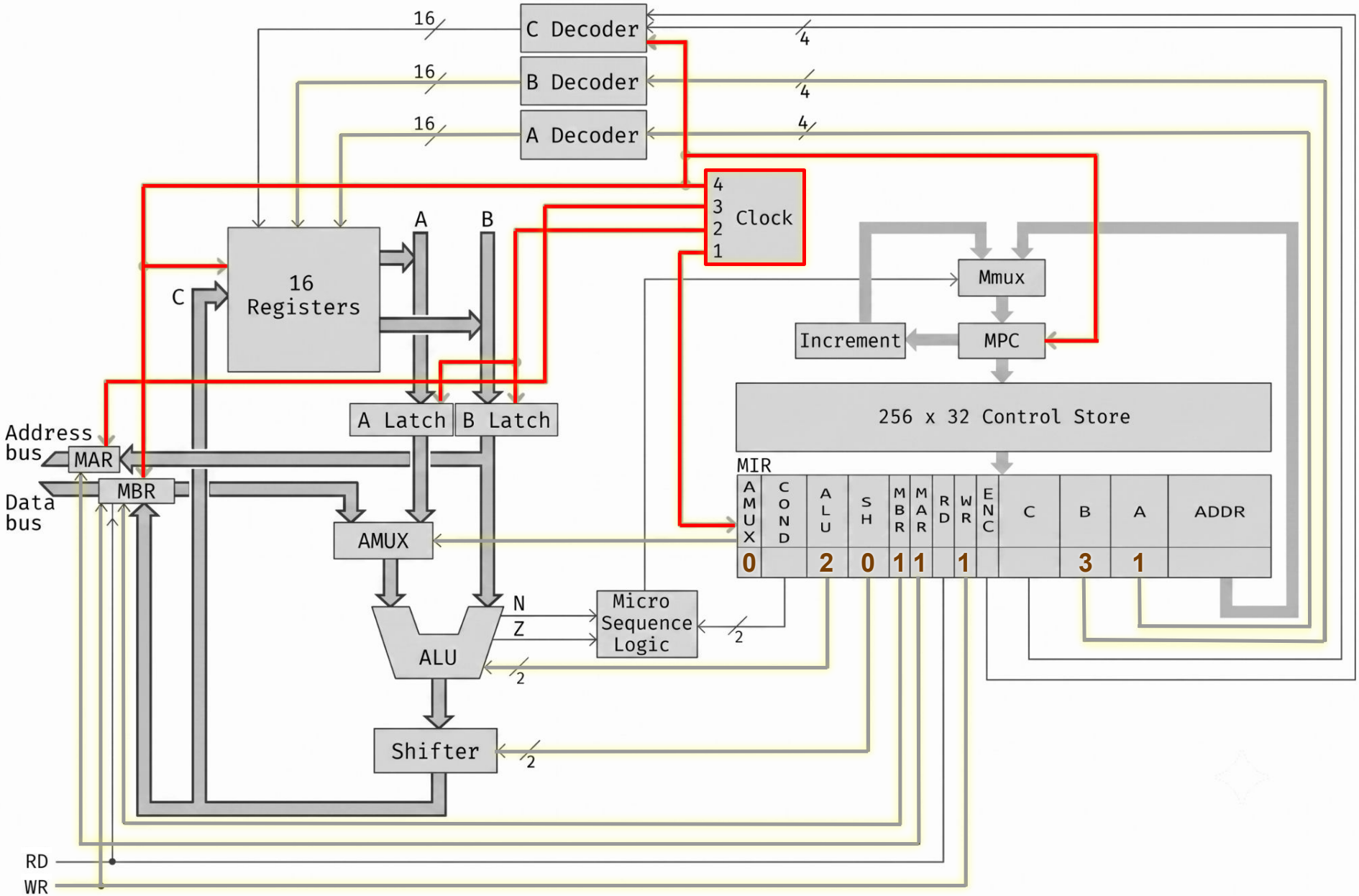
EXPRESIÓN	AMUX	COND	ALU	SH	MBR	MAR	RD	WR	ENC	C	B	A	ADDR
mar := ir ; mbr := ac ; *wr* ;	0	0	2	0	1	1	0	1	0	0	3	1	0

0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac

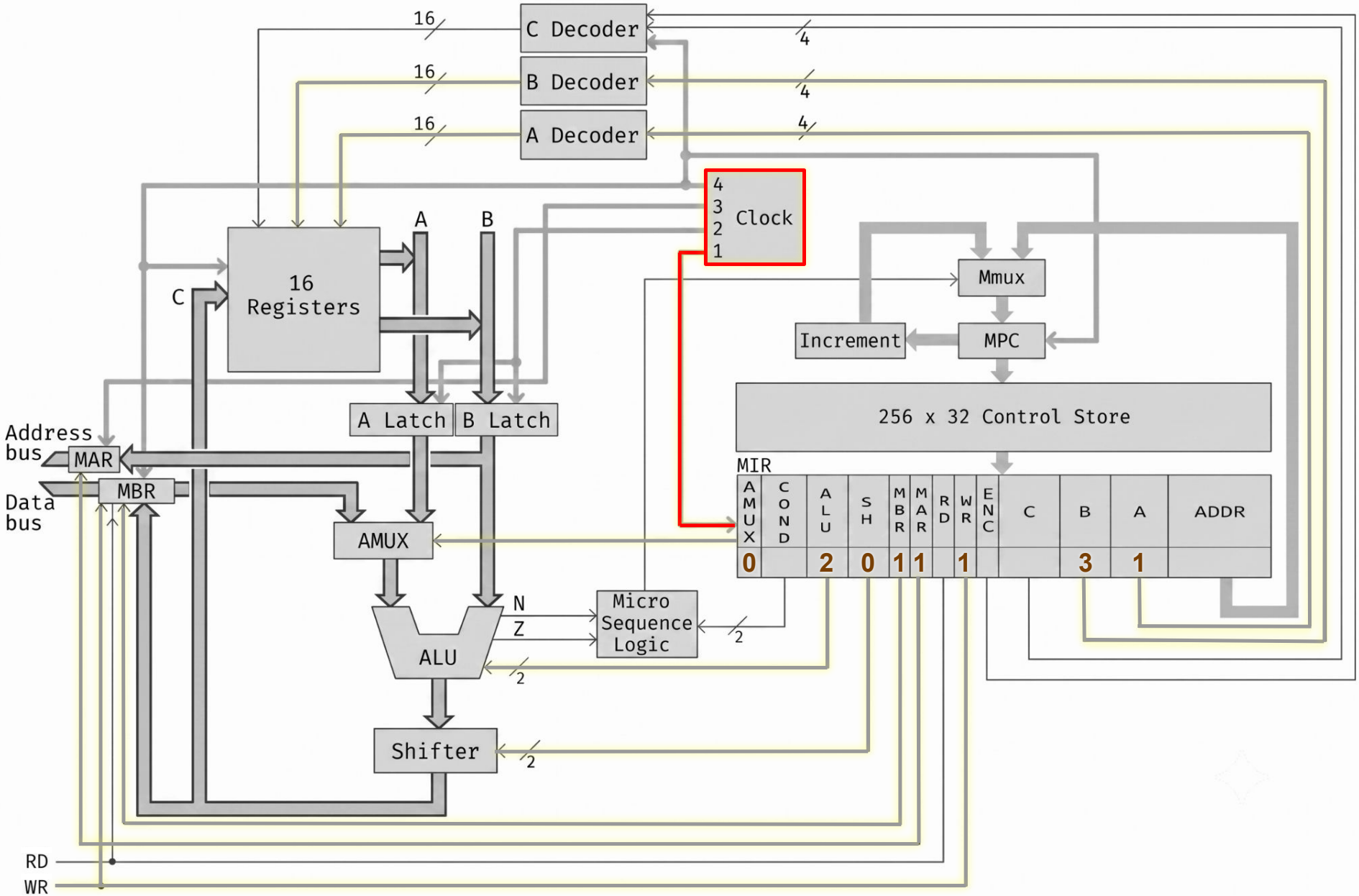
STORE DIRECT



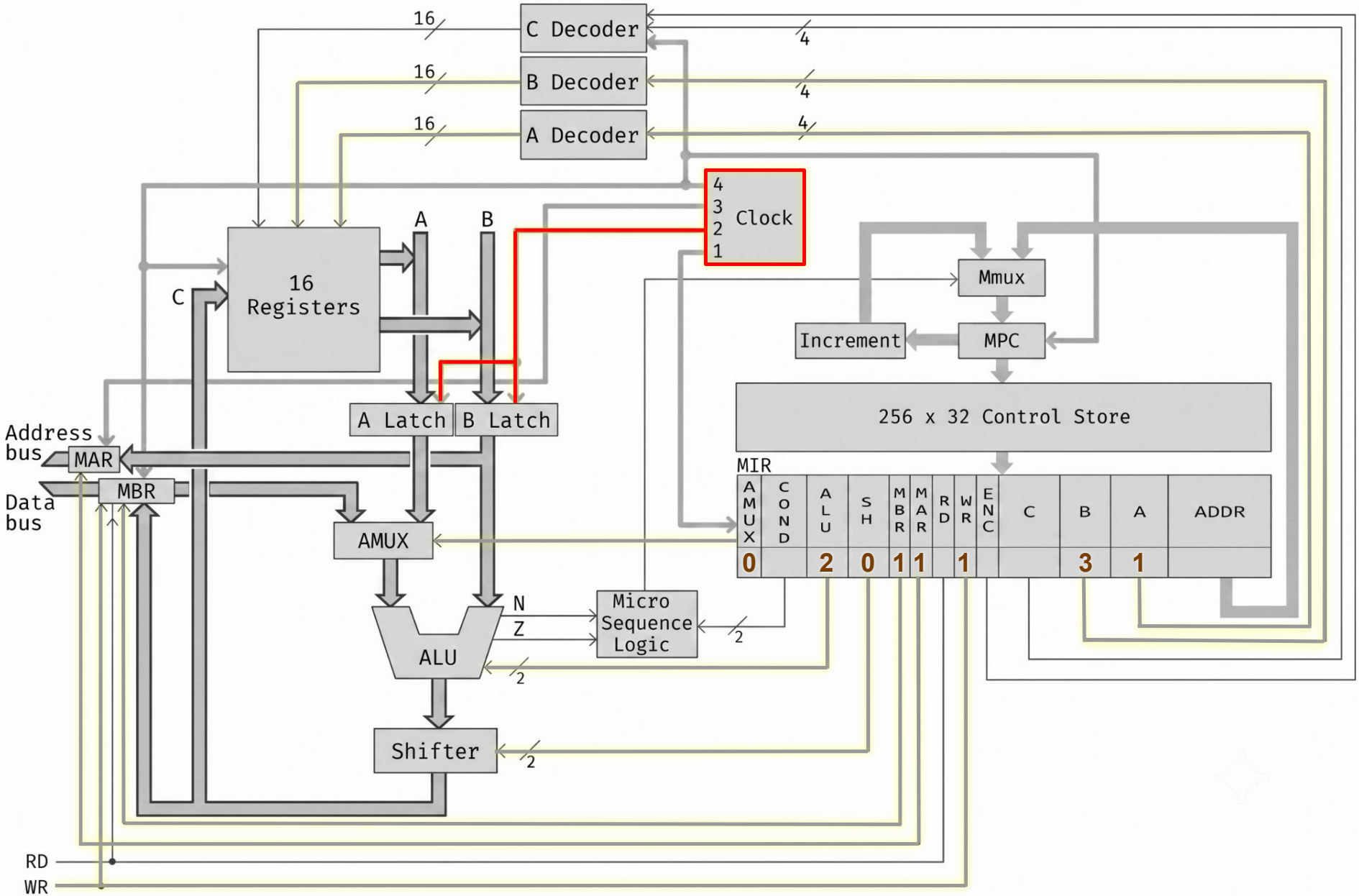
EXPRESIÓN	AMUX	COND	ALU	SH	MBR	MAR	RD	WR	ENC	C	B	A	ADDR
mar := ir ; mbr := ac ; *wr* ;	0	0	2	0	1	1	0	1	0	0	3	1	0



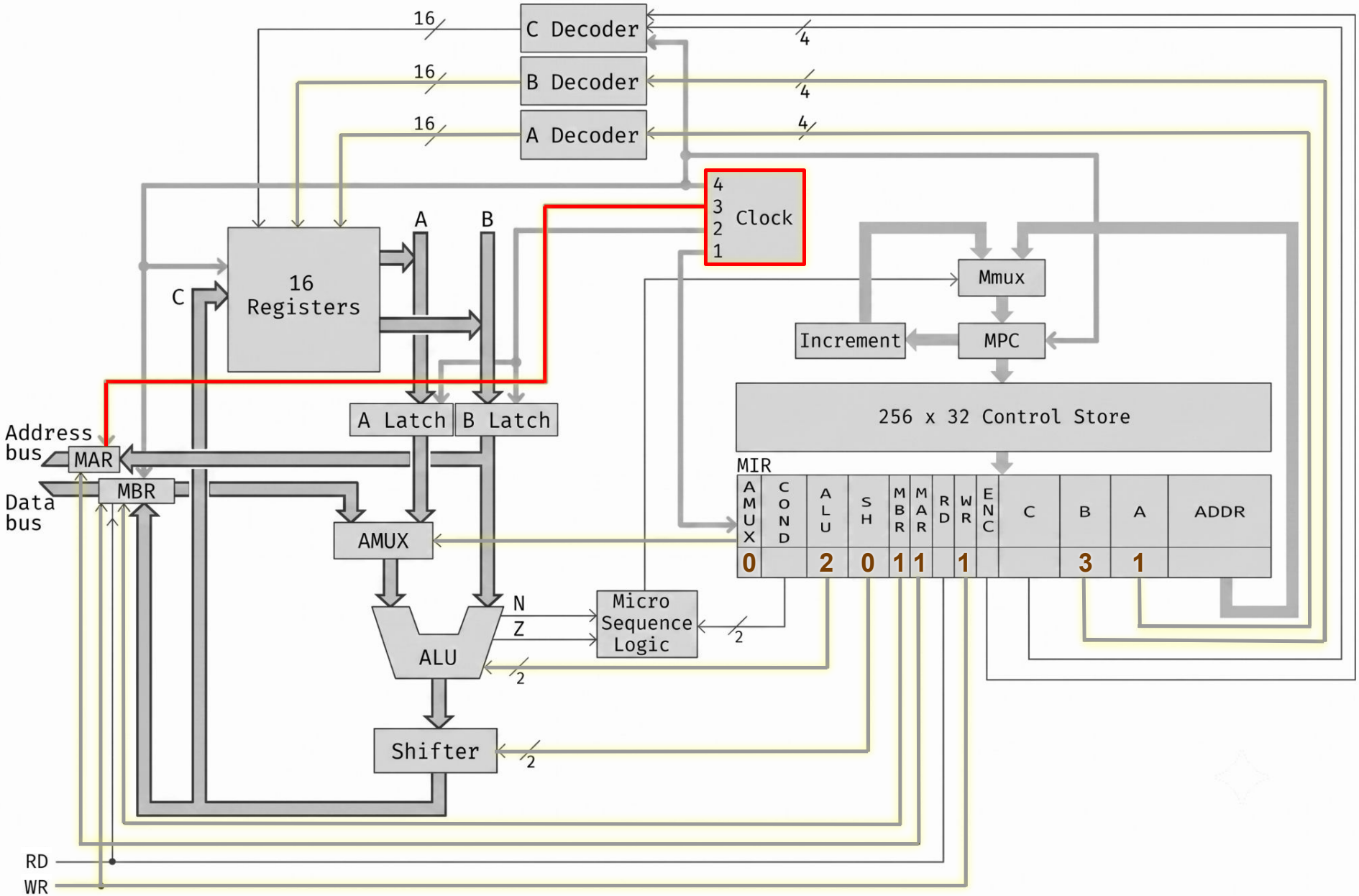
EXPRESIÓN	AMUX	COND	ALU	SH	MBR	MAR	RD	WR	ENC	C	B	A	ADDR
mar := ir ; mbr := ac ; *wr* ;	0	0	2	0	1	1	0	1	0	0	3	1	0



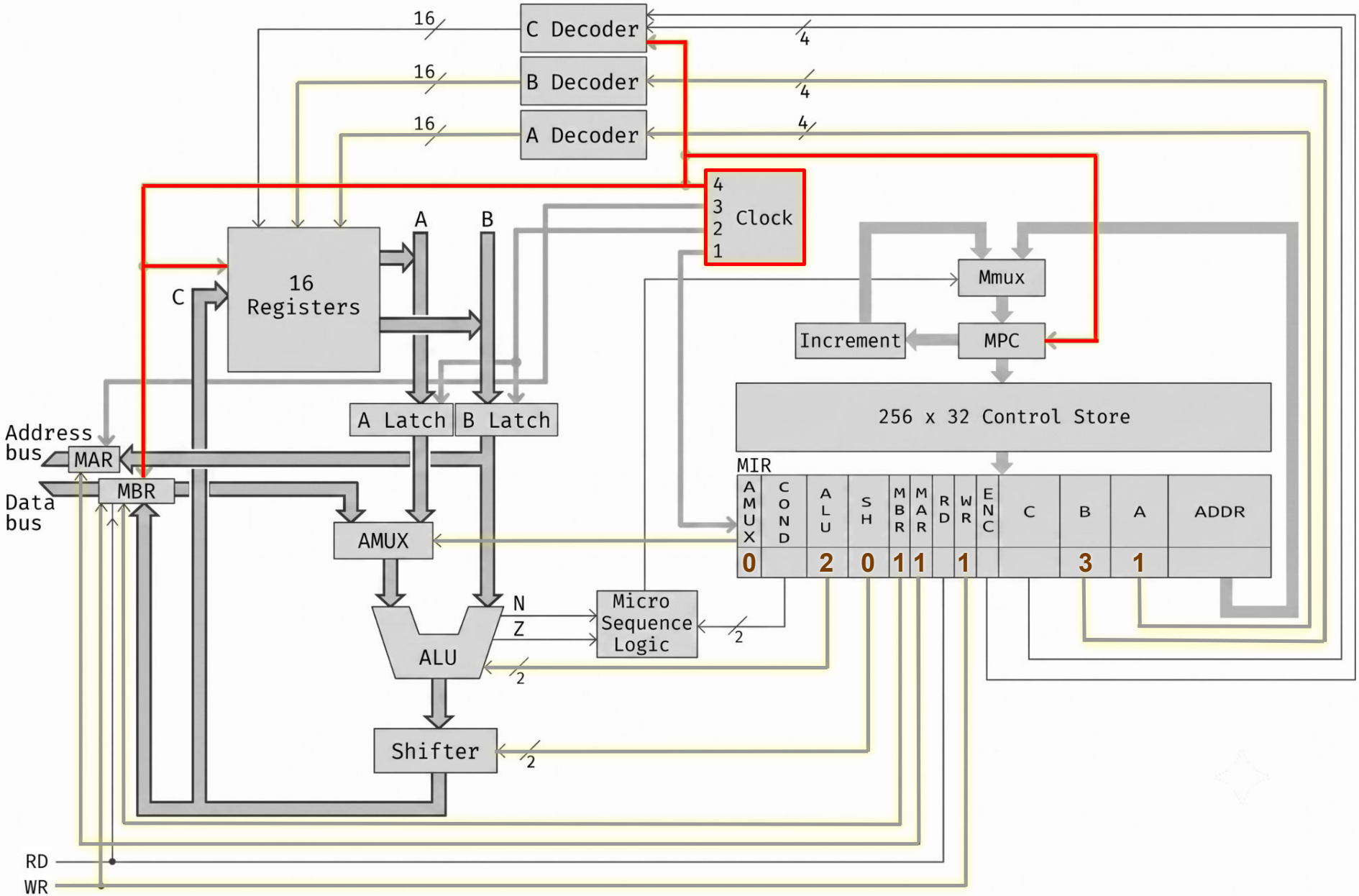
EXPRESIÓN	AMUX	COND	ALU	SH	MBR	MAR	RD	WR	ENC	C	B	A	ADDR
mar := ir ; mbr := ac ; *wr* ;	0	0	2	0	1	1	0	1	0	0	3	1	0



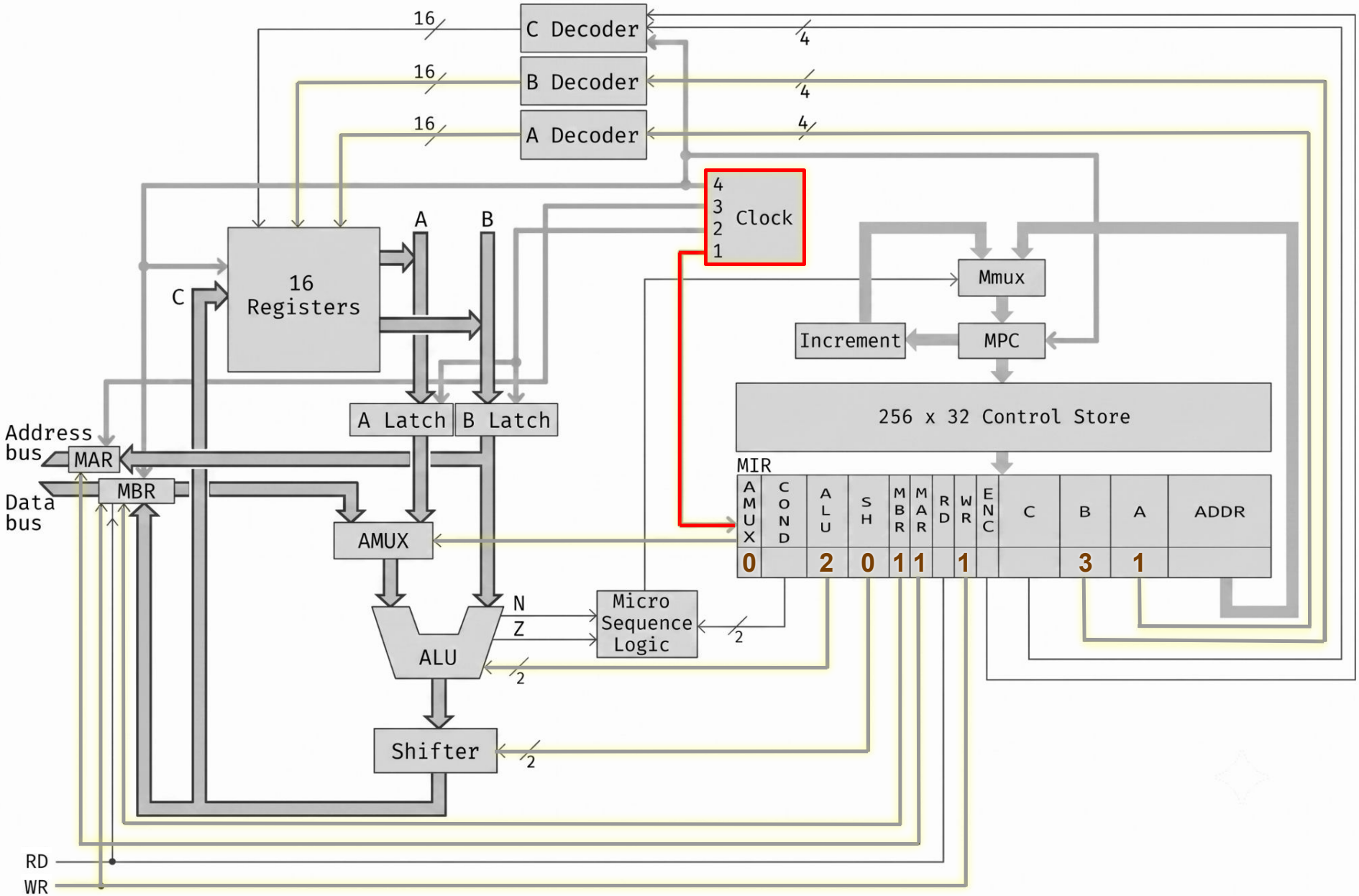
EXPRESIÓN	AMUX	COND	ALU	SH	MBR	MAR	RD	WR	ENC	C	B	A	ADDR
mar := ir ; mbr := ac ; *wr* ;	0	0	2	0	1	1	0	1	0	0	3	1	0



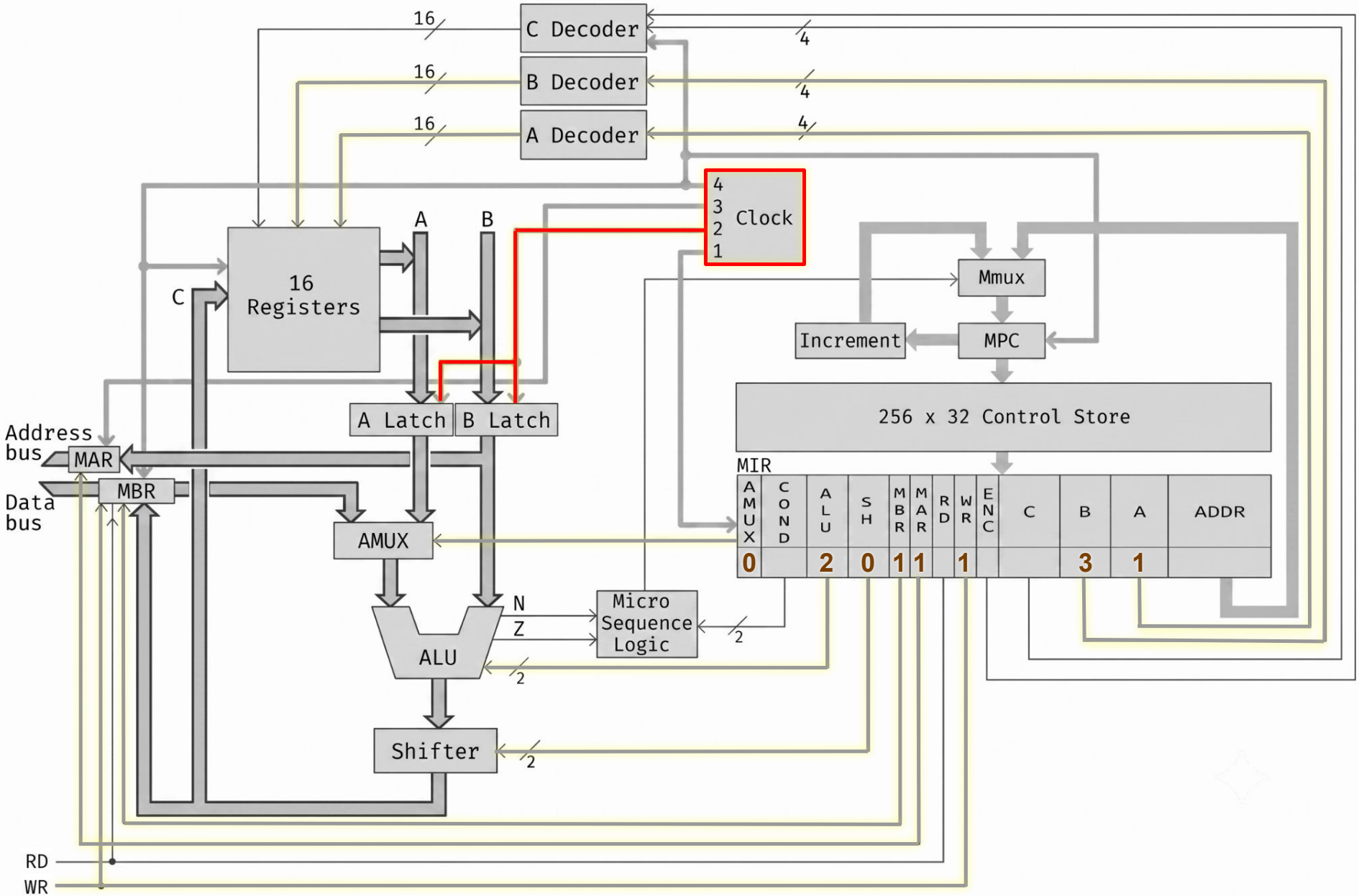
EXPRESIÓN	AMUX	COND	ALU	SH	MBR	MAR	RD	WR	ENC	C	B	A	ADDR
mar := ir ; mbr := ac ; *wr* ;	0	0	2	0	1	1	0	1	0	0	3	1	0



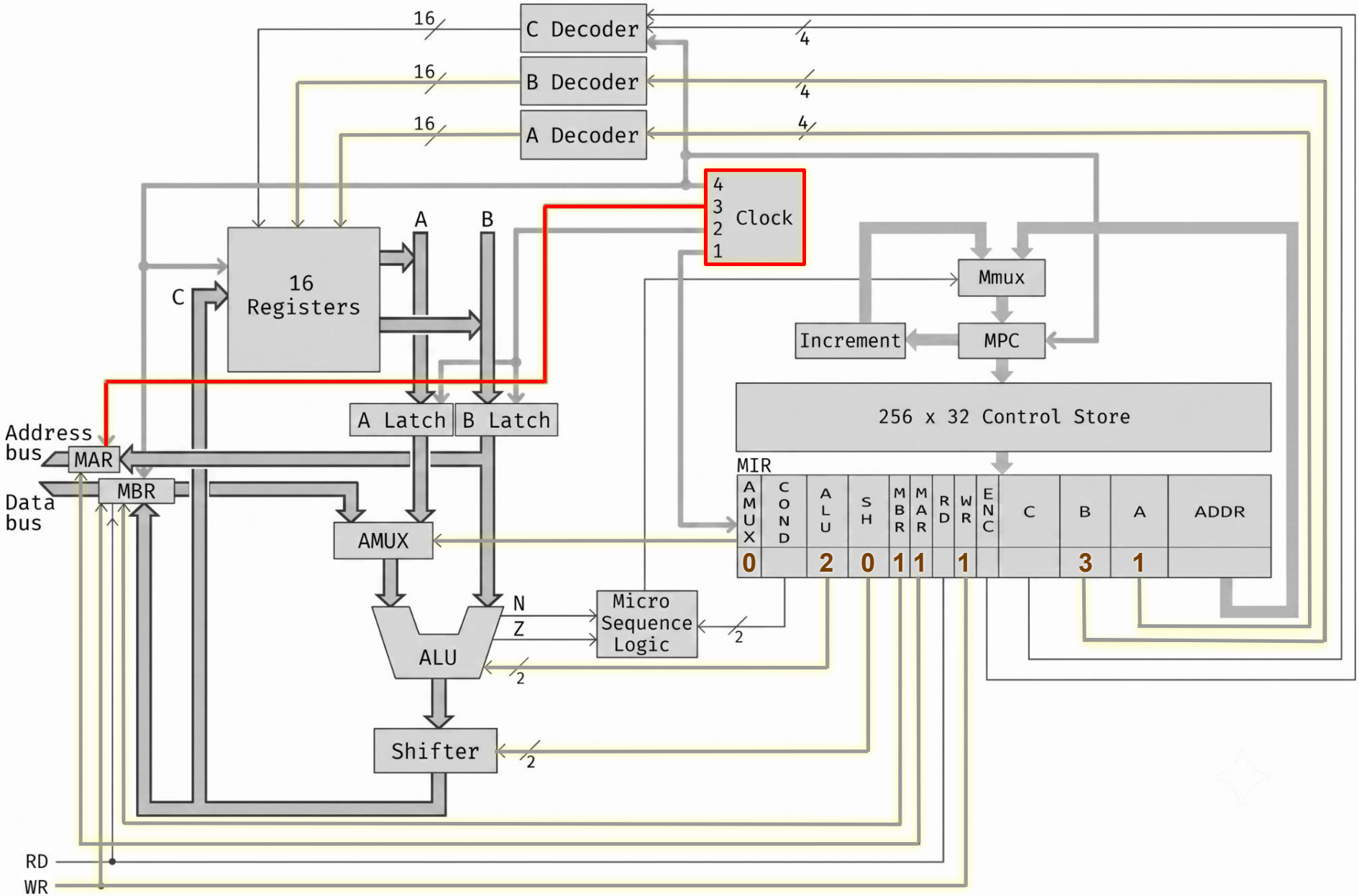
EXPRESIÓN	AMUX	COND	ALU	SH	MBR	MAR	RD	WR	ENC	C	B	A	ADDR
mar := ir ; mbr := ac ; *wr* ;	0	0	2	0	1	1	0	1	0	0	3	1	0



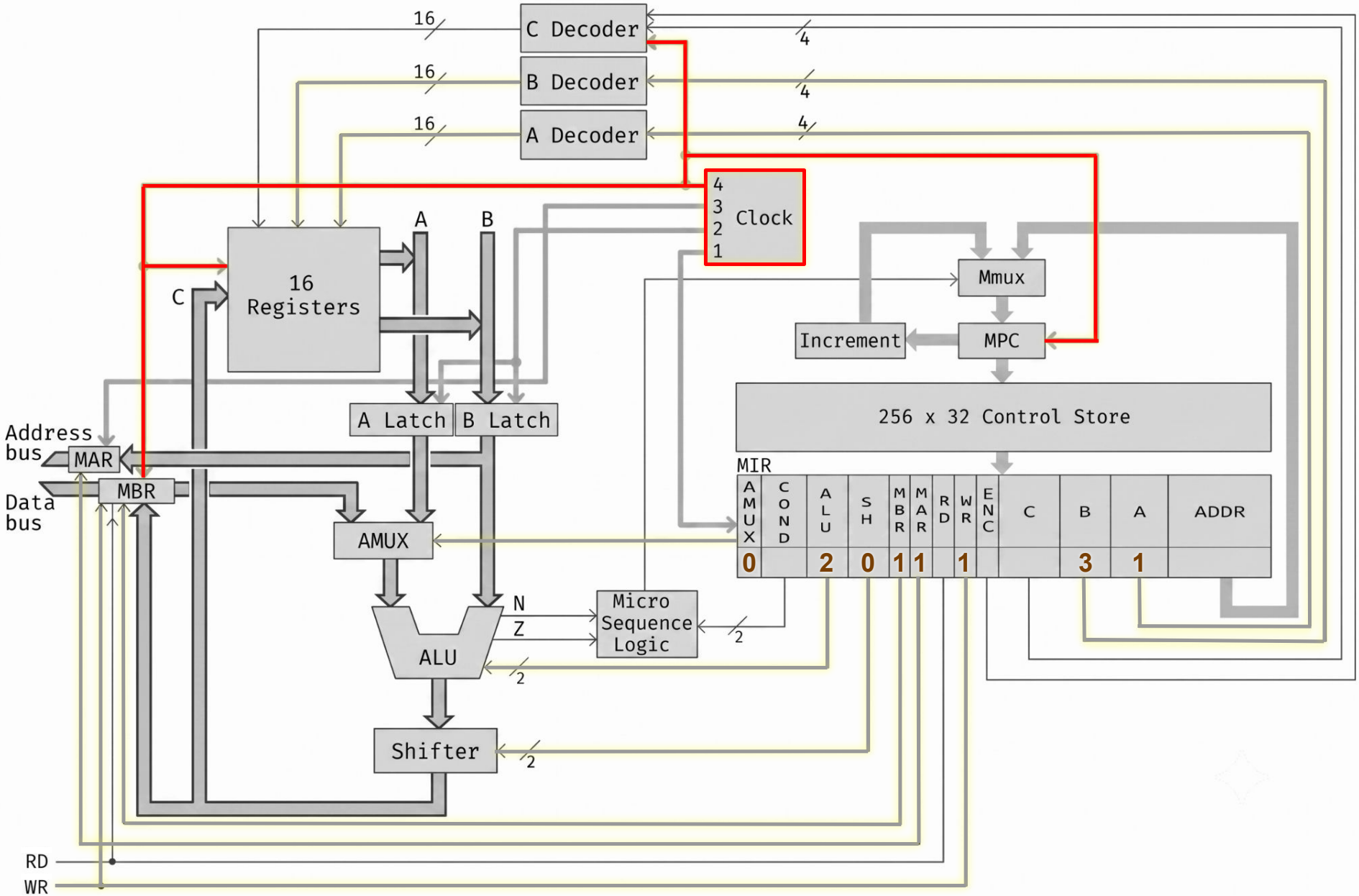
EXPRESIÓN	AMUX	COND	ALU	SH	MBR	MAR	RD	WR	ENC	C	B	A	ADDR
mar := ir ; mbr := ac ; *wr* ;	0	0	2	0	1	1	0	1	0	0	3	1	0



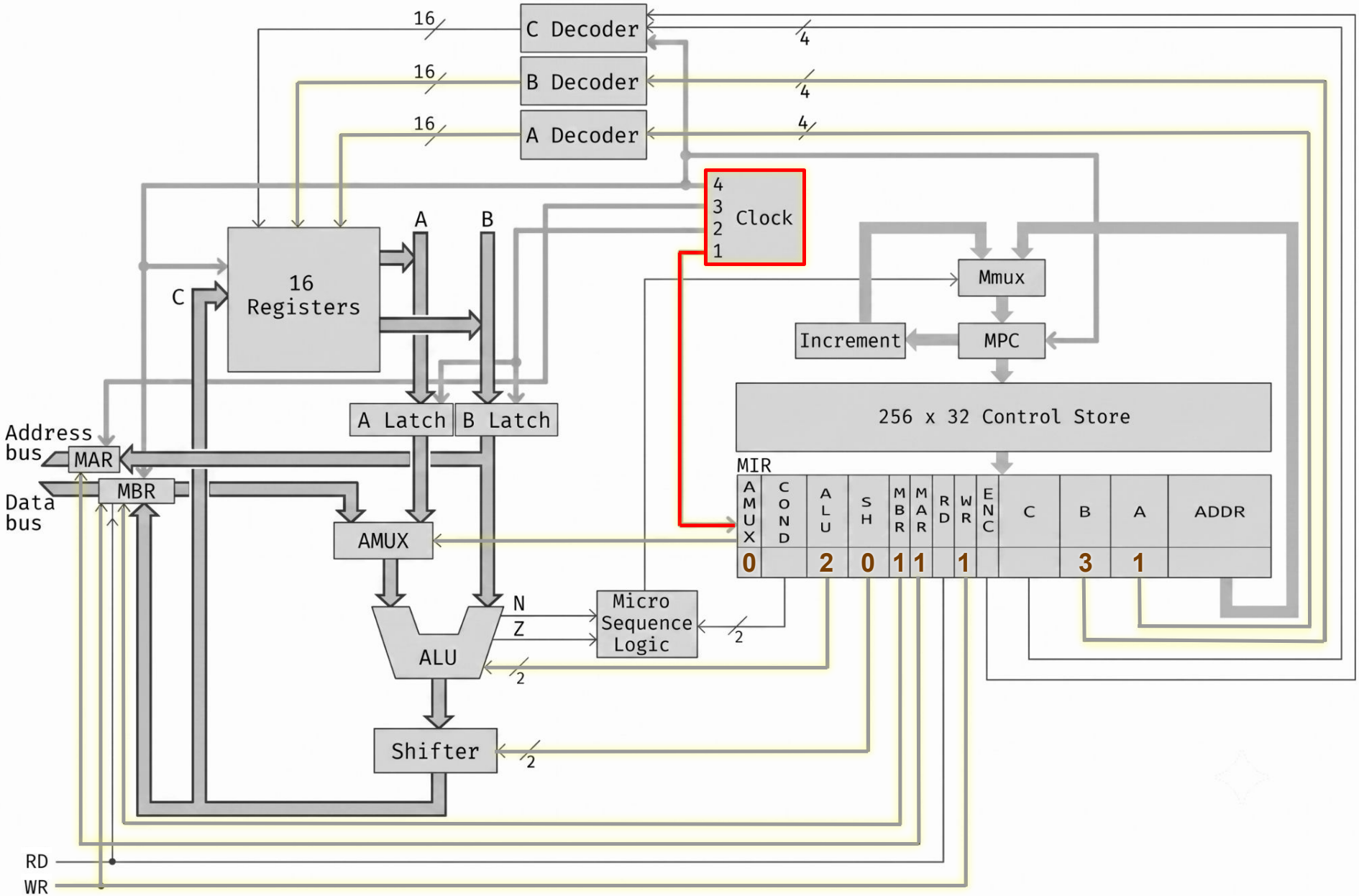
EXPRESIÓN	AMUX	COND	ALU	SH	MBR	MAR	RD	WR	ENC	C	B	A	ADDR
mar := ir ; mbr := ac ; *wr* ;	0	0	2	0	1	1	0	1	0	0	3	1	0



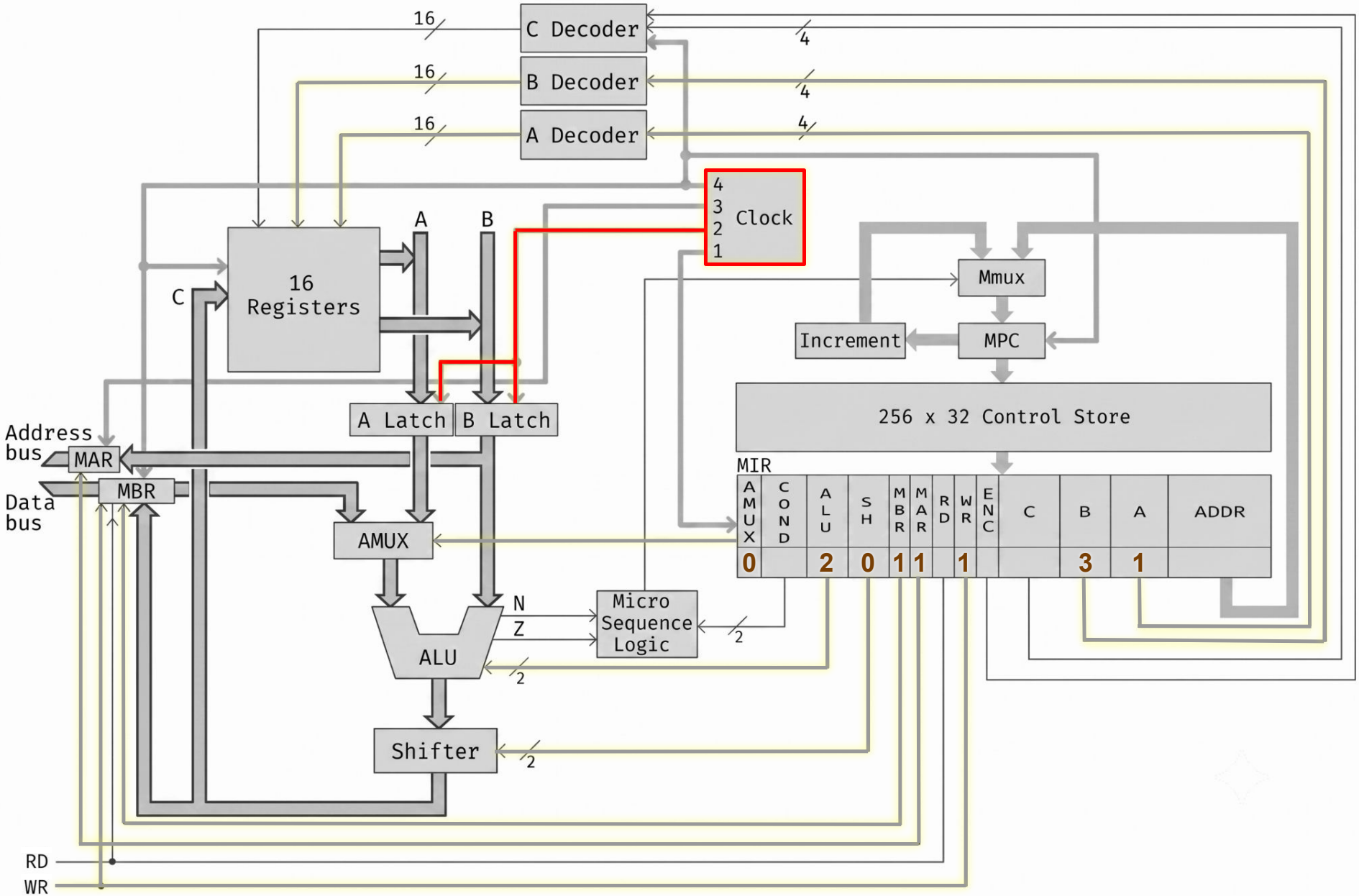
EXPRESIÓN	AMUX	COND	ALU	SH	MBR	MAR	RD	WR	ENC	C	B	A	ADDR
mar := ir ; mbr := ac ; *wr* ;	0	0	2	0	1	1	0	1	0	0	3	1	0



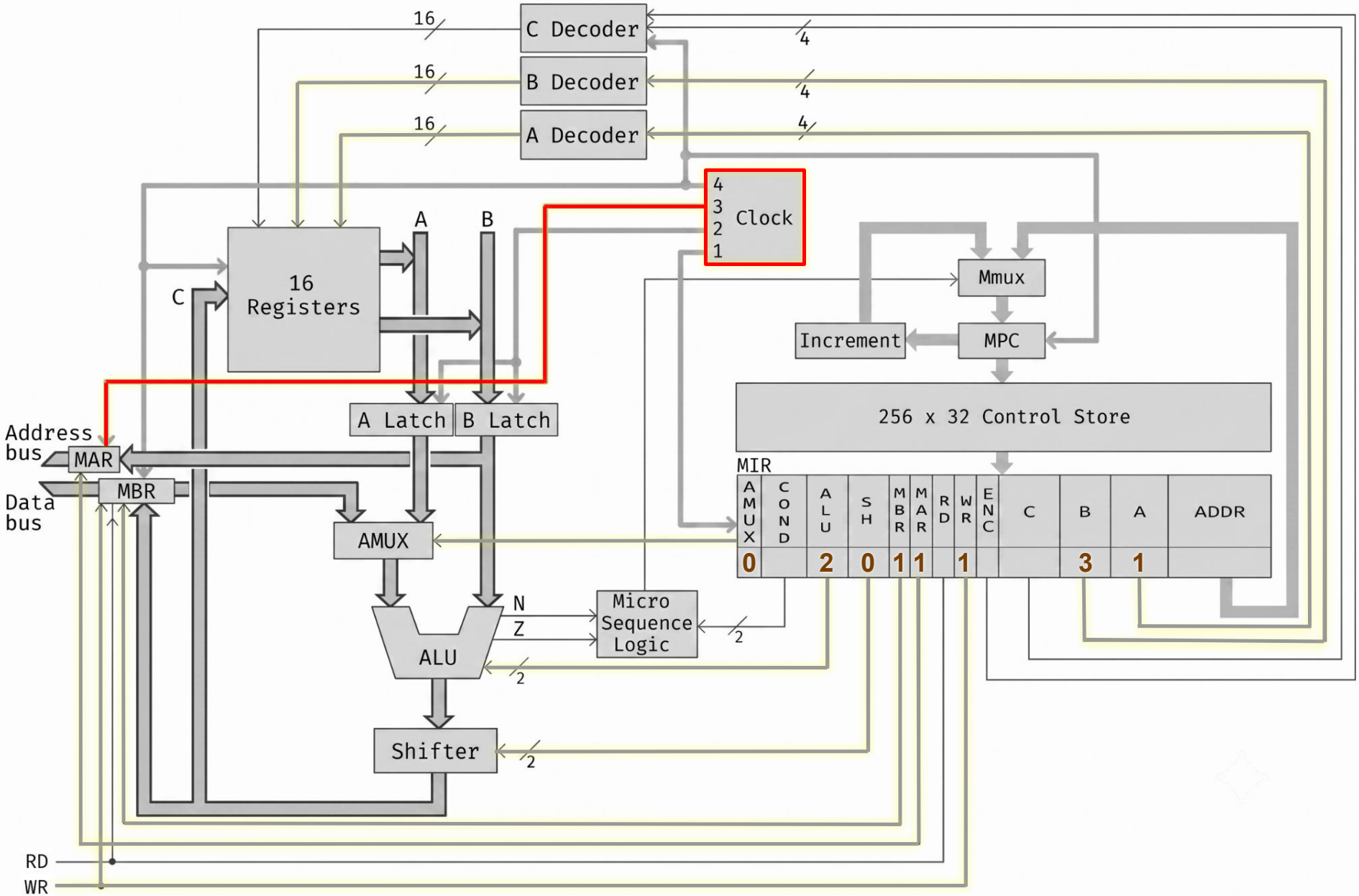
EXPRESIÓN	AMUX	COND	ALU	SH	MBR	MAR	RD	WR	ENC	C	B	A	ADDR
mar := ir ; mbr := ac ; *wr* ;	0	0	2	0	1	1	0	1	0	0	3	1	0



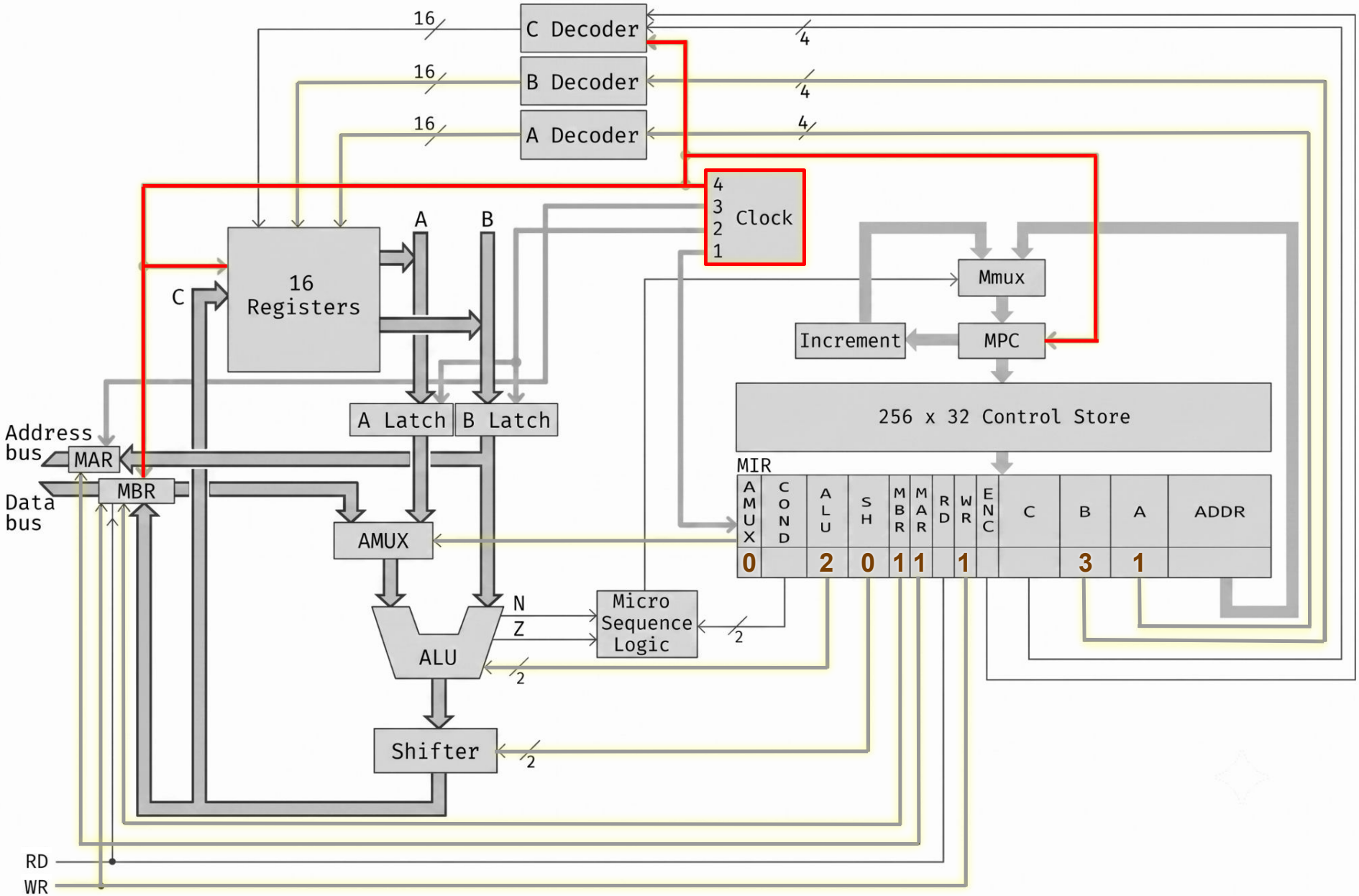
EXPRESIÓN	AMUX	COND	ALU	SH	MBR	MAR	RD	WR	ENC	C	B	A	ADDR
mar := ir ; mbr := ac ; *wr* ;	0	0	2	0	1	1	0	1	0	0	3	1	0



EXPRESIÓN	AMUX	COND	ALU	SH	MBR	MAR	RD	WR	ENC	C	B	A	ADDR
mar := ir ; mbr := ac ; *wr* ;	0	0	2	0	1	1	0	1	0	0	3	1	0



EXPRESIÓN	AMUX	COND	ALU	SH	MBR	MAR	RD	WR	ENC	C	B	A	ADDR
mar := ir ; mbr := ac ; *wr* ;	0	0	2	0	1	1	0	1	0	0	3	1	0



```
0: mar := pc ; rd ;                                  {ciclo principal}
1: pc := pc + 1 ; rd ;                               {incrementa pc}
2: ir := mbr ; if n then goto 28 ;                   {salva y decodifica mbr}
3: tir := lshift(ir + ir) ; if n then goto 19 ;
4: tir := lshift(tir) ; if n then goto 11 ;          {000x o 001x?}
5: alu := tir ; if n then goto 9 ;                   {0000 o 0001?}
6: mar := ir ; rd ;                                  {0000 = LODD}
7: rd ;
8: ac := mbr ; goto 0 ;
9: mar := ir ; mbr := ac ; wr ;                      {0001 = STOD}
10: wr ; goto 0 ;
11: alu := tir ; if n then goto 15 ;                 {0010 o 0011?}
12: mar := ir ; rd ;                                 {0010 = ADDD}
13: rd ;
14: ac := mbr + ac ; goto 0 ;
15: mar := ir ; rd ;                                 {0011 = SUBD}
16: ac := ac + 1 ; rd ;                              {Nota: x - y = x + 1 + no y}
17: a := inv(mbr) ;
18: ac := ac + a ; goto 0 ;
19: tir := lshift(tir) ; if n then goto 25 ;         {010x o 011x?}
20: alu := tir ; if n then goto 23 ;                 {0100 o 0101?}
21: alu := ac ; if n then goto 0 ;                   {0100 = JPOS}
22: pc := band(ir , amask) ; goto 0 ;                {realiza el salto}
23: alu := ac ; if z then goto 22 ;                  {0101 = JZER}
24: goto 0 ;                                         {no se produce el salto}
25: alu := tir ; if n then goto 27 ;                 {0110 o 0111?}
26: pc := band(ir , amask) ; goto 0 ;                {0110 = JUMP}
27: ac := band(ir , amask) ; goto 0 ;                {0111 = LOCO}
28: tir := lshift(ir + ir) ; if n then goto 40 ;     {10xx o 11xx?}
29: tir := lshift(tir) ; if n then goto 35 ;         {100x o 101x?}
30: alu := tir ; if n then goto 33 ;                 {1000 o 1001?}
31: a := ir + sp ;                                   {1000 = LODL}
32: mar := a ; rd ; goto 7 ;
33: a := ir + sp ;                                   {1001 = STOL}
34: mar := a ; mbr := ac ; wr ; goto 10 ;
35: alu := tir ; if n then goto 38 ;                 {1010 o 1011?}
36: a := ir + sp ;                                   {1010 = ADDL}
37: mar := a ; rd ; goto 13 ;
38: a := ir + sp ;                                   {1011 = SUBL}
39: mar := a ; rd ; goto 16 ;
40: tir := lshift(tir) ; if n then goto 46 ;         {110x o 111x?}
41: alu := tir ; if n then goto 44 ;                 {1100 o 1101?}
42: alu := ac ; if n then goto 22 ;                  {1100 = JNEG}
43: goto 0 ;
44: alu := ac ; if z then goto 0 ;                   {1101 = JNZE}
45: pc := band(ir , amask) ; goto 0 ;
46: tir := lshift(tir) ; if n then goto 50 ;
47: sp := sp + (-1) ;                                {1110 = CALL}
48: mar := sp ; mbr := pc ; wr ;
49: pc := band(ir , amask) ; wr ; goto 0 ;
50: tir := lshift(tir) ; if n then goto 65 ;         {111, examina addr}
51: tir := lshift(tir) ; if n then goto 59 ;
52: alu := tir ; if n then goto 56 ;
53: mar := ac ; rd ;                                 {1111000 = PSHI}
54: sp := sp + (-1) ; rd ;
55: mar := sp ; wr ; goto 10 ;
56: mar := sp ; sp := sp + 1 ; rd ;                  {1111001 = POPI}
57: wr ;
58: mar := ac ; wr ; goto 10 ;
59: alu := tir ; if n then goto 62 ;
60: sp := sp + (-1) ;                                {1111010 = PUSH}
61: mar := sp ; mbr := ac ; wr ; goto 10 ;
62: mar := sp ; sp := sp + 1 ; rd ;                  {1111011 = POP}
63: rd ;
64: ac := mbr ; goto 0 ;
65: tir := lshift(tir) ; if n then goto 73 ;
66: alu := tir ; if n then goto 70 ;
67: mar := sp ; sp := sp + 1 ; rd ;                  {1111100 = RETN}
68: rd ;
69: pc := mbr ; goto 0 ;
70: a := ac ;                                        {1111101 = SWAP}
71: ac := sp ;
72: sp := a ; goto 0 ;
73: alu := tir ; if n then goto 76 ;
74: a := band(ir , smask) ;                          {1111110 = INSP}
75: sp := sp + a ; goto 0 ;
76: a := band(ir , smask) ;                          {1111111 = DESP}
77: a := inv(a) ;
78: a := a + 1 ; goto 75 ;
```

Microprograma

```
0: mar := pc ; rd ;                                  {ciclo principal}
1: pc := pc + 1 ; rd ;                               {incrementa pc}
2: ir := mbr ; if n then goto 28 ;                   {salva y decodifica mbr}
3: tir := lshift(ir + ir) ; if n then goto 19 ;
4: tir := lshift(tir) ; if n then goto 11 ;          {000x o 001x?}
5: alu := tir ; if n then goto 9 ;                   {0000 o 0001?}
6: mar := ir ; rd ;                                  {0000 = LODD}
7: rd ;
8: ac := mbr ; goto 0 ;
9: mar := ir ; mbr := ac ; wr ;                      {0001 = STOD}
10: wr ; goto 0 ;
11: alu := tir ; if n then goto 15 ;                 {0010 o 0011?}
12: mar := ir ; rd ;                                 {0010 = ADDD}
13: rd ;
14: ac := mbr + ac ; goto 0 ;
15: mar := ir ; rd ;                                 {0011 = SUBD}
16: ac := ac + 1 ; rd ;                              {Nota: x - y = x + 1 + no y}
17: a := inv(mbr) ;
18: ac := ac + a ; goto 0 ;
19: tir := lshift(tir) ; if n then goto 25 ;         {010x o 011x?}
20: alu := tir ; if n then goto 23 ;                 {0100 o 0101?}
21: alu := ac ; if n then goto 0 ;                   {0100 = JPOS}
22: pc := band(ir , amask) ; goto 0 ;                {realiza el salto}
23: alu := ac ; if z then goto 22 ;                  {0101 = JZER}
24: goto 0 ;                                         {no se produce el salto}
25: alu := tir ; if n then goto 27 ;                 {0110 o 0111?}
26: pc := band(ir , amask) ; goto 0 ;                {0110 = JUMP}
27: ac := band(ir , amask) ; goto 0 ;                {0111 = LOCO}
28: tir := lshift(ir + ir) ; if n then goto 40 ;     {10xx o 11xx?}
29: tir := lshift(tir) ; if n then goto 35 ;         {100x o 101x?}
30: alu := tir ; if n then goto 33 ;                 {1000 o 1001?}
31: a := ir + sp ;                                   {1000 = LODL}
32: mar := a ; rd ; goto 7 ;
33: a := ir + sp ;                                   {1001 = STOL}
34: mar := a ; mbr := ac ; wr ; goto 10 ;
35: alu := tir ; if n then goto 38 ;                 {1010 o 1011?}
36: a := ir + sp ;                                   {1010 = ADDL}
37: mar := a ; rd ; goto 13 ;
38: a := ir + sp ;                                   {1011 = SUBL}
39: mar := a ; rd ; goto 16 ;
40: tir := lshift(tir) ; if n then goto 46 ;         {110x o 111x?}
41: alu := tir ; if n then goto 44 ;                 {1100 o 1101?}
42: alu := ac ; if n then goto 22 ;                  {1100 = JNEG}
43: goto 0 ;
44: alu := ac ; if z then goto 0 ;                   {1101 = JNZE}
45: pc := band(ir , amask) ; goto 0 ;
46: tir := lshift(tir) ; if n then goto 50 ;
47: sp := sp + (-1) ;                                {1110 = CALL}
48: mar := sp ; mbr := pc ; wr ;
49: pc := band(ir , amask) ; wr ; goto 0 ;
50: tir := lshift(tir) ; if n then goto 65 ;         {111, examina addr}
51: tir := lshift(tir) ; if n then goto 59 ;
52: alu := tir ; if n then goto 56 ;
53: mar := ac ; rd ;                                 {1111000 = PSHI}
54: sp := sp + (-1) ; rd ;
55: mar := sp ; wr ; goto 10 ;
56: mar := sp ; sp := sp + 1 ; rd ;                  {1111001 = POPI}
57: rd ;
58: mar := ac ; wr ; goto 10 ;
59: alu := tir ; if n then goto 62 ;
60: sp := sp + (-1) ;                                {1111010 = PUSH}
61: mar := sp ; mbr := ac ; wr ; goto 10 ;
62: mar := sp ; sp := sp + 1 ; rd ;                  {1111011 = POP}
63: rd ;
64: ac := mbr ; goto 0 ;
65: tir := lshift(tir) ; if n then goto 73 ;
66: alu := tir ; if n then goto 70 ;
67: mar := sp ; sp := sp + 1 ; rd ;                  {1111100 = RETN}
68: rd ;
69: pc := mbr ; goto 0 ;
70: a := ac ;                                        {1111101 = SWAP}
71: ac := sp ;
72: sp := a ; goto 0 ;
73: alu := tir ; if n then goto 76 ;
74: a := band(ir , smask) ;                          {1111110 = INSP}
75: sp := sp + a ; goto 0 ;
76: a := band(ir , smask) ;                          {1111111 = DESP}
77: a := inv(a) ;
78: a := a + 1 ; goto 75 ;
```

EXPRESIÓN	AMUX	COND	ALU	SH	MBR	MAR	RD	WR	ENC	C	B	A	ADDR
mar := ir ; mbr := ac ; *wr* ;	**0**	**0**	**2**	**0**	**1**	**1**	**0**	**1**	**0**	**0**	**3**	**1**	**0**

0001xxxxxxxxxxx	STOD	Store direct	m[x]:=ac

STORE DIRECT

8: ac := mbr ; **goto** 0 ;
9: mar := ir ; mbr := ac ; *wr* ; {0001 = **STOD**}
10: *wr* ; **goto** 0 ;
11: alu := tir ; **if** n **then goto** 15 ; {0010 o 0011?}
12: mar := ir ; *rd* ; {0010 = **ADDD**}
13: *rd* ;
14: ac := mbr + ac ; **goto** 0 ;
15: mar := ir ; *rd* ; {0011 = **SUBD**}
16: ac := ac + 1 ; *rd* ; {Nota: x - y = x + 1 + no y}
17: a := *inv*(mbr) ;
18: ac := ac + a ; **goto** 0 ;
19: tir := *lshift*(tir) ; **if** n **then goto** 25 ; {010x o 011x?}
20: alu := tir ; **if** n **then goto** 23 ; {0100 o 0101?}
21: alu := ac ; **if** n **then goto** 0 ; {0100 = **JPOS**}
22: pc := *band*(ir , amask) ; **goto** 0 ; {realiza el salto}
23: alu := ac ; **if** z **then goto** 22 ; {0101 = **JZER**}
24: **goto** 0 ; {no se produce el salto}
25: alu := tir ; **if** n **then goto** 27 ; {0110 o 0111?}
26: pc := *band*(ir , amask) ; **goto** 0 ; {0110 = **JUMP**}
27: ac := *band*(ir , amask) ; **goto** 0 ; {0111 = **LOCO**}
28: tir := *lshift*(ir + ir) ; **if** n **then goto** 40 ; {10xx o 11xx?}
29: tir := *lshift*(tir) ; **if** n **then goto** 35 ; {100x o 101x?}
30: alu := tir ; **if** n **then goto** 33 ; {1000 o 1001?}
31: a := ir + sp ; {1000 = **LODL**}
32: mar := a ; *rd* ; **goto** 7 ;
33: a := ir + sp ; {1001 = **STOL**}
34: mar := a ; mbr := ac ; *wr* ; **goto** 10 ;
35: alu := tir ; **if** n **then goto** 38 ; {1010 o 1011?}
36: a := ir + sp ; {1010 = **ADDL**}
37: mar := a ; *rd* ; **goto** 13 ;
38: a := ir + sp ; {1011 = **SUBL**}
39: mar := a ; *rd* ; **goto** 16 ;

48: mar := sp ; mbr := pc ; *wr* ;
49: pc := *band*(ir , amask) ; *wr* ; **goto** 0 ;
50: tir := *lshift*(tir) ; **if** n **then goto** 65 ; {111, examina addr}
51: tir := *lshift*(tir) ; **if** n **then goto** 59 ;
52: alu := tir ; **if** n **then goto** 56 ;
53: mar := ac ; rd ; {1111000 = **PSHI**}
54: sp := sp + (-1) ; rd ;
55: mar := sp ; *wr* ; **goto** 10 ;
56: mar := sp ; sp := sp + 1 ; *rd* ; {1111001 = **POPI**}
57: *rd* ;
58: mar := ac ; *wr* ; **goto** 10 ;
59: alu := tir ; **if** n **then goto** 62 ;
60: sp := sp + (-1) ; {1111010 = **PUSH**}
61: mar := sp ; mbr := ac ; *wr* ; **goto** 10 ;
62: mar := sp ; sp := sp + 1 ; *rd* ; {1111011 = **POP**}
63: *rd* ;
64: ac := mbr ; **goto** 0 ;
65: tir := *lshift*(tir) ; **if** n **then goto** 73 ;
66: alu := tir ; **if** n **then goto** 70 ;
67: mar := sp ; sp := sp + 1 ; *rd* ; {1111100 = **RETN**}
68: *rd* ;
69: pc := mbr ; **goto** 0 ;
70: a := ac ; {1111101 = **SWAP**}
71: ac := sp ;
72: sp := a ; **goto** 0 ;
73: alu := tir ; **if** n **then goto** 76 ;
74: a := *band*(ir , smask) ; {1111110 = **INSP**}
75: sp := sp + a ; **goto** 0 ;
76: a := *band*(ir , smask) ; {1111111 = **DESP**}
77: a := *inv*(a) ;
78: a := a + 1 ; **goto** 75 ;

EXPRESIÓN	AMUX	COND	ALU	SH	MBR	MAR	RD	WR	ENC	C	B	A	ADDR
wr ; goto 0 ;	**0**	**3**	**0**	**0**	**0**	**0**	**0**	**1**	**0**	**0**	**0**	**0**	**0**

0001xxxxxxxxxxx	STOD	Store direct	m[x]:=ac

STORE DIRECT

8: ac := mbr ; **goto** 0 ;
9: mar := ir ; mbr := ac ; *wr* ; {0001 = **STOD**}
10: *wr* ; **goto** 0 ;
11: alu := tir ; **if** n **then goto** 15 ; {0010 o 0011?}
12: mar := ir ; *rd* ; {0010 = **ADDD**}
13: *rd* ;
14: ac := mbr + ac ; **goto** 0 ;
15: mar := ir ; *rd* ; {0011 = **SUBD**}
16: ac := ac + 1 ; *rd* ; {Nota: x - y = x + 1 + no y}
17: a := *inv*(mbr) ;
18: ac := ac + a ; **goto** 0 ;
19: tir := *lshift*(tir) ; **if** n **then goto** 25 ; {010x o 011x?}
20: alu := tir ; **if** n **then goto** 23 ; {0100 o 0101?}
21: alu := ac ; **if** n **then goto** 0 ; {0100 = **JPOS**}
22: pc := *band*(ir , amask) ; **goto** 0 ; {realiza el salto}
23: alu := ac ; **if** z **then goto** 22 ; {0101 = **JZER**}
24: **goto** 0 ; {no se produce el salto}
25: alu := tir ; **if** n **then goto** 27 ; {0110 o 0111?}
26: pc := *band*(ir , amask) ; **goto** 0 ; {0110 = **JUMP**}
27: ac := *band*(ir , amask) ; **goto** 0 ; {0111 = **LOCO**}
28: tir := *lshift*(ir + ir) ; **if** n **then goto** 40 ; {10xx o 11xx?}
29: tir := *lshift*(tir) ; **if** n **then goto** 35 ; {100x o 101x?}
30: alu := tir ; **if** n **then goto** 33 ; {1000 o 1001?}
31: a := ir + sp ; {1000 = **LODL**}
32: mar := a ; *rd* ; **goto** 7 ;
33: a := ir + sp ; {1001 = **STOL**}
34: mar := a ; mbr := ac ; *wr* ; **goto** 10 ;
35: alu := tir ; **if** n **then goto** 38 ; {1010 o 1011?}
36: a := ir + sp ; {1010 = **ADDL**}
37: mar := a ; *rd* ; **goto** 13 ;
38: a := ir + sp ; {1011 = **SUBL**}
39: mar := a ; *rd* ; **goto** 16 ;

48: mar := sp ; mbr := pc ; *wr* ;
49: pc := *band*(ir , amask) ; *wr* ; **goto** 0 ;
50: tir := *lshift*(tir) ; **if** n **then goto** 65 ; {111, examina addr}
51: tir := *lshift*(tir) ; **if** n **then goto** 59 ;
52: alu := tir ; **if** n **then goto** 56 ;
53: mar := ac ; rd ; {1111000 = **PSHI**}
54: sp := sp + (-1) ; rd ;
55: mar := sp ; *wr* ; **goto** 10 ;
56: mar := sp ; sp := sp + 1 ; *rd* ; {1111001 = **POPI**}
57: *rd* ;
58: mar := ac ; *wr* ; **goto** 10 ;
59: alu := tir ; **if** n **then goto** 62 ;
60: sp := sp + (-1) ; {1111010 = **PUSH**}
61: mar := sp ; mbr := ac ; *wr* ; **goto** 10 ;
62: mar := sp ; sp := sp + 1 ; *rd* ; {1111011 = **POP**}
63: *rd* ;
64: ac := mbr ; **goto** 0 ;
65: tir := *lshift*(tir) ; **if** n **then goto** 73 ;
66: alu := tir ; **if** n **then goto** 70 ;
67: mar := sp ; sp := sp + 1 ; *rd* ; {1111100 = **RETN**}
68: *rd* ;
69: pc := mbr ; **goto** 0 ;
70: a := ac ; {1111101 = **SWAP**}
71: ac := sp ;
72: sp := a ; **goto** 0 ;
73: alu := tir ; **if** n **then goto** 76 ;
74: a := *band*(ir , smask) ; {1111110 = **INSP**}
75: sp := sp + a ; **goto** 0 ;
76: a := *band*(ir , smask) ; {1111111 = **DESP**}
77: a := *inv*(a) ;
78: a := a + 1 ; **goto** 75 ;

```
0: mar := pc ; rd ;                                  {ciclo principal}
1: pc := pc + 1 ; rd ;                               {incrementa pc}
2: ir := mbr ; if n then goto 28 ;                   {salva y decodifica mbr}
3: tir := lshift(ir + ir) ; if n then goto 19 ;
4: tir := lshift(tir) ; if n then goto 11 ;          {000x o 001x?}
5: alu := tir ; if n then goto 9 ;                   {0000 o 0001?}
6: mar := ir ; rd ;                                  {0000 = LODD}
7: rd ;
8: ac := mbr ; goto 0 ;
9: mar := ir ; mbr := ac ; wr ;                      {0001 = STOD}
10: wr ; goto 0 ;
11: alu := tir ; if n then goto 15 ;                 {0010 o 0011?}
12: mar := ir ; rd ;                                 {0010 = ADDD}
13: rd ;
14: ac := mbr + ac ; goto 0 ;
15: mar := ir ; rd ;                                 {0011 = SUBD}
16: ac := ac + 1 ; rd ;                              {Nota: x - y = x + 1 + no y}
17: a := inv(mbr) ;
18: ac := ac + a ; goto 0 ;
19: tir := lshift(tir) ; if n then goto 25 ;         {010x o 011x?}
20: alu := tir ; if n then goto 23 ;                 {0100 o 0101?}
21: alu := ac ; if n then goto 0 ;                   {0100 = JPOS}
22: pc := band(ir , amask) ; goto 0 ;                {realiza el salto}
23: alu := ac ; if z then goto 22 ;                  {0101 = JZER}
24: goto 0 ;                                         {no se produce el salto}
25: alu := tir ; if n then goto 27 ;                 {0110 o 0111?}
26: pc := band(ir , amask) ; goto 0 ;                {0110 = JUMP}
27: ac := band(ir , amask) ; goto 0 ;                {0111 = LOCO}
28: tir := lshift(ir + ir) ; if n then goto 40 ;     {10xx o 11xx?}
29: tir := lshift(tir) ; if n then goto 35 ;         {100x o 101x?}
30: alu := tir ; if n then goto 33 ;                 {1000 o 1001?}
31: a := ir + sp ;                                   {1000 = LODL}
32: mar := a ; rd ; goto 7 ;
33: a := ir + sp ;                                   {1001 = STOL}
34: mar := a ; mbr := ac ; wr ; goto 10 ;
35: alu := tir ; if n then goto 38 ;                 {1010 o 1011?}
36: a := ir + sp ;                                   {1010 = ADDL}
37: mar := a ; rd ; goto 13 ;
38: a := ir + sp ;                                   {1011 = SUBL}
39: mar := a ; rd ; goto 16 ;
```

Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxxxxxxxxxx	ADDD	Add direct	ac:=ac+m[x]
0011xxxxxxxxxxxx	SUBD	Subtract direct	ac:=ac-m[x]
0100xxxxxxxxxxxx	JPOS	Jump positive	if ac>=0 then pc:=x
0101xxxxxxxxxxxx	JZER	Jump zero	if ac=0 then pc:=x
0110xxxxxxxxxxxx	JUMP	Jump	pc:=x
0111xxxxxxxxxxxx	LOCO	Load constant	ac:=x (0<=x<=4095)
1000xxxxxxxxxxxx	LODL	Load local	ac:=m[sp+x]
1001xxxxxxxxxxxx	STOL	Store local	m[x+sp]:=ac
1010xxxxxxxxxxxx	ADDL	Add local	ac:=ac+m[sp+x]
1011xxxxxxxxxxxx	SUBL	Subtract local	ac:=ac-m[sp+x]
1100xxxxxxxxxxxx	JNEG	Jump negative	if ac<0 then pc:=x
1101xxxxxxxxxxxx	JNZE	Jump nonzero	if ac<>0 then pc:=x
1110xxxxxxxxxxxx	CALL	Call procedure	sp:=sp-1; m[sp]:=pc; pc:=x
1111000000000000	PSHI	Push indirect	sp:=sp-1; m[sp]:=m[ac]
1111001000000000	POPI	Pop indirect	m[ac]:=m[sp]; sp:=sp+1
1111010000000000	PUSH	Push onto stack	sp:=sp-1; m[sp]:=ac
1111011000000000	POP	Pop from stack	ac:=m[sp]; sp:=sp+ 1
1111100000000000	RETN	Return	pc:=m[sp]; sp:=sp+ 1
1111101000000000	SWAP	Swap ac sp	tmp:=ac; ac:=sp; sp:=tmp
11111100yyyyyyyy	INSP	Increment sp	sp:=sp+y (0<=y<=255)
11111110yyyyyyyy	DESP	Decrement sp	sp:=sp-y (0<=y<=255)

Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxxxxxxxxxx	ADDD	Add direct	ac:=ac+m[x]
0011xxxxxxxxxxxx	SUBD	Subtract direct	ac:=ac-m[x]
0100xxxxxxxxxxxx	JPOS	Jump positive	if ac>=0 then pc:=x
0101xxxxxxxxxxxx	JZER	Jump zero	if ac=0 then pc:=x
0110xxxxxxxxxxxx	JUMP	Jump	pc:=x
0111xxxxxxxxxxxx	LOCO	Load constant	ac:=x (0<=x<=4095)
1000xxxxxxxxxxxx	LODL	Load local	ac:=m[sp+x]
1001xxxxxxxxxxxx	STOL	Store local	m[x+sp]:=ac
1010xxxxxxxxxxxx	ADDL	Add local	ac:=ac+m[sp+x]
1011xxxxxxxxxxxx	SUBL	Subtract local	ac:=ac-m[sp+x]
1100xxxxxxxxxxxx	JNEG	Jump negative	if ac<0 then pc:=x
1101xxxxxxxxxxxx	JNZE	Jump nonzero	if ac<>0 then pc:=x
1110xxxxxxxxxxxx	CALL	Call procedure	sp:=sp-1; m[sp]:=pc; pc:=x
1111000000000000	PSHI	Push indirect	sp:=sp-1; m[sp]:=m[ac]
1111001000000000	POPI	Pop indirect	m[ac]:=m[sp]; sp:=sp+1
1111010000000000	PUSH	Push onto stack	sp:=sp-1; m[sp]:=ac
1111011000000000	POP	Pop from stack	ac:=m[sp]; sp:=sp+ 1
1111100000000000	RETN	Return	pc:=m[sp]; sp:=sp+ 1
1111101000000000	SWAP	Swap ac sp	tmp:=ac; ac:=sp; sp:=tmp
11111100yyyyyyyy	INSP	Increment sp	sp:=sp+y (0<=y<=255)
11111110yyyyyyyy	DESP	Decrement sp	sp:=sp-y (0<=y<=255)

```
40: tir := lshift(tir) ; if n then goto 46 ;        {110x o 111x?}
41: alu := tir ; if n then goto 44 ;                {1100 o 1101?}
42: alu := ac ; if n then goto 22 ;                 {1100 = JNEG}
43: goto 0 ;
44: alu := ac ; if z then goto 0 ;                  {1101 = JNZE}
45: pc := band(ir , amask) ; goto 0 ;
46: tir := lshift(tir) ; if n then goto 50 ;
47: sp := sp + (-1) ;                               {1110 = CALL}
48: mar := sp ; mbr := pc ; wr ;
49: pc := band(ir , amask) ; wr ; goto 0 ;
50: tir := lshift(tir) ; if n then goto 65 ;        {111, examina addr}
51: tir := lshift(tir) ; if n then goto 59 ;
52: alu := tir ; if n then goto 56 ;
53: mar := ac ; rd ;                                {1111000 = PSHI}
54: sp := sp + (-1) ; rd ;
55: mar := sp ; wr ; goto 10 ;
56: mar := sp ; sp := sp + 1 ; rd ;                 {1111001 = POPI}
57: rd ;
58: mar := ac ; wr ; goto 10 ;
59: alu := tir ; if n then goto 62 ;
60: sp := sp + (-1) ;                               {1111010 = PUSH}
61: mar := sp ; mbr := ac ; wr ; goto 10 ;
62: mar := sp ; sp := sp + 1 ; rd ;                 {1111011 = POP}
63: rd ;
64: ac := mbr ; goto 0 ;
65: tir := lshift(tir) ; if n then goto 73 ;
66: alu := tir ; if n then goto 70 ;
67: mar := sp ; sp := sp + 1 ; rd ;                 {1111100 = RETN}
68: rd ;
69: pc := mbr ; goto 0 ;
70: a := ac ;                                       {1111101 = SWAP}
71: ac := sp ;
72: sp := a ; goto 0 ;
73: alu := tir ; if n then goto 76 ;
74: a := band(ir , smask) ;                         {1111110 = INSP}
75: sp := sp + a ; goto 0 ;
76: a := band(ir , smask) ;                         {1111111 = DESP}
77: a := inv(a) ;
78: a := a + 1 ; goto 75 ;
```

```
0: mar := pc ; rd ;                                  {ciclo principal}
1: pc := pc + 1 ; rd ;                               {incrementa pc}
2: ir := mbr ; if n then goto 28 ;                   {salva y decodifica mbr}
3: tir := lshift(ir + ir) ; if n then goto 19 ;
4: tir := lshift(tir) ; if n then goto 11 ;          {000x o 001x?}
5: alu := tir ; if n then goto 9 ;                   {0000 o 0001?}
6: mar := ir ; rd ;                                  {0000 = LODD}
7: rd ;
8: ac := mbr ; goto 0 ;
9: mar := ir ; mbr := ac ; wr ;                      {0001 = STOD}
10: wr ; goto 0 ;
11: alu := tir ; if n then goto 15 ;                 {0010 o 0011?}
12: mar := ir ; rd ;                                 {0010 = ADDD}
13: rd ;
14: ac := mbr + ac ; goto 0 ;
15: mar := ir ; rd ;                                 {0011 = SUBD}
16: ac := ac + 1 ; rd ;                              {Nota: x - y = x + 1 + no y}
17: a := inv(mbr) ;
18: ac := ac + a ; goto 0 ;
19: tir := lshift(tir) ; if n then goto 25 ;         {010x o 011x?}
20: alu := tir ; if n then goto 23 ;                 {0100 o 0101?}
21: alu := ac ; if n then goto 0 ;                   {0100 = JPOS}
22: pc := band(ir , amask) ; goto 0 ;                {realiza el salto}
23: alu := ac ; if z then goto 22 ;                  {0101 = JZER}
24: goto 0 ;                                         {no se produce el salto}
25: alu := tir ; if n then goto 27 ;                 {0110 o 0111?}
26: pc := band(ir , amask) ; goto 0 ;                {0110 = JUMP}
27: ac := band(ir , amask) ; goto 0 ;                {0111 = LOCO}
28: tir := lshift(ir + ir) ; if n then goto 40 ;     {10xx o 11xx?}
29: tir := lshift(tir) ; if n then goto 35 ;         {100x o 101x?}
30: alu := tir ; if n then goto 33 ;                 {1000 o 1001?}
31: a := ir + sp ;                                   {1000 = LODL}
32: mar := a ; rd ; goto 7 ;
33: a := ir + sp ;                                   {1001 = STOL}
34: mar := a ; mbr := ac ; wr ; goto 10 ;
35: alu := tir ; if n then goto 38 ;                 {1010 o 1011?}
36: a := ir + sp ;                                   {1010 = ADDL}
37: mar := a ; rd ; goto 13 ;
38: a := ir + sp ;                                   {1011 = SUBL}
39: mar := a ; rd ; goto 16 ;
```

Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxxxxxxxxxx	ADDD	Add direct	ac:=ac+m[x]
0011xxxxxxxxxxxx	SUBD	Subtract direct	ac:=ac-m[x]
0100xxxxxxxxxxxx	JPOS	Jump positive	if ac>=0 then pc:=x
0101xxxxxxxxxxxx	JZER	Jump zero	if ac=0 then pc:=x
0110xxxxxxxxxxxx	JUMP	Jump	pc:=x
0111xxxxxxxxxxxx	LOCO	Load constant	ac:=x (0<=x<=4095)
1000xxxxxxxxxxxx	LODL	Load local	ac:=m[sp+x]
1001xxxxxxxxxxxx	STOL	Store local	m[x+sp]:=ac
1010xxxxxxxxxxxx	ADDL	Add local	ac:=ac+m[sp+x]
1011xxxxxxxxxxxx	SUBL	Subtract local	ac:=ac-m[sp+x]
1100xxxxxxxxxxxx	JNEG	Jump negative	if ac<0 then pc:=x
1101xxxxxxxxxxxx	JNZE	Jump nonzero	if ac<>0 then pc:=x
1110xxxxxxxxxxxx	CALL	Call procedure	sp:=sp-1; m[sp]:=pc; pc:=x
1111000000000000	PSHI	Push indirect	sp:=sp-1; m[sp]:=m[ac]
1111001000000000	POPI	Pop indirect	m[ac]:=m[sp]; sp:=sp+1
1111010000000000	PUSH	Push onto stack	sp:=sp-1; m[sp]:=ac
1111011000000000	POP	Pop from stack	ac:=m[sp]; sp:=sp+ 1
1111100000000000	RETN	Return	pc:=m[sp]; sp:=sp+ 1
1111101000000000	SWAP	Swap ac sp	tmp:=ac; ac:=sp; sp:=tmp
11111100yyyyyyyy	INSP	Increment sp	sp:=sp+y (0<=y<=255)
11111110yyyyyyyy	DESP	Decrement sp	sp:=sp-y (0<=y<=255)

```
0: mar := pc ; rd ;                                  {ciclo principal}
1: pc := pc + 1 ; rd ;                               {incrementa pc}
2: ir := mbr ; if n then goto 28 ;                   {salva y decodifica mbr}
3: tir := lshift(ir + ir) ; if n then goto 19 ;
4: tir := lshift(tir) ; if n then goto 11 ;          {000x o 001x?}
5: alu := tir ; if n then goto 9 ;                   {0000 o 0001?}
6: mar := ir ; rd ;                                  {0000 = LODD}
7: rd ;
8: ac := mbr ; goto 0 ;
9: mar := ir ; mbr := ac ; wr ;                      {0001 = STOD}
10: wr ; goto 0 ;
11: alu := tir ; if n then goto 15 ;                 {0010 o 0011?}
12: mar := ir ; rd ;                                 {0010 = ADDD}
13: rd ;
14: ac := mbr + ac ; goto 0 ;
15: mar := ir ; rd ;                                 {0011 = SUBD}
16: ac := ac + 1 ; rd ;                              {Nota: x - y = x + 1 + no y}
17: a := inv(mbr) ;
18: ac := ac + a ; goto 0 ;
19: tir := lshift(tir) ; if n then goto 25 ;         {010x o 011x?}
20: alu := tir ; if n then goto 23 ;                 {0100 o 0101?}
21: alu := ac ; if n then goto 0 ;                   {0100 = JPOS}
22: pc := band(ir , amask) ; goto 0 ;                {realiza el salto}
23: alu := ac ; if z then goto 22 ;                  {0101 = JZER}
24: goto 0 ;                                         {no se produce el salto}
25: alu := tir ; if n then goto 27 ;                 {0110 o 0111?}
26: pc := band(ir , amask) ; goto 0 ;                {0110 = JUMP}
27: ac := band(ir , amask) ; goto 0 ;                {0111 = LOCO}
28: tir := lshift(ir + ir) ; if n then goto 40 ;     {10xx o 11xx?}
29: tir := lshift(tir) ; if n then goto 35 ;         {100x o 101x?}
30: alu := tir ; if n then goto 33 ;                 {1000 o 1001?}
31: a := ir + sp ;                                   {1000 = LODL}
32: mar := a ; rd ; goto 7 ;
33: a := ir + sp ;                                   {1001 = STOL}
34: mar := a ; mbr := ac ; wr ; goto 10 ;
35: alu := tir ; if n then goto 38 ;                 {1010 o 1011?}
36: a := ir + sp ;                                   {1010 = ADDL}
37: mar := a ; rd ; goto 13 ;
38: a := ir + sp ;                                   {1011 = SUBL}
39: mar := a ; rd ; goto 16 ;
```

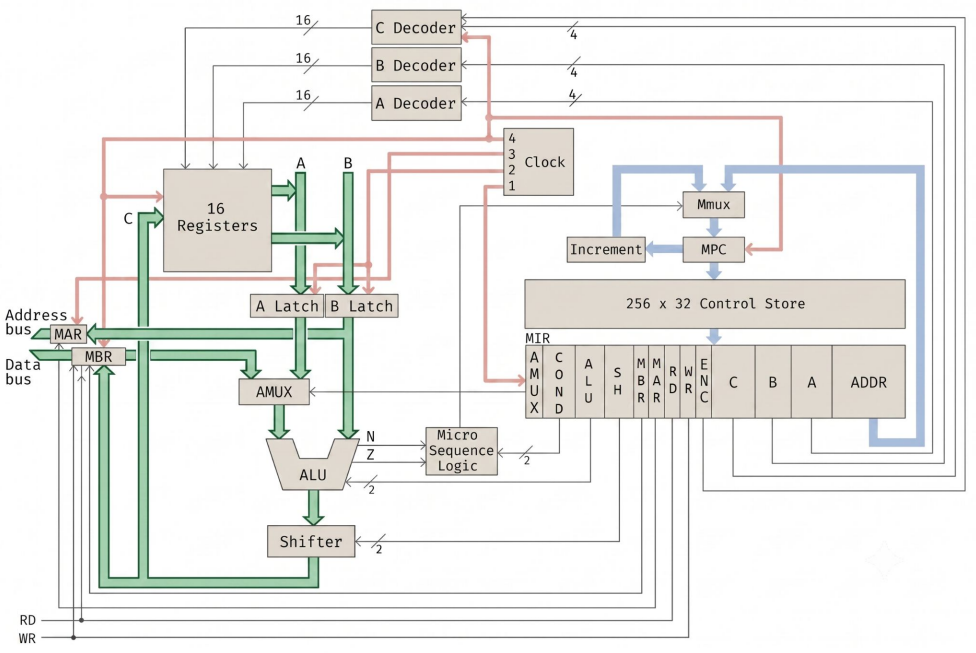
Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxx			]]
0011xxxx			]
0100xxxx			hen pc:=x
0101xxxx			en pc:=x
0110xxxx			
0111xxxx			x<=4095)
1000xxxx			]
1001xxxx			c
1010xxxx			p+x]
1011xxxx			p+x]
1100xxxxxxxxxxxx	JNEG	Jump negative	if ac<0 then pc:=x
1101xxxxxxxxxxxx	JNZE	Jump nonzero	if ac<>0 then pc:=x
1110xxxxxxxxxxxx	CALL	Call procedure	sp:=sp-1; m[sp]:=pc; pc:=x
1111000000000000	PSHI	Push indirect	sp:=sp-1; m[sp]:=m[ac]
1111001000000000	POPI	Pop indirect	m[ac]:=m[sp]; sp:=sp+1
1111010000000000	PUSH	Push onto stack	sp:=sp-1; m[sp]:=ac
1111011000000000	POP	Pop from stack	ac:=m[sp]; sp:=sp+ 1
1111100000000000	RETN	Return	pc:=m[sp]; sp:=sp+ 1
1111101000000000	SWAP	Swap ac sp	tmp:=ac; ac:=sp; sp:=tmp
11111100yyyyyyyy	INSP	Increment sp	sp:=sp+y (0<=y<=255)
11111110yyyyyyyy	DESP	Decrement sp	sp:=sp-y (0<=y<=255)

```
3F7  . . .
3F8  STOD 9
3F9  JUMP 3F7
3FA  . . .
```

```
0: mar := pc ; rd ;                                  {ciclo principal}
1: pc := pc + 1 ; rd ;                               {incrementa pc}
2: ir := mbr ; if n then goto 28 ;                   {salva y decodifica mbr}
3: tir := lshift(ir + ir) ; if n then goto 19 ;
4: tir := lshift(tir) ; if n then goto 11 ;          {000x o 001x?}
5: alu := tir ; if n then goto 9 ;                   {0000 o 0001?}
6: mar := ir ; rd ;                                  {0000 = LODD}
7: rd ;
8: ac := mbr ; goto 0 ;
9: mar := ir ; mbr := ac ; wr ;                      {0001 = STOD}
10: wr ; goto 0 ;
11: alu := tir ; if n then goto 15 ;                 {0010 o 0011?}
12: mar := ir ; rd ;                                 {0010 = ADDD}
13: rd ;
14: ac := mbr + ac ; goto 0 ;
15: mar := ir ; rd ;                                 {0011 = SUBD}
16: ac := ac + 1 ; rd ;                              {Nota: x - y = x + 1 + no y}
17: a := inv(mbr) ;
18: ac := ac + a ; goto 0 ;
19: tir := lshift(tir) ; if n then goto 25 ;         {010x o 011x?}
20: alu := tir ; if n then goto 23 ;                 {0100 o 0101?}
21: alu := ac ; if n then goto 0 ;                   {0100 = JPOS}
22: pc := band(ir , amask) ; goto 0 ;                {realiza el salto}
23: alu := ac ; if z then goto 22 ;                  {0101 = JZER}
24: goto 0 ;                                         {no se produce el salto}
25: alu := tir ; if n then goto 27 ;                 {0110 o 0111?}
26: pc := band(ir , amask) ; goto 0 ;                {0110 = JUMP}
27: ac := band(ir , amask) ; goto 0 ;                {0111 = LOCO}
28: tir := lshift(ir + ir) ; if n then goto 40 ;     {10xx o 11xx?}
29: tir := lshift(tir) ; if n then goto 35 ;         {100x o 101x?}
30: alu := tir ; if n then goto 33 ;                 {1000 o 1001?}
31: a := ir + sp ;                                   {1000 = LODL}
32: mar := a ; rd ; goto 7 ;
33: a := ir + sp ;                                   {1001 = STOL}
34: mar := a ; mbr := ac ; wr ; goto 10 ;
35: alu := tir ; if n then goto 38 ;                 {1010 o 1011?}
36: a := ir + sp ;                                   {1010 = ADDL}
37: mar := a ; rd ; goto 13 ;
38: a := ir + sp ;                                   {1011 = SUBL}
39: mar := a ; rd ; goto 16 ;
```

Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxx			]
0011xxxx			]
0100xxxx			hen pc:=x
0101xxxx			en pc:=x
0110xxxx			
0111xxxx			x<=4095)
1000xxxx			]
1001xxxx			c
1010xxxx			p+x]
1011xxxx			p+x]

```
3F7  . . .
3F8  STOD  9
3F9  JUMP  3F7
3FA  . . .
```

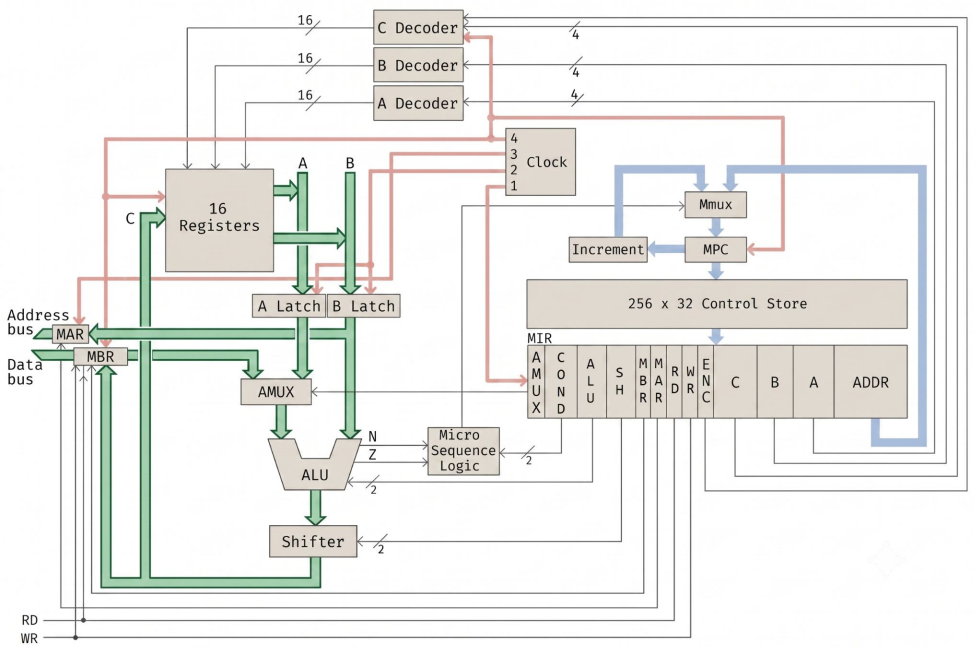


```
0: mar := pc ; rd ;                                  {ciclo principal}
1: pc := pc + 1 ; rd ;                               {incrementa pc}
2: ir := mbr ; if n then goto 28 ;                   {salva y decodifica mbr}
3: tir := lshift(ir + ir) ; if n then goto 19 ;
4: tir := lshift(tir) ; if n then goto 11 ;          {000x o 001x?}
5: alu := tir ; if n then goto 9 ;                   {0000 o 0001?}
6: mar := ir ; rd ;                                  {0000 = LODD}
7: rd ;
8: ac := mbr ; goto 0 ;
9: mar := ir ; mbr := ac ; wr ;                      {0001 = STOD}
10: wr ; goto 0 ;
11: alu := tir ; if n then goto 15 ;                 {0010 o 0011?}
12: mar := ir ; rd ;                                 {0010 = ADDD}
13: rd ;
14: ac := mbr + ac ; goto 0 ;
15: mar := ir ; rd ;                                 {0011 = SUBD}
16: ac := ac + 1 ; rd ;                              {Nota: x - y = x + 1 + no y}
17: a := inv(mbr) ;
18: ac := ac + a ; goto 0 ;
19: tir := lshift(tir) ; if n then goto 25 ;         {010x o 011x?}
20: alu := tir ; if n then goto 23 ;                 {0100 o 0101?}
21: alu := ac ; if n then goto 0 ;                   {0100 = JPOS}
22: pc := band(ir , amask) ; goto 0 ;                {realiza el salto}
23: alu := ac ; if z then goto 22 ;                  {0101 = JZER}
24: goto 0 ;                                         {no se produce el salto}
25: alu := tir ; if n then goto 27 ;                 {0110 o 0111?}
26: pc := band(ir , amask) ; goto 0 ;                {0110 = JUMP}
27: ac := band(ir , amask) ; goto 0 ;                {0111 = LOCO}
28: tir := lshift(ir + ir) ; if n then goto 40 ;     {10xx o 11xx?}
29: tir := lshift(tir) ; if n then goto 35 ;         {100x o 101x?}
```

MAR	MBR	PC	AC	IR	TIR	ALU	SHR	N	Z
		3F8	00 41						

Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxx			]
0011xxxx			]
0100xxxx			hen pc:=x
0101xxxx			en pc:=x
0110xxxx			
0111xxxx			x<=4095)
1000xxxx			]
1001xxxx			c
1010xxxx			p+x]
1011xxxx			p+x]

```
3F7  . . .
3F8  STOD  9
3F9  JUMP  3F7
3FA  . . .
```



```
0: mar := pc ; rd ;                                  {ciclo principal}
1: pc := pc + 1 ; rd ;                               {incrementa pc}
2: ir := mbr ; if n then goto 28 ;                   {salva y decodifica mbr}
3: tir := lshift(ir + ir) ; if n then goto 19 ;
4: tir := lshift(tir) ; if n then goto 11 ;          {000x o 001x?}
5: alu := tir ; if n then goto 9 ;                   {0000 o 0001?}
6: mar := ir ; rd ;                                  {0000 = LODD}
7: rd ;
8: ac := mbr ; goto 0 ;
9: mar := ir ; mbr := ac ; wr ;                      {0001 = STOD}
10: wr ; goto 0 ;
11: alu := tir ; if n then goto 15 ;                 {0010 o 0011?}
12: mar := ir ; rd ;                                 {0010 = ADDD}
13: rd ;
14: ac := mbr + ac ; goto 0 ;
15: mar := ir ; rd ;                                 {0011 = SUBD}
16: ac := ac + 1 ; rd ;                              {Nota: x - y = x + 1 + no y}
17: a := inv(mbr) ;
18: ac := ac + a ; goto 0 ;
19: tir := lshift(tir) ; if n then goto 25 ;         {010x o 011x?}
20: alu := tir ; if n then goto 23 ;                 {0100 o 0101?}
21: alu := ac ; if n then goto 0 ;                   {0100 = JPOS}
22: pc := band(ir , amask) ; goto 0 ;                {realiza el salto}
23: alu := ac ; if z then goto 22 ;                  {0101 = JZER}
24: goto 0 ;                                         {no se produce el salto}
25: alu := tir ; if n then goto 27 ;                 {0110 o 0111?}
26: pc := band(ir , amask) ; goto 0 ;                {0110 = JUMP}
27: ac := band(ir , amask) ; goto 0 ;                {0111 = LOCO}
28: tir := lshift(ir + ir) ; if n then goto 40 ;     {10xx o 11xx?}
29: tir := lshift(tir) ; if n then goto 35 ;         {100x o 101x?}
```

MAR	MBR	PC	AC	IR	TIR	ALU	SHR	N	Z
		3F8	00 41						

Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxx			]
0011xxxx			]
0100xxxx			hen pc:=x
0101xxxx			en pc:=x
0110xxxx			
0111xxxx			x<=4095)
1000xxxx			]
1001xxxx			c
1010xxxx			p+x]
1011xxxx			p+x]

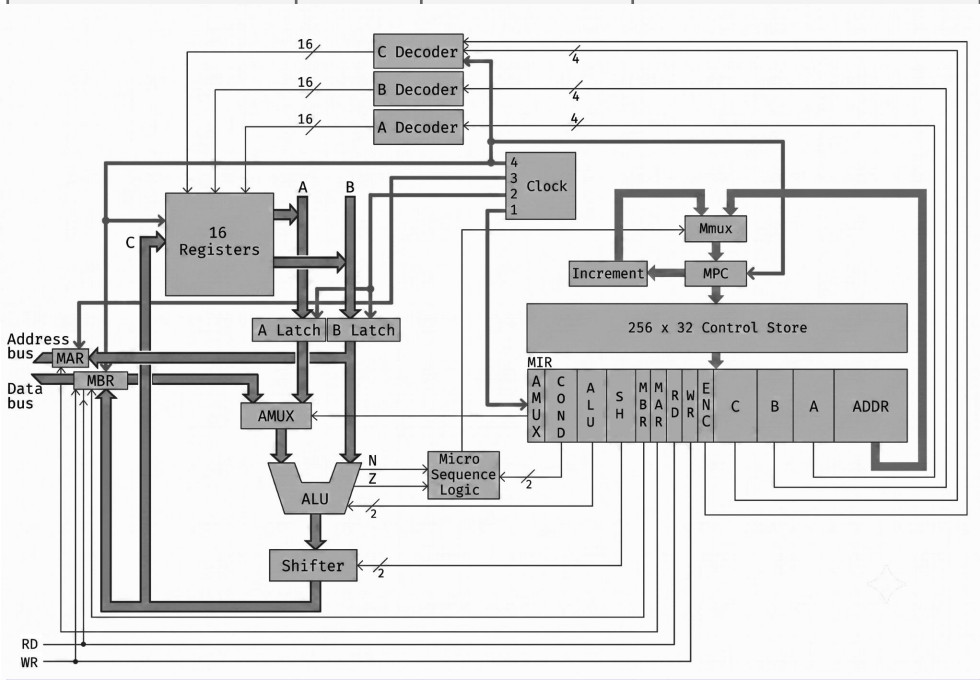
STORE DIRECT

3F8 **STOD** 9

Guardar el valor del AC en la memoria(9)

- mar:=ir
- mbr:=ac
- wr

0001 0000 0000 1001



```
0: mar := pc ; rd ;                                  {ciclo principal}
1: pc := pc + 1 ; rd ;                               {incrementa pc}
2: ir := mbr ; if n then goto 28 ;                   {salva y decodifica mbr}
3: tir := lshift(ir + ir) ; if n then goto 19 ;
4: tir := lshift(tir) ; if n then goto 11 ;          {000x o 001x?}
5: alu := tir ; if n then goto 9 ;                   {0000 o 0001?}
6: mar := ir ; rd ;                                  {0000 = LODD}
7: rd ;
8: ac := mbr ; goto 0 ;
9: mar := ir ; mbr := ac ; wr ;                      {0001 = STOD}
10: wr ; goto 0 ;
11: alu := tir ; if n then goto 15 ;                 {0010 o 0011?}
12: mar := ir ; rd ;                                 {0010 = ADDD}
13: rd ;
14: ac := mbr + ac ; goto 0 ;
15: mar := ir ; rd ;                                 {0011 = SUBD}
16: ac := ac + 1 ; rd ;                              {Nota: x - y = x + 1 + no y}
17: a := inv(mbr) ;
18: ac := ac + a ; goto 0 ;
19: tir := lshift(tir) ; if n then goto 25 ;         {010x o 011x?}
20: alu := tir ; if n then goto 23 ;                 {0100 o 0101?}
21: alu := ac ; if n then goto 0 ;                   {0100 = JPOS}
22: pc := band(ir , amask) ; goto 0 ;                {realiza el salto}
23: alu := ac ; if z then goto 22 ;                  {0101 = JZER}
24: goto 0 ;                                         {no se produce el salto}
25: alu := tir ; if n then goto 27 ;                 {0110 o 0111?}
26: pc := band(ir , amask) ; goto 0 ;                {0110 = JUMP}
27: ac := band(ir , amask) ; goto 0 ;                {0111 = LOCO}
28: tir := lshift(ir + ir) ; if n then goto 40 ;     {10xx o 11xx?}
29: tir := lshift(tir) ; if n then goto 35 ;         {100x o 101x?}
```

MAR	MBR	PC	AC	IR	TIR	ALU	SHR	N	Z
3F8		3F8	00 41						

Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxx			]
0011xxxx			]
0100xxxx			hen pc:=x
0101xxxx			en pc:=x
0110xxxx			
0111xxxx			x<=4095)
1000xxxx			]
1001xxxx			c
1010xxxx			p+x]
1011xxxx			p+x]

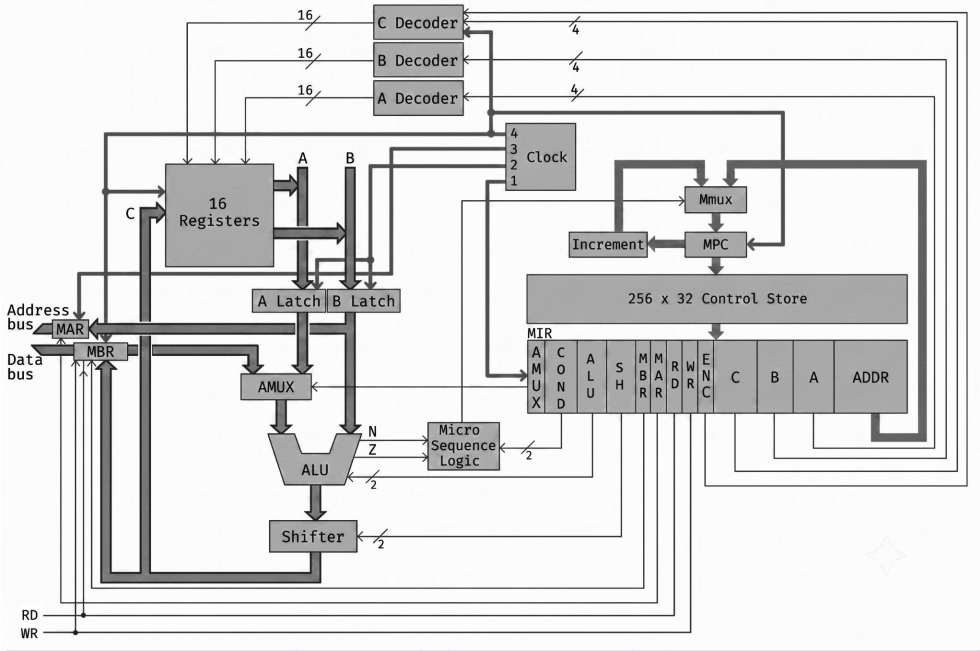
STORE DIRECT

3F8 **STOD** 9

Guardar el valor del AC en la memoria(9)

- mar:=ir
- mbr:=ac
- wr

0001 0000 0000 1001



```
0: mar := pc ; rd ;                                  {ciclo principal}
1: pc := pc + 1 ; rd ;                               {incrementa pc}
2: ir := mbr ; if n then goto 28 ;                   {salva y decodifica mbr}
3: tir := lshift(ir + ir) ; if n then goto 19 ;
4: tir := lshift(tir) ; if n then goto 11 ;          {000x o 001x?}
5: alu := tir ; if n then goto 9 ;                   {0000 o 0001?}
6: mar := ir ; rd ;                                  {0000 = LODD}
7: rd ;
8: ac := mbr ; goto 0 ;
9: mar := ir ; mbr := ac ; wr ;                      {0001 = STOD}
10: wr ; goto 0 ;
11: alu := tir ; if n then goto 15 ;                 {0010 o 0011?}
12: mar := ir ; rd ;                                 {0010 = ADDD}
13: rd ;
14: ac := mbr + ac ; goto 0 ;
15: mar := ir ; rd ;                                 {0011 = SUBD}
16: ac := ac + 1 ; rd ;                              {Nota: x - y = x + 1 + no y}
17: a := inv(mbr) ;
18: ac := ac + a ; goto 0 ;
19: tir := lshift(tir) ; if n then goto 25 ;         {010x o 011x?}
20: alu := tir ; if n then goto 23 ;                 {0100 o 0101?}
21: alu := ac ; if n then goto 0 ;                   {0100 = JPOS}
22: pc := band(ir , amask) ; goto 0 ;                {realiza el salto}
23: alu := ac ; if z then goto 22 ;                  {0101 = JZER}
24: goto 0 ;                                         {no se produce el salto}
25: alu := tir ; if n then goto 27 ;                 {0110 o 0111?}
26: pc := band(ir , amask) ; goto 0 ;                {0110 = JUMP}
27: ac := band(ir , amask) ; goto 0 ;                {0111 = LOCO}
28: tir := lshift(ir + ir) ; if n then goto 40 ;     {10xx o 11xx?}
29: tir := lshift(tir) ; if n then goto 35 ;         {100x o 101x?}
```

MAR	MBR	PC	AC	IR	TIR	ALU	SHR	N	Z
3F8	10 09	3F9	00 41						

Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxx			]
0011xxxx			]
0100xxxx			hen pc:=x
0101xxxx			en pc:=x
0110xxxx			
0111xxxx			x<=4095)
1000xxxx			]
1001xxxx			c
1010xxxx			p+x]
1011xxxx			p+x]

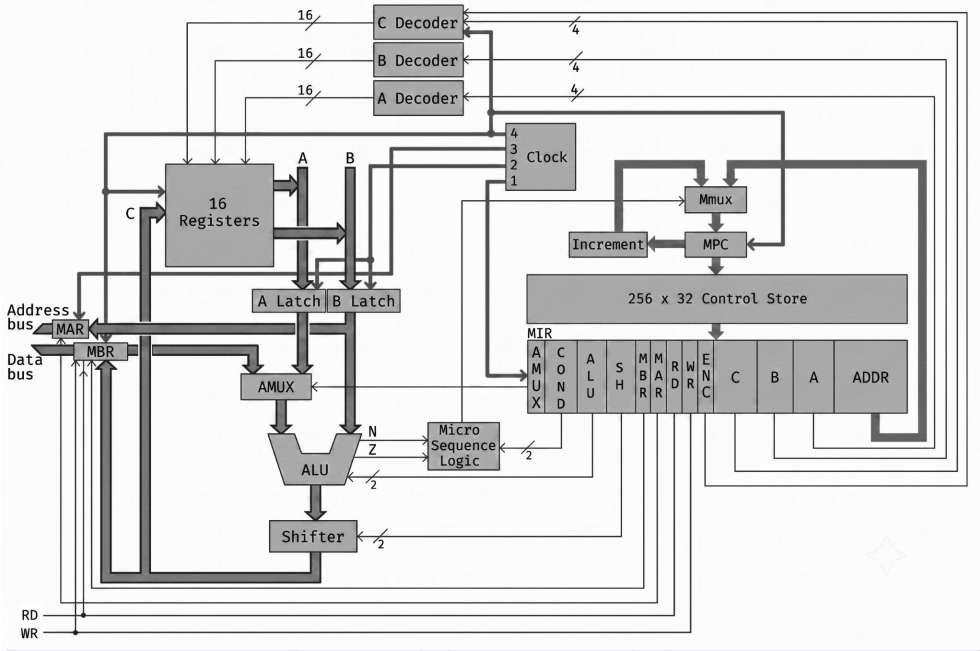
STORE DIRECT

3F8 **STOD** 9

Guardar el valor del AC en la memoria(9)

- mar:=ir
- mbr:=ac
- wr

0001 0000 0000 1001



```
0: mar := pc ; rd ;                                  {ciclo principal}
1: pc := pc + 1 ; rd ;                               {incrementa pc}
2: ir := mbr ; if n then goto 28 ;                   {salva y decodifica mbr}
3: tir := lshift(ir + ir) ; if n then goto 19 ;
4: tir := lshift(tir) ; if n then goto 11 ;          {000x o 001x?}
5: alu := tir ; if n then goto 9 ;                   {0000 o 0001?}
6: mar := ir ; rd ;                                  {0000 = LODD}
7: rd ;
8: ac := mbr ; goto 0 ;
9: mar := ir ; mbr := ac ; wr ;                      {0001 = STOD}
10: wr ; goto 0 ;
11: alu := tir ; if n then goto 15 ;                 {0010 o 0011?}
12: mar := ir ; rd ;                                 {0010 = ADDD}
13: rd ;
14: ac := mbr + ac ; goto 0 ;
15: mar := ir ; rd ;                                 {0011 = SUBD}
16: ac := ac + 1 ; rd ;                              {Nota: x - y = x + 1 + no y}
17: a := inv(mbr) ;
18: ac := ac + a ; goto 0 ;
19: tir := lshift(tir) ; if n then goto 25 ;         {010x o 011x?}
20: alu := tir ; if n then goto 23 ;                 {0100 o 0101?}
21: alu := ac ; if n then goto 0 ;                   {0100 = JPOS}
22: pc := band(ir , amask) ; goto 0 ;                {realiza el salto}
23: alu := ac ; if z then goto 22 ;                  {0101 = JZER}
24: goto 0 ;                                         {no se produce el salto}
25: alu := tir ; if n then goto 27 ;                 {0110 o 0111?}
26: pc := band(ir , amask) ; goto 0 ;                {0110 = JUMP}
27: ac := band(ir , amask) ; goto 0 ;                {0111 = LOCO}
28: tir := lshift(ir + ir) ; if n then goto 40 ;     {10xx o 11xx?}
29: tir := lshift(tir) ; if n then goto 35 ;         {100x o 101x?}
```

MAR	MBR	PC	AC	IR	TIR	ALU	SHR	N	Z
3F8	10 09	3F9	00 41	10 09		10 09	10 09	0	

Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxx			]]
0011xxxx			]
0100xxxx			hen pc:=x
0101xxxx			en pc:=x
0110xxxx			
0111xxxx			x<=4095)
1000xxxx			]
1001xxxx			c
1010xxxx			p+x]
1011xxxx			p+x]

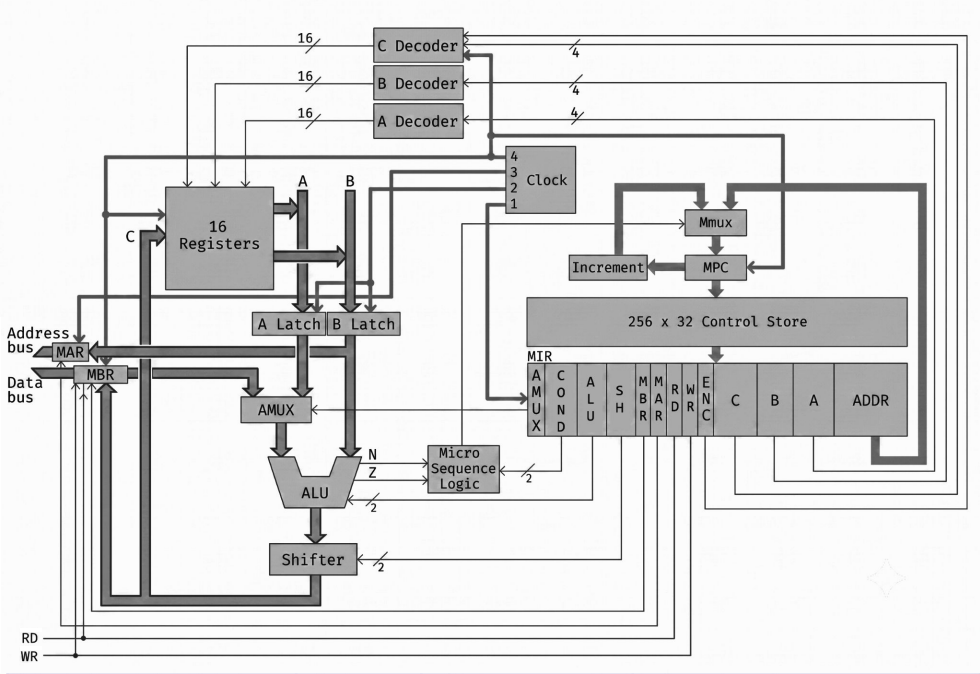
STORE DIRECT

3F8 **STOD** 9

Guardar el valor del AC en la memoria(9)

- mar:=ir
- mbr:=ac
- wr

0001 0000 0000 1001



```
0: mar := pc ; rd ;                                  {ciclo principal}
1: pc := pc + 1 ; rd ;                               {incrementa pc}
2: ir := mbr ; if n then goto 28 ;                   {salva y decodifica mbr}
3: tir := lshift(ir + ir) ; if n then goto 19 ;
4: tir := lshift(tir) ; if n then goto 11 ;          {000x o 001x?}
5: alu := tir ; if n then goto 9 ;                   {0000 o 0001?}
6: mar := ir ; rd ;                                  {0000 = LODD}
7: rd ;
8: ac := mbr ; goto 0 ;
9: mar := ir ; mbr := ac ; wr ;                      {0001 = STOD}
10: wr ; goto 0 ;
11: alu := tir ; if n then goto 15 ;                 {0010 o 0011?}
12: mar := ir ; rd ;                                 {0010 = ADDD}
13: rd ;
14: ac := mbr + ac ; goto 0 ;
15: mar := ir ; rd ;                                 {0011 = SUBD}
16: ac := ac + 1 ; rd ;                              {Nota: x - y = x + 1 + no y}
17: a := inv(mbr) ;
18: ac := ac + a ; goto 0 ;
19: tir := lshift(tir) ; if n then goto 25 ;         {010x o 011x?}
20: alu := tir ; if n then goto 23 ;                 {0100 o 0101?}
21: alu := ac ; if n then goto 0 ;                   {0100 = JPOS}
22: pc := band(ir , amask) ; goto 0 ;                {realiza el salto}
23: alu := ac ; if z then goto 22 ;                  {0101 = JZER}
24: goto 0 ;                                         {no se produce el salto}
25: alu := tir ; if n then goto 27 ;                 {0110 o 0111?}
26: pc := band(ir , amask) ; goto 0 ;                {0110 = JUMP}
27: ac := band(ir , amask) ; goto 0 ;                {0111 = LOCO}
28: tir := lshift(ir + ir) ; if n then goto 40 ;     {10xx o 11xx?}
29: tir := lshift(tir) ; if n then goto 35 ;         {100x o 101x?}
```

MAR	MBR	PC	AC	IR	TIR	ALU	SHR	N	Z
3F8	10 09	3F9	00 41	10 09	40 24	20 12	40 24	0	

Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxx			
0011xxxx			]
0100xxxx			hen pc:=x
0101xxxx			en pc:=x
0110xxxx			
0111xxxx			x<=4095)
1000xxxx			]
1001xxxx			c
1010xxxx			p+x]
1011xxxx			p+x]

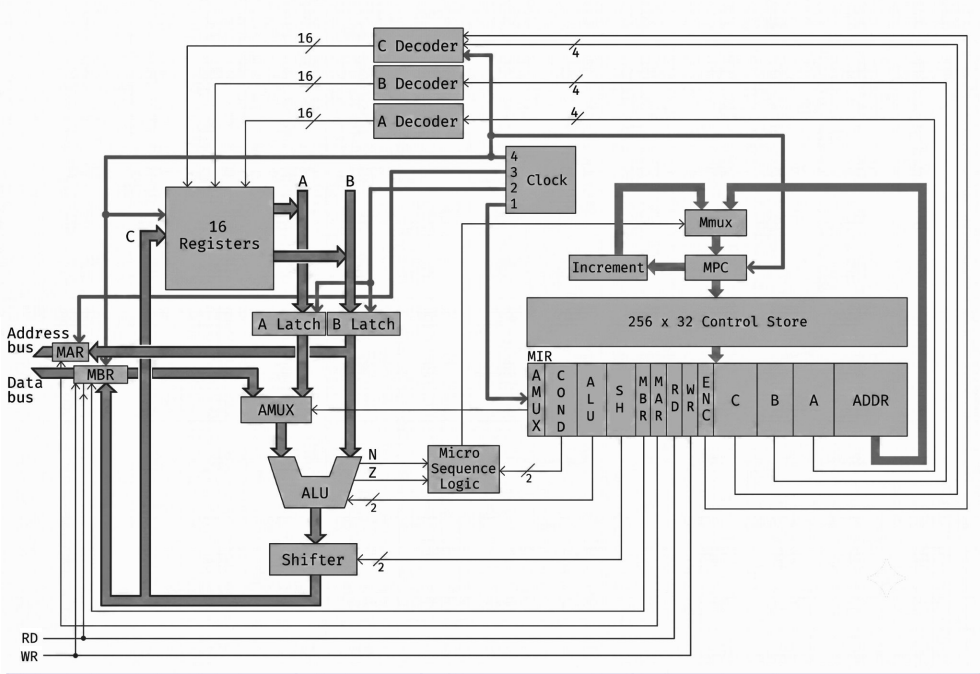
STORE DIRECT

3F8 **STOD** **9**

Guardar el valor del AC en la memoria(9)

- mar:=ir
- mbr:=ac
- wr

0001 0000 0000 1001



```
0: mar := pc ; rd ;                                  {ciclo principal}
1: pc := pc + 1 ; rd ;                               {incrementa pc}
2: ir := mbr ; if n then goto 28 ;                   {salva y decodifica mbr}
3: tir := lshift(ir + ir) ; if n then goto 19 ;
4: tir := lshift(tir) ; if n then goto 11 ;          {000x o 001x?}
5: alu := tir ; if n then goto 9 ;                   {0000 o 0001?}
6: mar := ir ; rd ;                                  {0000 = LODD}
7: rd ;
8: ac := mbr ; goto 0 ;
9: mar := ir ; mbr := ac ; wr ;                      {0001 = STOD}
10: wr ; goto 0 ;
11: alu := tir ; if n then goto 15 ;                 {0010 o 0011?}
12: mar := ir ; rd ;                                 {0010 = ADDD}
13: rd ;
14: ac := mbr + ac ; goto 0 ;
15: mar := ir ; rd ;                                 {0011 = SUBD}
16: ac := ac + 1 ; rd ;                              {Nota: x - y = x + 1 + no y}
17: a := inv(mbr) ;
18: ac := ac + a ; goto 0 ;
19: tir := lshift(tir) ; if n then goto 25 ;         {010x o 011x?}
20: alu := tir ; if n then goto 23 ;                 {0100 o 0101?}
21: alu := ac ; if n then goto 0 ;                   {0100 = JPOS}
22: pc := band(ir , amask) ; goto 0 ;                {realiza el salto}
23: alu := ac ; if z then goto 22 ;                  {0101 = JZER}
24: goto 0 ;                                         {no se produce el salto}
25: alu := tir ; if n then goto 27 ;                 {0110 o 0111?}
26: pc := band(ir , amask) ; goto 0 ;                {0110 = JUMP}
27: ac := band(ir , amask) ; goto 0 ;                {0111 = LOCO}
28: tir := lshift(ir + ir) ; if n then goto 40 ;     {10xx o 11xx?}
29: tir := lshift(tir) ; if n then goto 35 ;         {100x o 101x?}
```

MAR	MBR	PC	AC	IR	TIR	ALU	SHR	N	Z
3F8	10 09	3F9	00 41	10 09	80 48	40 24	80 48	0	

Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxx			
0011xxxx			]
0100xxxx			hen pc:=x
0101xxxx			en pc:=x
0110xxxx			
0111xxxx			x<=4095)
1000xxxx			]
1001xxxx			c
1010xxxx			p+x]
1011xxxx			p+x]

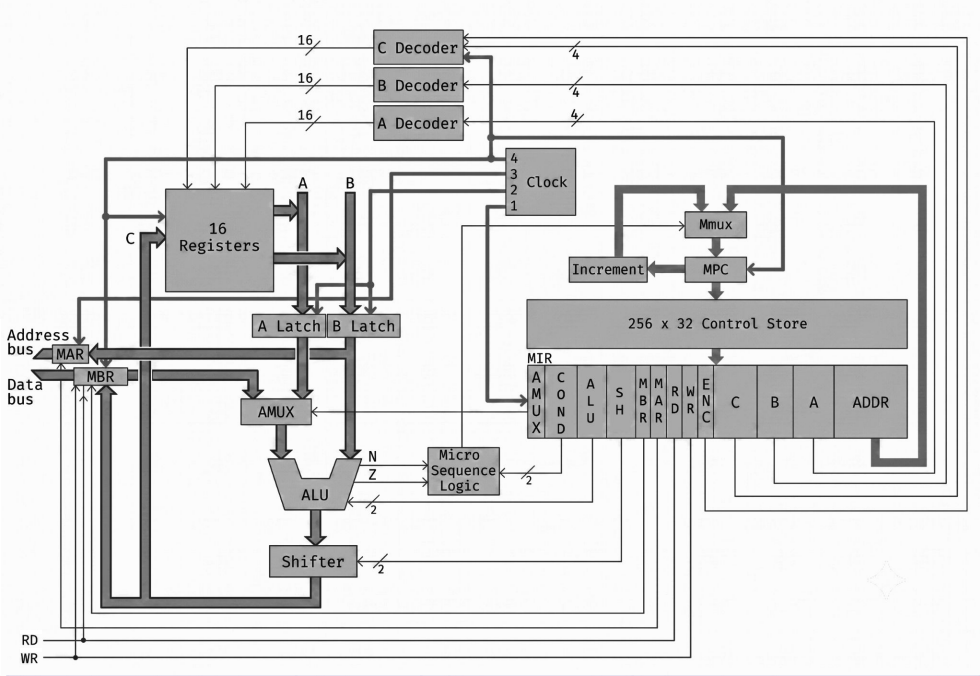
STORE DIRECT

3F8 **STOD** 9

Guardar el valor del AC en la memoria(9)

- mar:=ir
- mbr:=ac
- wr

0001 0000 0000 1001



```
0: mar := pc ; rd ;                                  {ciclo principal}
1: pc := pc + 1 ; rd ;                               {incrementa pc}
2: ir := mbr ; if n then goto 28 ;                   {salva y decodifica mbr}
3: tir := lshift(ir + ir) ; if n then goto 19 ;
4: tir := lshift(tir) ; if n then goto 11 ;          {000x o 001x?}
5: alu := tir ; if n then goto 9 ;                   {0000 o 0001?}
6: mar := ir ; rd ;                                  {0000 = LODD}
7: rd ;
8: ac := mbr ; goto 0 ;
9: mar := ir ; mbr := ac ; wr ;                      {0001 = STOD}
10: wr ; goto 0 ;
11: alu := tir ; if n then goto 15 ;                 {0010 o 0011?}
12: mar := ir ; rd ;                                 {0010 = ADDD}
13: rd ;
14: ac := mbr + ac ; goto 0 ;
15: mar := ir ; rd ;                                 {0011 = SUBD}
16: ac := ac + 1 ; rd ;                              {Nota: x - y = x + 1 + no y}
17: a := inv(mbr) ;
18: ac := ac + a ; goto 0 ;
19: tir := lshift(tir) ; if n then goto 25 ;         {010x o 011x?}
20: alu := tir ; if n then goto 23 ;                 {0100 o 0101?}
21: alu := ac ; if n then goto 0 ;                   {0100 = JPOS}
22: pc := band(ir , amask) ; goto 0 ;                {realiza el salto}
23: alu := ac ; if z then goto 22 ;                  {0101 = JZER}
24: goto 0 ;                                         {no se produce el salto}
25: alu := tir ; if n then goto 27 ;                 {0110 o 0111?}
26: pc := band(ir , amask) ; goto 0 ;                {0110 = JUMP}
27: ac := band(ir , amask) ; goto 0 ;                {0111 = LOCO}
28: tir := lshift(ir + ir) ; if n then goto 40 ;     {10xx o 11xx?}
29: tir := lshift(tir) ; if n then goto 35 ;         {100x o 101x?}
```

MAR	MBR	PC	AC	IR	TIR	ALU	SHR	N	Z
3F8	10 09	3F9	00 41	10 09	80 48	80 48	80 48	1	

Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxx			
0011xxxx			]
0100xxxx			hen pc:=x
0101xxxx			en pc:=x
0110xxxx			
0111xxxx			x<=4095)
1000xxxx			]
1001xxxx			c
1010xxxx			p+x]
1011xxxx			p+x]

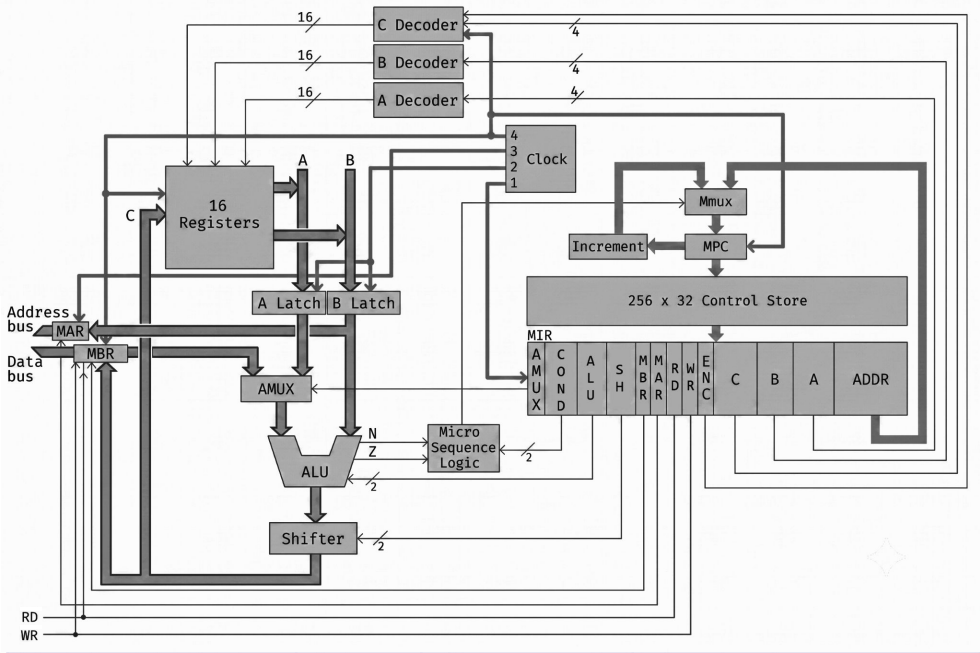
STORE DIRECT

3F8 **STOD** 9

Guardar el valor del AC en la memoria(9)

- mar:=ir
- mbr:=ac
- wr

0001 0000 0000 1001



```
0: mar := pc ; rd ;                                  {ciclo principal}
1: pc := pc + 1 ; rd ;                               {incrementa pc}
2: ir := mbr ; if n then goto 28 ;                   {salva y decodifica mbr}
3: tir := lshift(ir + ir) ; if n then goto 19 ;
4: tir := lshift(tir) ; if n then goto 11 ;          {000x o 001x?}
5: alu := tir ; if n then goto 9 ;                   {0000 o 0001?}
6: mar := ir ; rd ;                                  {0000 = LODD}
7: rd ;
8: ac := mbr ; goto 0 ;
9: mar := ir ; mbr := ac ; wr ;                      {0001 = STOD}
10: wr ; goto 0 ;
11: alu := tir ; if n then goto 15 ;                 {0010 o 0011?}
12: mar := ir ; rd ;                                 {0010 = ADDD}
13: rd ;
14: ac := mbr + ac ; goto 0 ;
15: mar := ir ; rd ;                                 {0011 = SUBD}
16: ac := ac + 1 ; rd ;                              {Nota: x - y = x + 1 + no y}
17: a := inv(mbr) ;
18: ac := ac + a ; goto 0 ;
19: tir := lshift(tir) ; if n then goto 25 ;         {010x o 011x?}
20: alu := tir ; if n then goto 23 ;                 {0100 o 0101?}
21: alu := ac ; if n then goto 0 ;                   {0100 = JPOS}
22: pc := band(ir , amask) ; goto 0 ;                {realiza el salto}
23: alu := ac ; if z then goto 22 ;                  {0101 = JZER}
24: goto 0 ;                                         {no se produce el salto}
25: alu := tir ; if n then goto 27 ;                 {0110 o 0111?}
26: pc := band(ir , amask) ; goto 0 ;                {0110 = JUMP}
27: ac := band(ir , amask) ; goto 0 ;                {0111 = LOCO}
28: tir := lshift(ir + ir) ; if n then goto 40 ;     {10xx o 11xx?}
29: tir := lshift(tir) ; if n then goto 35 ;         {100x o 101x?}
```

MAR	MBR	PC	AC	IR	TIR	ALU	SHR	N	Z
009	00 41	3F9	00 41	10 09	80 48	00 41	00 41	0	

Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxx			]]
0011xxxx			]
0100xxxx			hen pc:=x
0101xxxx			en pc:=x
0110xxxx			
0111xxxx			x<=4095)
1000xxxx			]
1001xxxx			c
1010xxxx			p+x]
1011xxxx			p+x]

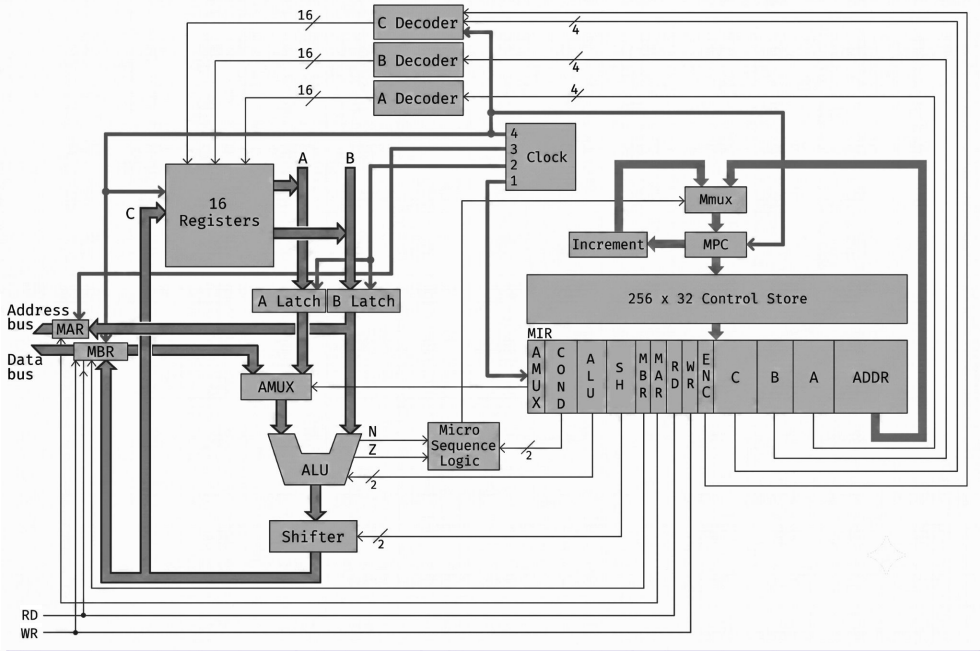
STORE DIRECT

3F8 **STOD** 9

Guardar el valor del AC en la memoria(9)

- mar:=ir
- mbr:=ac
- wr

0001 0000 0000 1001



```
0: mar := pc ; rd ;                                  {ciclo principal}
1: pc := pc + 1 ; rd ;                               {incrementa pc}
2: ir := mbr ; if n then goto 28 ;                   {salva y decodifica mbr}
3: tir := lshift(ir + ir) ; if n then goto 19 ;
4: tir := lshift(tir) ; if n then goto 11 ;          {000x o 001x?}
5: alu := tir ; if n then goto 9 ;                   {0000 o 0001?}
6: mar := ir ; rd ;                                  {0000 = LODD}
7: rd ;
8: ac := mbr ; goto 0 ;
9: mar := ir ; mbr := ac ; wr ;                      {0001 = STOD}
10: wr ; goto 0 ;
11: alu := tir ; if n then goto 15 ;                 {0010 o 0011?}
12: mar := ir ; rd ;                                 {0010 = ADDD}
13: rd ;
14: ac := mbr + ac ; goto 0 ;
15: mar := ir ; rd ;                                 {0011 = SUBD}
16: ac := ac + 1 ; rd ;                              {Nota: x - y = x + 1 + no y}
17: a := inv(mbr) ;
18: ac := ac + a ; goto 0 ;
19: tir := lshift(tir) ; if n then goto 25 ;         {010x o 011x?}
20: alu := tir ; if n then goto 23 ;                 {0100 o 0101?}
21: alu := ac ; if n then goto 0 ;                   {0100 = JPOS}
22: pc := band(ir , amask) ; goto 0 ;                {realiza el salto}
23: alu := ac ; if z then goto 22 ;                  {0101 = JZER}
24: goto 0 ;                                         {no se produce el salto}
25: alu := tir ; if n then goto 27 ;                 {0110 o 0111?}
26: pc := band(ir , amask) ; goto 0 ;                {0110 = JUMP}
27: ac := band(ir , amask) ; goto 0 ;                {0111 = LOCO}
28: tir := lshift(ir + ir) ; if n then goto 40 ;     {10xx o 11xx?}
29: tir := lshift(tir) ; if n then goto 35 ;         {100x o 101x?}
```

MAR	MBR	PC	AC	IR	TIR	ALU	SHR	N	Z
009	00 41	3F9	00 41	10 09	80 48	00 41	00 41	**0**	

Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxx			]]
0011xxxx			]
0100xxxx			hen pc:=x
0101xxxx			en pc:=x
0110xxxx			
0111xxxx			x<=4095)
1000xxxx			]
1001xxxx			c
1010xxxx			p+x]
1011xxxx			p+x]

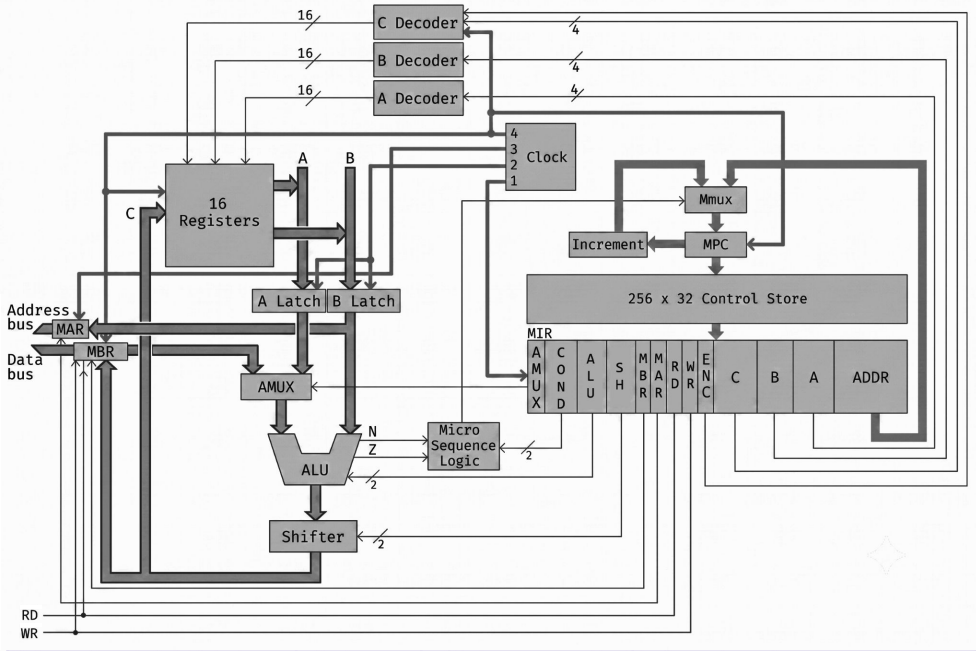
STORE DIRECT

3F8 **STOD** **9**

Guardar el valor del AC en la memoria(9)

- mar:=ir
- mbr:=ac
- wr

0001 0000 0000 1001



```
0: mar := pc ; rd ;                                  {ciclo principal}
1: pc := pc + 1 ; rd ;                               {incrementa pc}
2: ir := mbr ; if n then goto 28 ;                   {salva y decodifica mbr}
3: tir := lshift(ir + ir) ; if n then goto 19 ;
4: tir := lshift(tir) ; if n then goto 11 ;          {000x o 001x?}
5: alu := tir ; if n then goto 9 ;                   {0000 o 0001?}
6: mar := ir ; rd ;                                  {0000 = LODD}
7: rd ;
8: ac := mbr ; goto 0 ;
9: mar := ir ; mbr := ac ; wr ;                      {0001 = STOD}
10: wr ; goto 0 ;
11: alu := tir ; if n then goto 15 ;                 {0010 o 0011?}
12: mar := ir ; rd ;                                 {0010 = ADDD}
13: rd ;
14: ac := mbr + ac ; goto 0 ;
15: mar := ir ; rd ;                                 {0011 = SUBD}
16: ac := ac + 1 ; rd ;                              {Nota: x - y = x + 1 + no y}
17: a := inv(mbr) ;
18: ac := ac + a ; goto 0 ;
19: tir := lshift(tir) ; if n then goto 25 ;         {010x o 011x?}
20: alu := tir ; if n then goto 23 ;                 {0100 o 0101?}
21: alu := ac ; if n then goto 0 ;                   {0100 = JPOS}
22: pc := band(ir , amask) ; goto 0 ;                {realiza el salto}
23: alu := ac ; if z then goto 22 ;                  {0101 = JZER}
24: goto 0 ;                                         {no se produce el salto}
25: alu := tir ; if n then goto 27 ;                 {0110 o 0111?}
26: pc := band(ir , amask) ; goto 0 ;                {0110 = JUMP}
27: ac := band(ir , amask) ; goto 0 ;                {0111 = LOCO}
28: tir := lshift(ir + ir) ; if n then goto 40 ;     {10xx o 11xx?}
29: tir := lshift(tir) ; if n then goto 35 ;         {100x o 101x?}
```

MAR	MBR	PC	AC	IR	TIR	ALU	SHR	N	Z
3F9	00 41	3F9	00 41	10 09	80 48	00 41	00 41	**0**	

Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxx			]
0011xxxx			]
0100xxxx			hen pc:=x
0101xxxx			en pc:=x
0110xxxx			
0111xxxx			x<=4095)
1000xxxx			]
1001xxxx			c
1010xxxx			p+x]
1011xxxx			p+x]

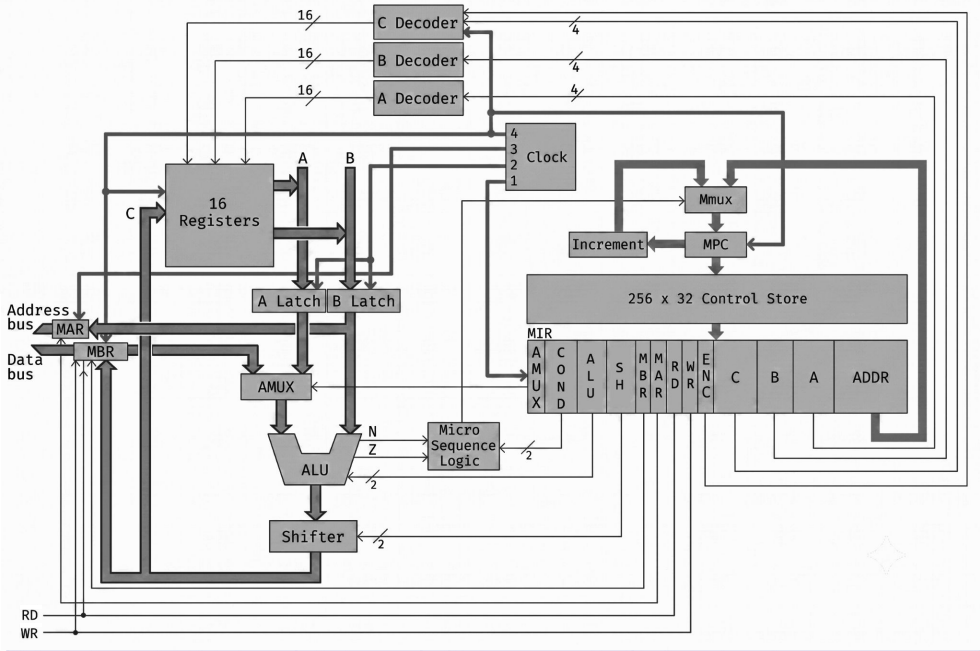
STORE DIRECT

3F8 **STOD** **9**

Guardar el valor del AC en la memoria(9)

- mar:=ir
- mbr:=ac
- wr

0001 0000 0000 1001



```
0: mar := pc ; rd ;                                  {ciclo principal}
1: pc := pc + 1 ; rd ;                               {incrementa pc}
2: ir := mbr ; if n then goto 28 ;                   {salva y decodifica mbr}
3: tir := lshift(ir + ir) ; if n then goto 19 ;
4: tir := lshift(tir) ; if n then goto 11 ;          {000x o 001x?}
5: alu := tir ; if n then goto 9 ;                   {0000 o 0001?}
6: mar := ir ; rd ;                                  {0000 = LODD}
7: rd ;
8: ac := mbr ; goto 0 ;
9: mar := ir ; mbr := ac ; wr ;                      {0001 = STOD}
10: wr ; goto 0 ;
11: alu := tir ; if n then goto 15 ;                 {0010 o 0011?}
12: mar := ir ; rd ;                                 {0010 = ADDD}
13: rd ;
14: ac := mbr + ac ; goto 0 ;
15: mar := ir ; rd ;                                 {0011 = SUBD}
16: ac := ac + 1 ; rd ;                              {Nota: x - y = x + 1 + no y}
17: a := inv(mbr) ;
18: ac := ac + a ; goto 0 ;
19: tir := lshift(tir) ; if n then goto 25 ;         {010x o 011x?}
20: alu := tir ; if n then goto 23 ;                 {0100 o 0101?}
21: alu := ac ; if n then goto 0 ;                   {0100 = JPOS}
22: pc := band(ir , amask) ; goto 0 ;                {realiza el salto}
23: alu := ac ; if z then goto 22 ;                  {0101 = JZER}
24: goto 0 ;                                         {no se produce el salto}
25: alu := tir ; if n then goto 27 ;                 {0110 o 0111?}
26: pc := band(ir , amask) ; goto 0 ;                {0110 = JUMP}
27: ac := band(ir , amask) ; goto 0 ;                {0111 = LOCO}
28: tir := lshift(ir + ir) ; if n then goto 40 ;     {10xx o 11xx?}
29: tir := lshift(tir) ; if n then goto 35 ;         {100x o 101x?}
```

MAR	MBR	PC	AC	IR	TIR	ALU	SHR	N	Z
3F9	63 F7	3FA	00 41	10 09	80 48	00 41	00 41	**0**	

Binario	Mnem	Instrucción	Significado
0101xxxxxxxxxxxx	JZER	Jump zero	if ac=0 then pc:=x
0110xxxxxxxxxxxx	JUMP	Jump	pc:=x
0111xxxx			x<=4095)
1000xxxx			]
1001xxxx			c
1010xxxx			p+x]
1011xxxx			p+x]
1100xxxx			en pc:=x
1101xxxx			hen pc:=x
1110xxxx			m[sp]:=pc;
11110000			m[sp]:=m[ac]
11110010	POP		p]; sp:=sp+1

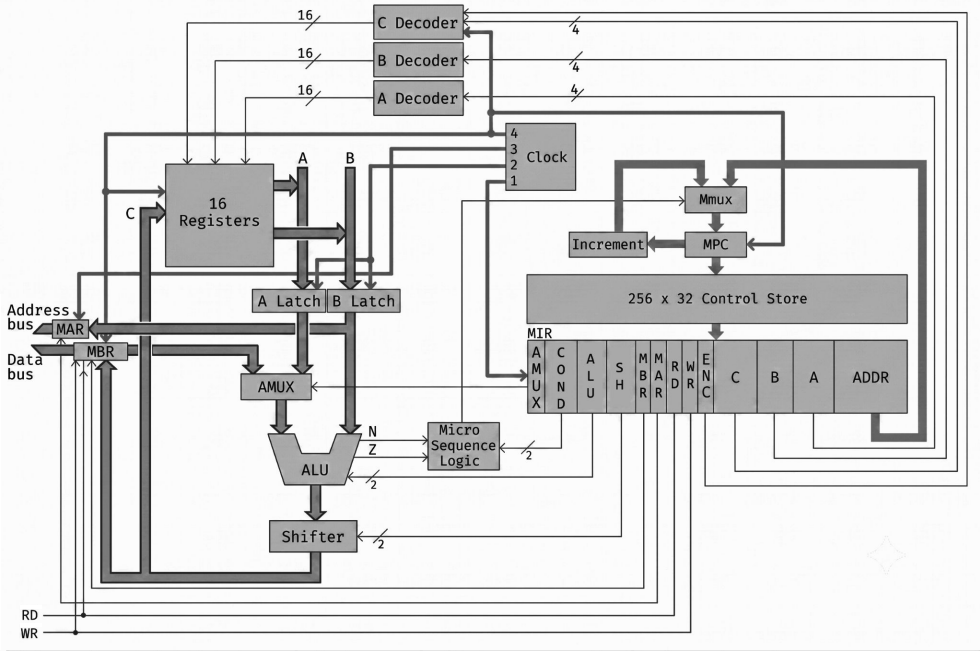
JUMP

3F9 **JUMP** **3F7**

Salto incondicional

- pc:=band(ir, amask)

0110 0011 1111 0111



```
0: mar := pc ; rd ;                                  {ciclo principal}
1: pc := pc + 1 ; rd ;                               {incrementa pc}
2: ir := mbr ; if n then goto 28 ;                   {salva y decodifica mbr}
3: tir := lshift(ir + ir) ; if n then goto 19 ;
4: tir := lshift(tir) ; if n then goto 11 ;          {000x o 001x?}
5: alu := tir ; if n then goto 9 ;                   {0000 o 0001?}
6: mar := ir ; rd ;                                  {0000 = LODD}
7: rd ;
8: ac := mbr ; goto 0 ;
9: mar := ir ; mbr := ac ; wr ;                      {0001 = STOD}
10: wr ; goto 0 ;
11: alu := tir ; if n then goto 15 ;                 {0010 o 0011?}
12: mar := ir ; rd ;                                 {0010 = ADDD}
13: rd ;
14: ac := mbr + ac ; goto 0 ;
15: mar := ir ; rd ;                                 {0011 = SUBD}
16: ac := ac + 1 ; rd ;                              {Nota: x - y = x + 1 + no y}
17: a := inv(mbr) ;
18: ac := ac + a ; goto 0 ;
19: tir := lshift(tir) ; if n then goto 25 ;         {010x o 011x?}
20: alu := tir ; if n then goto 23 ;                 {0100 o 0101?}
21: alu := ac ; if n then goto 0 ;                   {0100 = JPOS}
22: pc := band(ir , amask) ; goto 0 ;                {realiza el salto}
23: alu := ac ; if z then goto 22 ;                  {0101 = JZER}
24: goto 0 ;                                         {no se produce el salto}
25: alu := tir ; if n then goto 27 ;                 {0110 o 0111?}
26: pc := band(ir , amask) ; goto 0 ;                {0110 = JUMP}
27: ac := band(ir , amask) ; goto 0 ;                {0111 = LOCO}
28: tir := lshift(ir + ir) ; if n then goto 40 ;     {10xx o 11xx?}
29: tir := lshift(tir) ; if n then goto 35 ;         {100x o 101x?}
```

MAR	MBR	PC	AC	IR	TIR	ALU	SHR	N	Z
3F9	63 F7	3FA	00 41	63 F7	80 48	63 F7	63 F7	0	

Binario	Mnem	Instrucción	Significado
0101xxxxxxxxxxxx	JZER	Jump zero	if ac=0 then pc:=x
0110xxxxxxxxxxxx	JUMP	Jump	pc:=x
0111xxxx			x<=4095)
1000xxxx			]
1001xxxx			c
1010xxxx			p+x]
1011xxxx			p+x]
1100xxxx			en pc:=x
1101xxxx			hen pc:=x
1110xxxx			m[sp]:=pc;
11110000			m[sp]:=m[ac]
11110010	POP		p]; sp:=sp+1

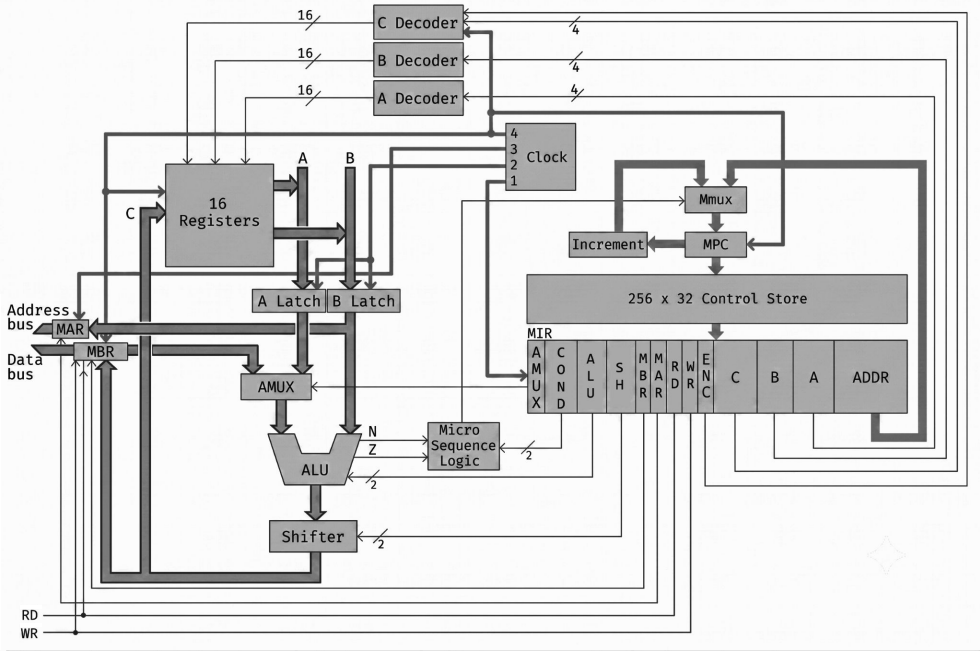
JUMP

3F9 **JUMP** **3F7**

Salto incondicional

- pc:=band(ir, amask)

0110 0011 1111 0111



```
0: mar := pc ; rd ;                                  {ciclo principal}
1: pc := pc + 1 ; rd ;                               {incrementa pc}
2: ir := mbr ; if n then goto 28 ;                   {salva y decodifica mbr}
3: tir := lshift(ir + ir) ; if n then goto 19 ;
4: tir := lshift(tir) ; if n then goto 11 ;          {000x o 001x?}
5: alu := tir ; if n then goto 9 ;                   {0000 o 0001?}
6: mar := ir ; rd ;                                  {0000 = LODD}
7: rd ;
8: ac := mbr ; goto 0 ;
9: mar := ir ; mbr := ac ; wr ;                      {0001 = STOD}
10: wr ; goto 0 ;
11: alu := tir ; if n then goto 15 ;                 {0010 o 0011?}
12: mar := ir ; rd ;                                 {0010 = ADDD}
13: rd ;
14: ac := mbr + ac ; goto 0 ;
15: mar := ir ; rd ;                                 {0011 = SUBD}
16: ac := ac + 1 ; rd ;                              {Nota: x - y = x + 1 + no y}
17: a := inv(mbr) ;
18: ac := ac + a ; goto 0 ;
19: tir := lshift(tir) ; if n then goto 25 ;         {010x o 011x?}
20: alu := tir ; if n then goto 23 ;                 {0100 o 0101?}
21: alu := ac ; if n then goto 0 ;                   {0100 = JPOS}
22: pc := band(ir , amask) ; goto 0 ;                {realiza el salto}
23: alu := ac ; if z then goto 22 ;                  {0101 = JZER}
24: goto 0 ;                                         {no se produce el salto}
25: alu := tir ; if n then goto 27 ;                 {0110 o 0111?}
26: pc := band(ir , amask) ; goto 0 ;                {0110 = JUMP}
27: ac := band(ir , amask) ; goto 0 ;                {0111 = LOCO}
28: tir := lshift(ir + ir) ; if n then goto 40 ;     {10xx o 11xx?}
29: tir := lshift(tir) ; if n then goto 35 ;         {100x o 101x?}
```

MAR	MBR	PC	AC	IR	TIR	ALU	SHR	N	Z
3F9	63 F7	3FA	00 41	63 F7	8F DC	C7 EE	8F DC	1	

Binario	Mnem	Instrucción	Significado
0101xxxxxxxxxxxx	JZER	Jump zero	if ac=0 then pc:=x
0110xxxxxxxxxxxx	JUMP	Jump	pc:=x
0111xxxx			(0<=x<=4095)
1000xxxx			]
1001xxxx			c
1010xxxx			p+x]
1011xxxx			p+x]
1100xxxx			en pc:=x
1101xxxx			hen pc:=x
1110xxxx			m[sp]:=pc;
11110000			m[sp]:=m[ac]
11110010	POP		p]; sp:=sp+1

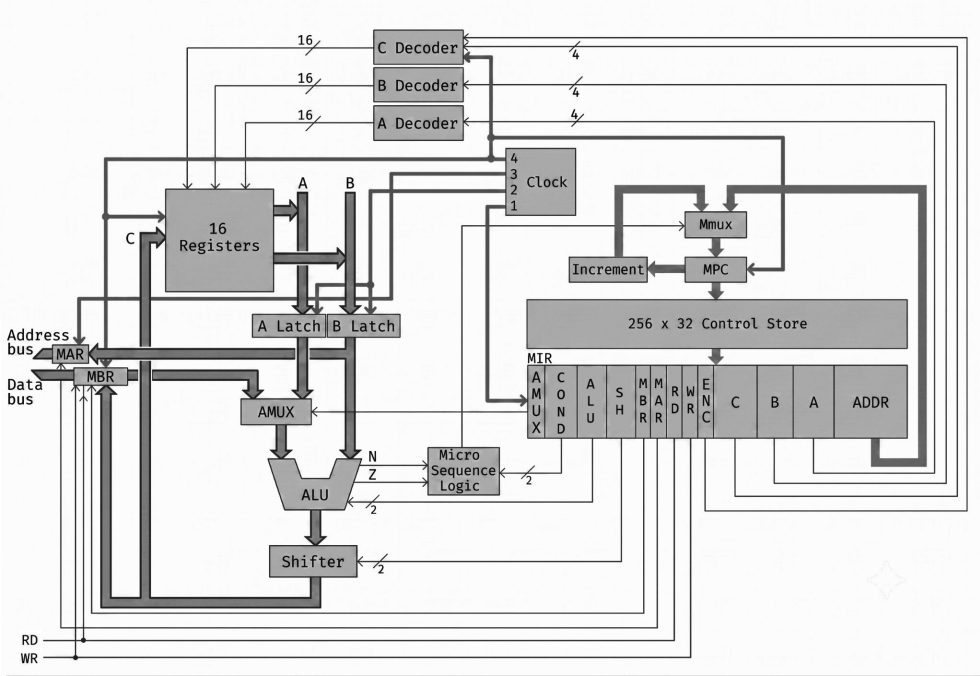
JUMP

3F9 **JUMP** **3F7**

Salto incondicional

- pc:=band(ir, amask)

0110 0011 1111 0111



```
0: mar := pc ; rd ;                                  {ciclo principal}
1: pc := pc + 1 ; rd ;                               {incrementa pc}
2: ir := mbr ; if n then goto 28 ;                   {salva y decodifica mbr}
3: tir := lshift(ir + ir) ; if n then goto 19 ;
4: tir := lshift(tir) ; if n then goto 11 ;          {000x o 001x?}
5: alu := tir ; if n then goto 9 ;                   {0000 o 0001?}
6: mar := ir ; rd ;                                  {0000 = LODD}
7: rd ;
8: ac := mbr ; goto 0 ;
9: mar := ir ; mbr := ac ; wr ;                      {0001 = STOD}
10: wr ; goto 0 ;
11: alu := tir ; if n then goto 15 ;                 {0010 o 0011?}
12: mar := ir ; rd ;                                 {0010 = ADDD}
13: rd ;
14: ac := mbr + ac ; goto 0 ;
15: mar := ir ; rd ;                                 {0011 = SUBD}
16: ac := ac + 1 ; rd ;                              {Nota: x - y = x + 1 + no y}
17: a := inv(mbr) ;
18: ac := ac + a ; goto 0 ;
19: tir := lshift(tir) ; if n then goto 25 ;         {010x o 011x?}
20: alu := tir ; if n then goto 23 ;                 {0100 o 0101?}
21: alu := ac ; if n then goto 0 ;                   {0100 = JPOS}
22: pc := band(ir , amask) ; goto 0 ;                {realiza el salto}
23: alu := ac ; if z then goto 22 ;                  {0101 = JZER}
24: goto 0 ;                                         {no se produce el salto}
25: alu := tir ; if n then goto 27 ;                 {0110 o 0111?}
26: pc := band(ir , amask) ; goto 0 ;                {0110 = JUMP}
27: ac := band(ir , amask) ; goto 0 ;                {0111 = LOCO}
28: tir := lshift(ir + ir) ; if n then goto 40 ;     {10xx o 11xx?}
29: tir := lshift(tir) ; if n then goto 35 ;         {100x o 101x?}
```

MAR	MBR	PC	AC	IR	TIR	ALU	SHR	N	Z
3F9	63 F7	3FA	00 41	63 F7	1F B8	8F DC	1F B8	1	

Binario	Mnem	Instrucción	Significado
0101xxxxxxxxxxxx	JZER	Jump zero	if ac=0 then pc:=x
0110xxxxxxxxxxxx	JUMP	Jump	pc:=x
0111xxxx			(0<=x<=4095)
1000xxxx			]
1001xxxx			c
1010xxxx			p+x]
1011xxxx			p+x]
1100xxxx			en pc:=x
1101xxxx			hen pc:=x
1110xxxx			m[sp]:=pc;
11110000			m[sp]:=m[ac]
11110010	POP	Pop indirect	p]; sp:=sp+1

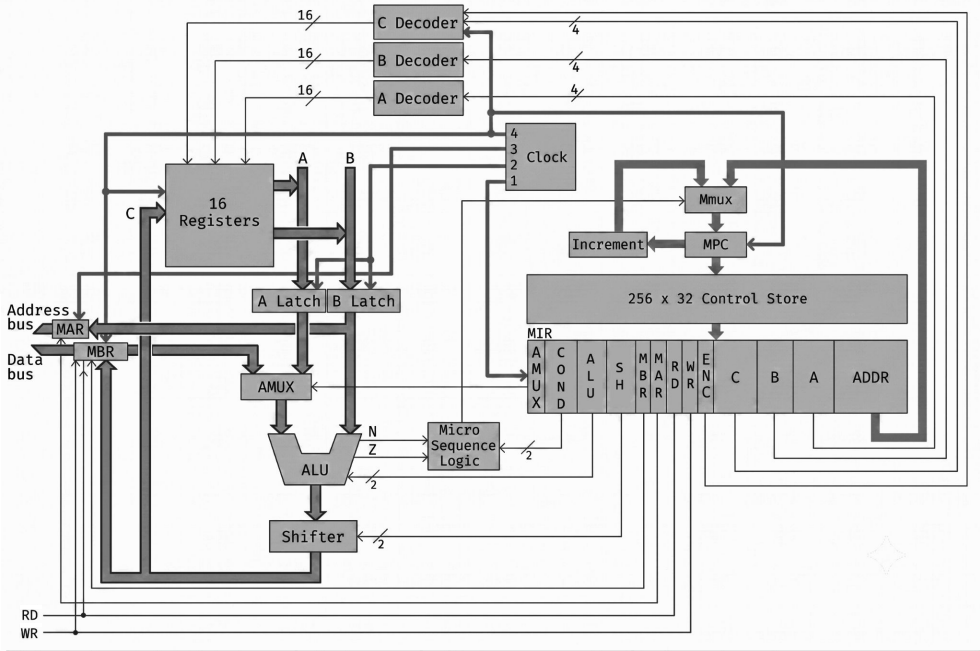
JUMP

3F9 **JUMP** **3F7**

Salto incondicional

- pc:=band(ir, amask)

0110 0011 1111 0111



```
0: mar := pc ; rd ;                                  {ciclo principal}
1: pc := pc + 1 ; rd ;                               {incrementa pc}
2: ir := mbr ; if n then goto 28 ;                   {salva y decodifica mbr}
3: tir := lshift(ir + ir) ; if n then goto 19 ;
4: tir := lshift(tir) ; if n then goto 11 ;          {000x o 001x?}
5: alu := tir ; if n then goto 9 ;                   {0000 o 0001?}
6: mar := ir ; rd ;                                  {0000 = LODD}
7: rd ;
8: ac := mbr ; goto 0 ;
9: mar := ir ; mbr := ac ; wr ;                      {0001 = STOD}
10: wr ; goto 0 ;
11: alu := tir ; if n then goto 15 ;                 {0010 o 0011?}
12: mar := ir ; rd ;                                 {0010 = ADDD}
13: rd ;
14: ac := mbr + ac ; goto 0 ;
15: mar := ir ; rd ;                                 {0011 = SUBD}
16: ac := ac + 1 ; rd ;                              {Nota: x - y = x + 1 + no y}
17: a := inv(mbr) ;
18: ac := ac + a ; goto 0 ;
19: tir := lshift(tir) ; if n then goto 25 ;         {010x o 011x?}
20: alu := tir ; if n then goto 23 ;                 {0100 o 0101?}
21: alu := ac ; if n then goto 0 ;                   {0100 = JPOS}
22: pc := band(ir , amask) ; goto 0 ;                {realiza el salto}
23: alu := ac ; if z then goto 22 ;                  {0101 = JZER}
24: goto 0 ;                                         {no se produce el salto}
25: alu := tir ; if n then goto 27 ;                 {0110 o 0111?}
26: pc := band(ir , amask) ; goto 0 ;                {0110 = JUMP}
27: ac := band(ir , amask) ; goto 0 ;                {0111 = LOCO}
28: tir := lshift(ir + ir) ; if n then goto 40 ;     {10xx o 11xx?}
29: tir := lshift(tir) ; if n then goto 35 ;         {100x o 101x?}
```

MAR	MBR	PC	AC	IR	TIR	ALU	SHR	N	Z
3F9	63 F7	3FA	00 41	63 F7	1F B8	1F B8	1F B8	0	

Binario	Mnem	Instrucción	Significado
0101xxxxxxxxxxxx	JZER	Jump zero	if ac=0 then pc:=x
0110xxxxxxxxxxxx	JUMP	Jump	pc:=x
0111xxxx			x<=4095)
1000xxxx			]
1001xxxx			c
1010xxxx			p+x]
1011xxxx			p+x]
1100xxxx			en pc:=x
1101xxxx			hen pc:=x
1110xxxx			m[sp]:=pc;
11110000			m[sp]:=m[ac]
11110010	POP		p]; sp:=sp+1

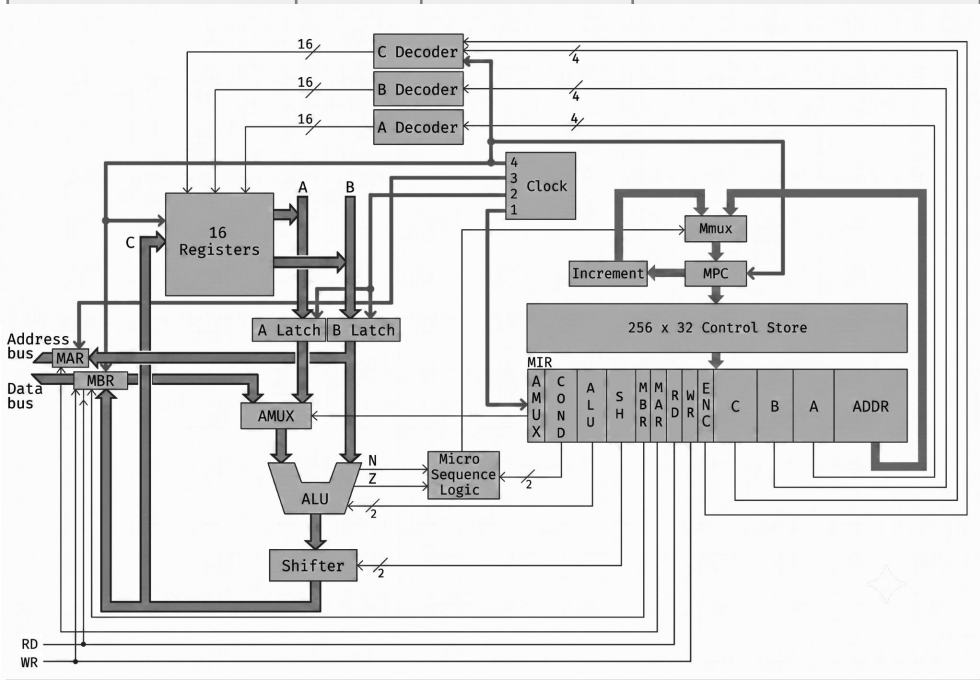
JUMP

3F9 **JUMP** **3F7**

Salto incondicional

- pc:=band(ir, amask)

0110 0011 1111 0111



```
0: mar := pc ; rd ;                                  {ciclo principal}
1: pc := pc + 1 ; rd ;                               {incrementa pc}
2: ir := mbr ; if n then goto 28 ;                   {salva y decodifica mbr}
3: tir := lshift(ir + ir) ; if n then goto 19 ;
4: tir := lshift(tir) ; if n then goto 11 ;          {000x o 001x?}
5: alu := tir ; if n then goto 9 ;                   {0000 o 0001?}
6: mar := ir ; rd ;                                  {0000 = LODD}
7: rd ;
8: ac := mbr ; goto 0 ;
9: mar := ir ; mbr := ac ; wr ;                      {0001 = STOD}
10: wr ; goto 0 ;
11: alu := tir ; if n then goto 15 ;                 {0010 o 0011?}
12: mar := ir ; rd ;                                 {0010 = ADDD}
13: rd ;
14: ac := mbr + ac ; goto 0 ;
15: mar := ir ; rd ;                                 {0011 = SUBD}
16: ac := ac + 1 ; rd ;                              {Nota: x - y = x + 1 + no y}
17: a := inv(mbr) ;
18: ac := ac + a ; goto 0 ;
19: tir := lshift(tir) ; if n then goto 25 ;         {010x o 011x?}
20: alu := tir ; if n then goto 23 ;                 {0100 o 0101?}
21: alu := ac ; if n then goto 0 ;                   {0100 = JPOS}
22: pc := band(ir , amask) ; goto 0 ;                {realiza el salto}
23: alu := ac ; if z then goto 22 ;                  {0101 = JZER}
24: goto 0 ;                                         {no se produce el salto}
25: alu := tir ; if n then goto 27 ;                 {0110 o 0111?}
26: pc := band(ir , amask) ; goto 0 ;                {0110 = JUMP}
27: ac := band(ir , amask) ; goto 0 ;                {0111 = LOCO}
28: tir := lshift(ir + ir) ; if n then goto 40 ;     {10xx o 11xx?}
29: tir := lshift(tir) ; if n then goto 35 ;         {100x o 101x?}
```

MAR	MBR	PC	AC	IR	TIR	ALU	SHR	N	Z
3F9	63 F7	3F7	00 41	63 F7	1F B8	03 F7	03 F7	0	

Binario	Mnem	Instrucción	Significado
0101xxxxxxxxxxxx	JZER	Jump zero	if ac=0 then pc:=x
0110xxxxxxxxxxxx	JUMP	Jump	pc:=x
0111xxxx			x<=4095)
1000xxxx			]
1001xxxx			c
1010xxxx			p+x]
1011xxxx			p+x]
1100xxxx			en pc:=x
1101xxxx			hen pc:=x
1110xxxx			m[sp]:=pc;
11110000			m[sp]:=m[ac]
11110010	POP		p]; sp:=sp+1

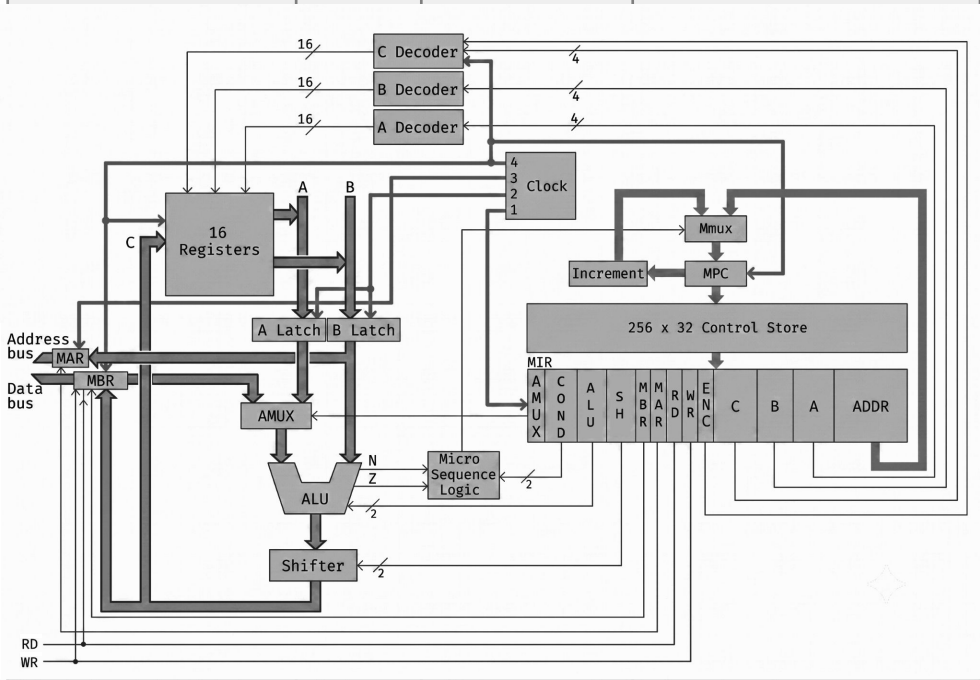
JUMP

3F9 **JUMP** **3F7**

Salto incondicional

- pc:=band(ir, amask)

0110 0011 1111 0111



```
0: mar := pc ; rd ;                                  {ciclo principal}
1: pc := pc + 1 ; rd ;                               {incrementa pc}
2: ir := mbr ; if n then goto 28 ;                   {salva y decodifica mbr}
3: tir := lshift(ir + ir) ; if n then goto 19 ;
4: tir := lshift(tir) ; if n then goto 11 ;          {000x o 001x?}
5: alu := tir ; if n then goto 9 ;                   {0000 o 0001?}
6: mar := ir ; rd ;                                  {0000 = LODD}
7: rd ;
8: ac := mbr ; goto 0 ;
9: mar := ir ; mbr := ac ; wr ;                      {0001 = STOD}
10: wr ; goto 0 ;
11: alu := tir ; if n then goto 15 ;                 {0010 o 0011?}
12: mar := ir ; rd ;                                 {0010 = ADDD}
13: rd ;
14: ac := mbr + ac ; goto 0 ;
15: mar := ir ; rd ;                                 {0011 = SUBD}
16: ac := ac + 1 ; rd ;                              {Nota: x - y = x + 1 + no y}
17: a := inv(mbr) ;
18: ac := ac + a ; goto 0 ;
19: tir := lshift(tir) ; if n then goto 25 ;         {010x o 011x?}
20: alu := tir ; if n then goto 23 ;                 {0100 o 0101?}
21: alu := ac ; if n then goto 0 ;                   {0100 = JPOS}
22: pc := band(ir , amask) ; goto 0 ;                {realiza el salto}
23: alu := ac ; if z then goto 22 ;                  {0101 = JZER}
24: goto 0 ;                                         {no se produce el salto}
25: alu := tir ; if n then goto 27 ;                 {0110 o 0111?}
26: pc := band(ir , amask) ; goto 0 ;                {0110 = JUMP}
27: ac := band(ir , amask) ; goto 0 ;                {0111 = LOCO}
28: tir := lshift(ir + ir) ; if n then goto 40 ;     {10xx o 11xx?}
29: tir := lshift(tir) ; if n then goto 35 ;         {100x o 101x?}
```

MAR	MBR	PC	AC	IR	TIR	ALU	SHR	N	Z
3F7	63 F7	3F7	00 41	63 F7	1F B8	03 F7	03 F7	**0**	

Binario	Mnem	Instrucción	Significado
0101xxxxxxxxxxxx	JZER	Jump zero	if ac=0 then pc:=x
0110xxxxxxxxxxxx	JUMP	Jump	pc:=x
0111xxxx			x<=4095)
1000xxxx			]
1001xxxx			c
1010xxxx			p+x]
1011xxxx			p+x]
1100xxxx			en pc:=x
1101xxxx			hen pc:=x
1110xxxx			m[sp]:=pc;
11110000			m[sp]:=m[ac]
11110010	POP		p]; sp:=sp+1

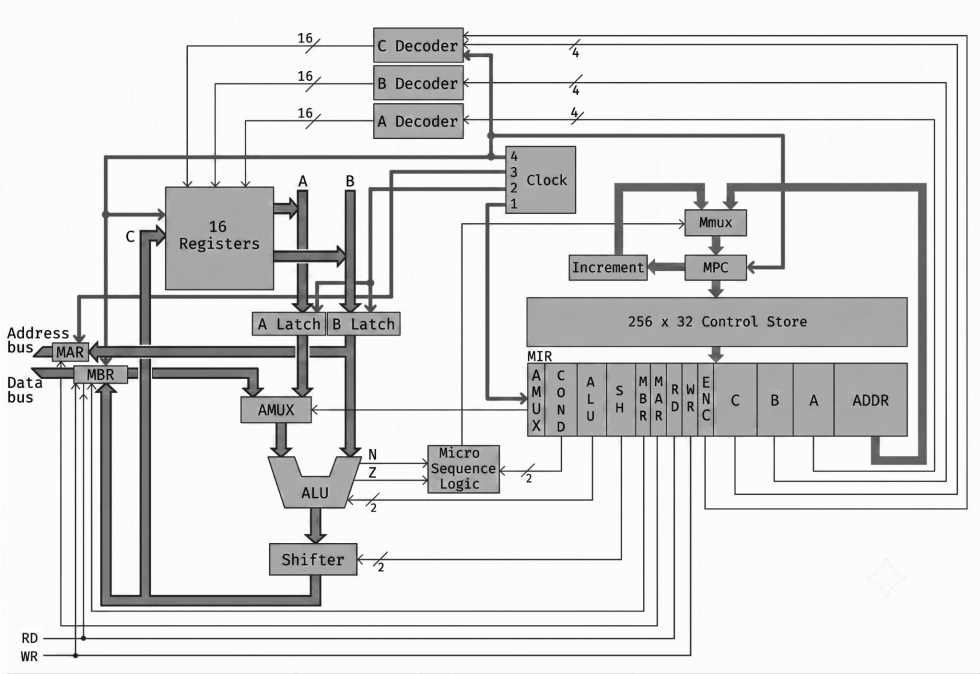
JUMP

3F9 **JUMP** **3F7**

Salto incondicional

- pc:=band(ir, amask)

0110 0011 1111 0111



```
0: mar := pc ; rd ;                          {ciclo principal}
1: pc := pc + 1 ; rd ;                       {incrementa pc}
2: ir := mbr ; if n then goto 28 ;           {salva y decodifica mbr}
3: tir := lshift(ir + ir) ; if n then goto 19 ;
4: tir := lshift(tir) ; if n then goto 11 ;  {000x o 001x?}
5: alu := tir ; if n then goto 9 ;           {0000 o 0001?}
6: mar := ir ; rd ;                          {0000 = LODD}
7: rd ;
8: ac := mbr ; goto 0 ;
9: mar := ir ; mbr := ac ; wr ;              {0001 = STOD}
10: wr ; goto 0 ;
11: alu := tir ; if n then goto 15 ;         {0010 o 0011?}
12: mar := ir ; rd ;                         {0010 = ADDD}
13: rd ;
14: ac := mbr + ac ; goto 0 ;
15: mar := ir ; rd ;                         {0011 = SUBD}
16: ac := ac + 1 ; rd ;                      {Nota: x = y = x + 1 + no y}
17: a := inv(mbr) ;
18: ac := ac + a ; goto 0 ;
19: tir := lshift(tir) ; if n then goto 25 ; {010x o 011x?}
20: alu := tir ; if n then goto 23 ;         {0100 o 0101?}
21: alu := ac ; if n then goto 0 ;           {0100 = JPOS}
22: pc := band(ir , amask) ; goto 0 ;        {realiza el salto}
23: alu := ac ; if z then goto 22 ;          {0101 = JZER}
24: goto 0 ;                                 {no se produce el salto}
25: alu := tir ; if n then goto 27 ;         {0110 o 0111?}
26: pc := band(ir , amask) ; goto 0 ;        {0110 = JUMP}
27: ac := band(ir , amask) ; goto 0 ;        {0111 = LOCO}
28: tir := lshift(ir + ir) ; if n then goto 40 ; {10xx o 11xx?}
29: tir := lshift(tir) ; if n then goto 35 ; {100x o 101x?}
```

MAR	MBR	PC	AC	IR	TIR	ALU	SHR	N	Z
3F7	63 F7	3F7	00 41	63 F7	1F B8	03 F7	03 F7	**0**	

Binario	Mnem	Instrucción	Significado
0101xxxxxxxxxxxx	JZER	Jump zero	if ac=0 then pc:=x
0110xxxxxxxxxxxx	JUMP	Jump	pc:=x
0111xxxx			(0<=x<=4095)
1000xxxx			]
1001xxxx			c
1010xxxx			p+x]
1011xxxx			p+x]
1100xxxx			en pc:=x
1101xxxx			hen pc:=x
1110xxxx			m[sp]:=pc;
11110000			m[sp]:=m[ac]
11110010	POP1		p]; sp:=sp+1

JUMP

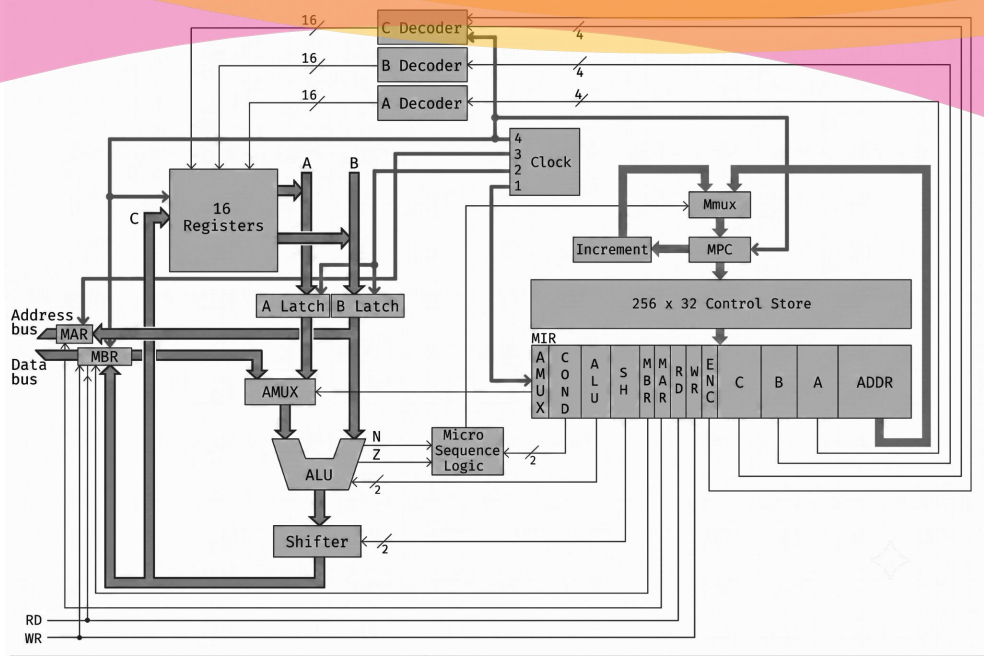
3F9 **JUMP** **3F7**

Salto incondicional

- pc:=band(ir, amask)

0110 0011 1111 0111

El ciclo se reinicia



Arquitectura Horizontal

Ejecución del microprograma

Fundamentos de Arquitectura de computadoras

El control del procesador. Análisis de la secuencias de pasos del micro programa para la ejecución de un programa de usuario.

- Control Store
- Micro instrucción
- Microprograma